

DOMAIN-DRIVEN DESIGN WITH LARAVEL

The only design approach you need

MARTIN JOO

Basic Concepts

- Domain-Driven Design
- Working With Data
- Value Objects
- Data Transfer Objects
- Repositories
- Custom Query Builders
- Services
- Actions
- ViewModels
- CQRS
- States And Transitions
- Domains And Applications
- Advantages And Disadvantages

Designing an E-mail Marketing Software

- Overview
 - Subscribers
 - Broadcasts
 - Sequences
 - Automations
 - Other Features
 - Why E-mail Marketing?

- User Stories

- Data Modeling
 - Subscribers
 - Broadcasts
 - SentMails
 - A Quick Note On Performance
 - Sequences
 - Automations

- Domains

Building an E-mail Marketing Software

- Setting Up Domains and Applications

- Subscribers
 - Creating A New Subscriber
 - Updating A Subscriber
 - View Models
 - Vue Component
 - Get Subscribers
 - The Real Power of DTOs and Actions
 - Conclusion

- Broadcasts
 - The Broadcast DTO
 - Handling Filters
 - Upserting A Broadcast
 - Filtering Subscribers
 - Sending A Broadcast
 - Calculating The Performance Of A Broadcast

Previewing A Broadcast

Get Broadcasts

Sequences

Creating A Sequence

Proceeding A Sequence

Refactoring

Updating The Subscriber's Status

Calculating The Performance Of A Sequence

Progress Of A Sequence

Dashboard And Reports

New subscriber counts

All-Time Performance

Subscribers

Automations

Upserting automations

Running automations

Conclusion

Thank You

Basic Concepts

Domain-Driven Design

First of all, we have to answer the most obvious question: what is Domain-Driven Design.

DDD is a software development approach that tries to bring the business language and the code as close together as possible. This is the most critical attribute of this approach. But for some reason, DDD is one of the most misunderstood and overcomplicated topics in the developer community, so I'll try to make it easy to understand.

DDD teaches us two main things:

- Strategic Design
- Technical Design

In my honest opinion, strategic design is way more important than the technical aspects. It's hard to summarize it in one cool sentence, but you will see what I mean in the rest of the book. For now, these are the essential pillars:

- Domains and namespaces. Later, I'll talk about what a domain is, but DDD teaches us to structure our code very expressive and logical.
- Choosing the proper names. For example, if the business refers to the users as "customers" or "employees," you should rename your User class to follow that convention.
- The classes and objects should express the intention behind them. In the most simple Laravel application, you have models and controllers. What do you think when you see a project with 50 models and 50 controllers? You can see the application's primary domain, but you have to dig deeper if you want to have a good understanding of the features, right? Now, what about 300 models and 500 controllers? You have no chance to reason about this kind of application.

In most projects, developers prefer technical terms over business concepts. That's natural. After all, we're technical people. But I have a question: are those technical terms significant?

Let me show you an example. This is a snippet from one of my applications I wrote on Oct 28, 2016, after finishing the Design Patterns book. Take a look at it (it's not Laravel):

```

class Search_View_Container_Factory_Project
{
    /**
     * @var Search_View_Container_Relation_Project
     */
    private static $_relationContainer;

    /**
     * @param array $data
     * @return Search_View_Container_Project
     */
    public static function createContainer(array $data)
    {
        if ($data['current'] == 'complex') {
            return self::createComplex($data);
        } else {
            return self::createSimple($data);
        }
    }

    /**
     * @param array $data
     * @return Search_View_Container_Project
     */
    private static function createSimple(array $data)
    {
        $container = new Search_View_Container_Project('simple');
        $container->setSearchTerm(Arr::get($data, 'search_term'));

        $relationContainer = new Search_View_Container_Relation_Project();
        $industryModel = new Model_Industry();
        $industries = $industryModel->getAll();

        foreach ($industries as $industry) {
            $industryItem = new Search_View_Container_Relation_Item(
                $industry,
            
```

```

        Search_View_Container_Relation_Item::TYPE_INDUSTRY,
        false
    );

    $relationContainer->addItem($industryItem,
Search_View_Container_Relation_Item::TYPE_INDUSTRY);
    }

    $container->setRelationContainer($relationContainer);
    return $container;
}

/**
 * @param array $models
 * @param int $type
 */
private static function addItem(array $models, $type)
{
    foreach ($models as $model) {
        $item = new Search_View_Container_Relation_Item($model, $type, true);
        self::$_relationContainer->addItem($item, $type);
    }
}
}

```

Today it's February 2, 2022. What do you think? After six years, do I have any clue what the heck is a `Search_View_Container_Relation_Item`? No, I have no idea what it is. I only know one thing for sure: **it does not help me**. This project is about freelancers and projects. This class does something with searching projects (I guess), but it does not reveal that intention. Did you ever hear a product manager saying: *wow, we got so much positive feedback on the Search View Container Factory Project feature?*

Maybe if I take a step back and look at the file structure, I have a better idea.

```
→ classes git:(master) tree ./Search
./Search
├── Complex
│   └── Project.php
├── Factory
│   └── Project.php
├── Relation
│   ├── Factory
│   │   └── Project.php
├── Simple
│   └── Project.php
└── View
    └── Container
        ├── Factory
        │   └── Project.php
        ├── Project.php
        ├── Relation
        │   └── Project.php
```

9 directories, 7 files

Nope, still have no idea. Here's my point:

Technical terms and overused patterns suck when it comes to high-level business applications.

So strategic design is all about **not building** projects like this one. And technical design gives you some valuable tools to achieve that. In the following pages, we'll talk about these concepts:

- Value Objects
- Data Transfer Objects
- Repositories
- Custom Query Builders
- Services
- Actions
- View Models
- CQRS
- States and Transitions
- Domains and Applications

Working With Data

Working with data is one of the most critical aspects of every business application. Unfortunately, PHP is not so good when it comes to this. In my opinion, one of the best and worst features of PHP is arrays. Especially associative arrays. The problems are:

- No type-hints
- Undocumented structure
- No restrictions. You can put product models, product IDs, and product arrays under the same key.

Associative arrays are big unstructured blobs of data. Don't get me wrong; they can be helpful but very annoying at the same time. Initially, PHP arrays tried to solve every problem: queues, stacks, lists, hash maps, and trees. Everything. But with its weak type system, it's tough to maintain this kind of data structure.

If you think about it, data plays a huge role in any business application:

- The request comes in. It contains the incoming data.
- The business layer processes this data.
- The database layer inserts this data into the DB.
- The response comes out. It includes the outgoing data.

So you have to work with data in every single layer of your application. Fortunately, Laravel and DDD give us some very clever concepts.

Value Objects

Value Object is an elementary class that contains mainly (but not only) scalar data. So it's a wrapper class that holds together related information. Let's see an example:

```
class Percent
{
    public readonly ?float $value;
    public readonly string $formatted;

    public function __construct(float $value)
    {
        $this->value = $value;

        if ($value === null) {
            $this->formatted = '';
        } else {
            $this->formatted = number_format($value * 100, 2) . '%';
        }
    }

    public static function from(?float $value): self
    {
        return new self($value);
    }
}
```

This class represents a percentage value. This simple class gives you three advantages:

- It encapsulates the logic that handles null values and represents them as percentages.
- You always have two decimal places (by default) in your percentages.
- Better types.

An important note: business logic or calculation is not part of a value object. The only exception I make is basic formatting.

By better types, I mean methods like this:

```
private function averageClickRate(int $total): Percent
{
    return Percent::from(
        SentMail::whereClicked() -> count() / $total
    );
}
```

You take a float value and make it a first-class citizen using a Percent value object. You don't have to worry about anymore if a method in your app returns a formatted string or a float number. Every percentage value can be expressed as a Percent object from now on. So you know, it contains the float number and the formatted string value.

The original definition of a value object states two more things:

- It's immutable. You have no setters and only read-only properties.
- It does not contain an ID or any other property related to the identification. Two value objects are equal only when the values are the same.

What else can be expressed as a value object? Almost anything, to name a few examples:

- Addresses. In an e-commerce application where you have to deal with shipping, it can be beneficial to use objects instead of strings. You can express each part of an address as a property:
 - City
 - ZIP code
 - Line 1
 - Line 2
- Numbers. Any financial application can benefit from using value objects when calculating metrics or comparing numbers. You can express some very high-level concepts, for example, Margin.
 - Top Line (such as revenue)
 - Bottom Line (such as net profit)
 - Margin (as a Percent, of course)
- Email addresses
- Or other application-specific concepts

Let's take a closer look at the Margin example:

```
class Margin
{
    public function __construct(
        public readonly float $topLine,
        public readonly float $bottomLine,
        public readonly float $margin,
    ) {}
}
```

Suppose you've worked with financial applications that deal with publicly traded companies. You know that a number like revenue is given in millions (or billions in some cases, for example, the market cap). So when you query Apple's revenue (which is 378 billion at the time of writing) from a finance API, you don't get 378,323,000,000 but 378,323, so we can express it in the code as well:

```
class Margin
{
    public function __construct(
        public readonly Millions $topLine,
        public readonly Millions $bottomLine,
        public readonly Percent $margin,
    ) {}
}
```

And we can use the `Margin` class like this:

```

class MetricsService
{
    public function profitMargin(IncomeStatement $incomeStatement): Margin
    {
        return new Margin(
            topLine: $incomeStatement->revenue,
            bottomLine: $incomeStatement->net_profit,
            margin: new Percent(
                $incomeStatement->net_profit->value / $incomeStatement->revenue-
>value
            ),
        );
    }
}

```

In this example, I assume that `revenue` and `netProfit` are instances of `Millions`. But isn't `IncomeStatement` an Eloquent model? Glad you asked. It is. And we can write a custom cast to convert floats to `Millions`:

```

class MillionsCast implements CastsAttributes
{
    /**
     * @param float $value
     */
    public function get($model, $key, $value, $attributes)
    {
        return new Millions($value);
    }

    /**
     * @param Millions $millions
     */
    public function set($model, $key, $millions, $attributes) {
        return [
            $key => $millions->value,
        ];
    }
}

```

```
}
```

It can be used in an Eloquent model, and here's how it works:

- When you're accessing an attribute on the model, the `get` method will be called. So `$incomeStatement->revenue` will return an instance of `Millions`.
- When you're setting an attribute on the model, the `set` method will be called. So `$incomeStatement->revenue = new Millions(1000)` will insert the `value` property (1000) from the `Millions` instance.

The last part is to use the cast in the model:

```
protected $casts = [
    'revenue' => MillionsCast::class,
    'net_profit' => MillionsCast::class,
];
```

So, in a nutshell, this is how you use a value object. To summarize it:

By using value objects, you can make objects from cohesive scalar data

The main benefits:

- It makes your code more high-level.
- It clarifies things and helps to avoid confusion. For example, now you know exactly that `Millions` contains a number stored in millions.
- It helps you deal with nullable values. You don't have to write `?float $revenue` anymore. You can write `Millions $revenue`.

In the introduction, I wrote that data is a crucial part of any application. I gave you this list:

- The request comes in. It contains the incoming data.
- **The business layer processes this data.**
- **The database layer inserts this data into the DB.**
- The response comes out. It includes the outgoing data.

As you can see in the cast and the other examples, a value object is mainly used inside (but not exclusively!) our application. In the next chapter, we'll discuss what happens at the boundaries (requests and responses).

Data Transfer Objects

The following important concept is data transfer object, or DTO for short. This is also a simple concept: it's a class that holds data. This data is then transferred between components. What are these components?

- Your application as a whole
- Classes inside your application

Let's take a look at a straightforward example:

```
class CourseController extends Controller
{
    public function store(Request $request): Course
    {
        $course = Course::create($request->course);

        foreach ($request->lessons as $lesson) {
            // $lesson is an array
            $course->lessons()->create($lesson);
        }

        foreach ($request->student_ids as $studentId) {
            $course->students()->attach($studentId);
        }

        return $course;
    }
}
```

This is an oversimplified example, of course. I'm working on an e-learning system, and you can believe me, the request for creating a new course is overwhelming.

After time this action will become more complicated, and you want to refactor it. Let's move this into a service (later, we'll talk about services in more detail. For now, it's a class that implements some business logic):

```

class CourseService
{
    public function create(array $data): Course
    {
        $course = Course::create($data);
        $this->createLessons($course, $data['lessons']);
        $this->addStudents($course, $data['student_ids']);

        return $course;
    }

    public function createLessons(Course $course, array $lessons): void
    {
        foreach ($lessons as $lesson) {
            // $lesson is an array
            $course->lessons()->create($lesson);
        }
    }

    public function addStudents(Course $course, array $studentIds): void
    {
        foreach ($studentIds as $studentId) {
            $course->students()->attach($studentId);
        }
    }
}

```

It looks okay, but can you spot the problems?

- Arguments like these: array \$data or array \$lessons
- Lines like this: \$data['lessons']

My biggest problem is that **I don't want to maintain and debug massive associative arrays five years later.**

The above example is very basic. Now, please imagine your favorite legacy project, where you have to work with methods like this:

```

public function createProduct(array $data)
{
    // Insert 673 lines of code here

    /**
     * You have to reverse-engineer this whole shit-show
     * just to get an idea about the shape of $data, right?
     */
}

```

DTOs can solve this problem by structuring your unstructured data. The same CourseService with DTOs:

```

class CourseService
{
    public function create(CourseData $data): Course
    {
        $course = Course::create($data->all());
        $this->createLessons($course, $data->lessons);
        $this->addStudents($course, $data->student_ids);

        return $course;
    }

    /**
     * @param Collection<LessonData> $lessons
     */
    public function createLessons(Course $course, Collection $lessons): void
    {
        foreach ($lessons as $lesson) {
            // $lesson is an instance of LessonData
            $course->lessons()->create($lesson);
        }
    }

    public function addStudents(Course $course, Collection $studentIds): void
    {

```

```

foreach ($studentIds as $studentId) {
    $course->students()->attach($studentId);
}
}
}

```

Now, instead of arrays, we have objects like `CourseData` and `LessonData`. Let's take a look inside `CourseData`:

```

class CourseData
{
    public function __construct(
        public readonly int ?$id,
        public readonly string $title,
        public readonly string $description,
        /** @var Collection<LessonData> */
        public readonly Collection $lessons,
        /** @var Collection<int> */
        public readonly Collection $student_ids,
    ) {}

    public static function fromArray(array $data): self
    {
        $lessons = collect($data['lessons'])
            ->map(fn (array $lesson) => LessonData::fromArray($lesson));

        return new self(
            Arr::get($data, 'id'),
            $data['title'],
            $data['description'],
            $lessons,
            collect($data['student_ids']),
        );
    }
}

```

The only place you have to deal with arrays with this approach is the DTO itself. Only the factory function will know anything about the ugly `$data` array. Every layer of your application will use a structured, type-hinted object.

As you can see, this class does not interact with `Request` or any other class that is environment-dependent so that you can use DTOs anywhere, including:

- Controllers
- Console Commands
- Services or Actions (covered later in the book)
- Models or Query Builders (covered later in the book)

And here's how you can use it from the `CourseController`:

```
class CourseController extends Controller
{
    public function store(
        Request $request,
        CourseService $courseService
    ): Course {
        return $courseService->create(
            CourseData::fromArray($request->all())
        );
    }
}
```

I think that's a much better approach, especially in larger projects. But now, we have another problem. Just imagine how many classes we need to create to store a course:

- CreateCourseRequest
- CourseData
- CourseResource
- LessonData
- LessonResource
- A few value objects here and there

And in this example, we have only two models! What if the domain model of this feature is much more complex? You can quickly end up with 10-15 classes to implement the CRUD functionality for courses. It's not the end of the world, but it can be very frustrating. Fortunately, we have an elegant solution. But first, let's summarize what a DTO is:

- It's an object that holds and transfers the data of a model.
- It can be used inside your application between components. Like in the example, when we created a DTO in the controller from a request and passed it to a service.
- But it can also be used outside of your application. So instead of having a request, a resource, and a DTO for the course, why not just have one DTO to rule them all?

Enter the [laravel-data](#) package by Spatie. You can use one DTO to act as a:

- Request (with validation rules)
- Resource
- And a simple DTO

Important note: If you want to use DTOs you don't need to go with laravel-data. You can write pure PHP objects, and you'll do just fine. But I find this package so helpful; I cannot imagine a large project without it.

This is what a laravel-data DTO looks like:

```
class SubscriberData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $email,
        public readonly string $first_name,
        public readonly ?string $last_name,
        /** @var DataCollection<TagData> */
        public readonly null|Lazy|DataCollection $tags,
        public readonly null|Lazy|FormData $form,
    ) {}
}
```

The basics are very similar to a pure PHP DTO, but we have this `Lazy` thing. I will talk about it later, but it's very similar to the `whenLoaded` method used in Laravel resources (it helps us avoid N+1 query problems). So we have a subscriber with a nested `TagData` collection and a nested `FormData` property.

This package can create a DTO from a request automatically. Since it can be used as a request, we can define validation rules:

```

public static function rules(): array
{
    return [
        'email' => [
            'required',
            'email',
            Rule::unique('subscribers', 'email')->ignore(request('subscriber')),
        ],
        'first_name' => ['required', 'string'],
        'last_name' => ['nullable', 'sometimes', 'string'],
        'tags' => ['nullable', 'sometimes', 'array'],
        'form_id' => ['nullable', 'sometimes', 'exists:forms,id'],
    ];
}

```

We can also specify how we want the package to create a `SubscriberData` from an HTTP request:

```

public static function fromRequest(Request $request): self
{
    return self::from([
        ... $request->all(),
        'tags' => TagData::collection(
            Tag::whereIn('id', $request->collect('tags'))->get()
        ),
        'form' => FormData::from(Form::find($request->form_id)),
    ]);
}

```

You can see that I run some DB queries to get the tags and the form. So instead of IDs (that come from the request), I have models which can map to `TagData` or `FormData`. Later I'll explain these things in more detail.

And look at the controller:

```
class CreateSubscriberController extends Controller
{
    public function __invoke(SubscriberData $data): SubscriberData
    {
        return SubscriberData::from(
            CreateSubscriberAction::execute($data)
        );
    }
}
```

You can inject any Data class, and transformation from the request will happen automatically! And as you can see, a Data object can be returned from a controller action and will be cast to JSON (including the nested properties).

In my opinion, it's a fantastic tool to have, so I'll use it heavily in the demo application later.

One more question to end up this chapter: what's the difference between value objects and DTOs?

- A DTO has an ID because it represents a model.
- A value object never has an ID. It represents a value, not an entity.

That's the main difference. However, in the Laravel community, I often see people mixing up the two concepts. You can even see people writing only value objects, and they use it as a DTO and as a VO.

And you know what? **It's perfectly okay, in my opinion.**

Of course, these rules are essential, but I don't see them as strict rules that you have to follow if you want DDD. I rather see them as guidelines. For example, later in the demo application, I will use a DTO instead of a value object because it's much more convenient in the given situation.

Repositories

Now that we've learned about data, we can move on to the database. Where to put your queries? How to structure your code? We've all asked these questions, and here's the answer: no one knows precisely. It's different for every project and every team.

One way to organize your queries is by using repositories. It's a controversial topic in the Laravel and PHP community. I won't be using them in the demo application, but it's a DDD concept and, to be honest, not a bad one. At the end of this chapter, I'll show you the Laravel equivalent of the repository pattern.

This is a repository class:

```
class ProductRepository
{
    public function create(ProductData $data): Product
    {
        Product::create($data->all());
        // Other database related code
    }

    public function search(
        PriceRange $priceRange,
        string $searchTerm
    ): Collection {
        return Product::query()
            ->whereBetween('price', [$priceRange->low, $priceRange->high])
            ->where('name', 'like', "%{$searchTerm}%")
            ->limit(10)
            ->get();
    }
}
```

It's a class for your database queries. That's it. So instead of writing your queries inside your controllers or models, you move them into a repository. Usually, each model has a repository, but that's not necessarily true every time.

You can use it like this:

```
class ProductSearchController extends Controller
{
    public function index(Request $request, ProductRepository $products)
    {
        return $products->search(
            PriceRange::from($request),
            $request->search_term
        );
    }
}
```

So it seems like a good concept; why are they being hated? There are several problems:

- Usually, each (or the vast majority of) model has a repository, such as the `ProductRepository` in this example. So you're literally just moving a query from one place to another, from the model to the repository. After six months of development, the `ProductRepository` will become a 5000 lines monster. Just like the `Product` model would have become a 5000 lines monster.
- By definition, when using repository classes, **every** database query should end up in one of them. So you need to move single-use queries as well. You'll quickly end up with a class containing methods used **one time in the entire application**.
- If you don't put every query inside a repository, then why do you even use repositories in the first place? In this case, you'll end up with an inconsistent structure. Some queries are in controllers or models; others live in a repository. In my experience, it's not an optimal solution.
- This is a personal one: I don't feel that `$products->search()` or `$this->products->getById($id)` fits well into Laravel.

However, repositories have some good attributes as well:

- You don't need to have one repository for every model. Let's imagine you have a big enterprise project with 200+ tables. One "module" or feature set is a very basic issue tracker. This feature set requires only six tables and 500 lines of database-related code. You don't have to spread these 500 lines of code in 6 repositories (as you would do with models). You can write only one class called `IssueTrackerRepository`. You cannot do that with models. I think it can be helpful in some situations.

To be honest, that's the only benefit I can think of. If you're already familiar with DDD, you probably heard something like this: *The repository pattern abstracts the data store and enables you to replace your database without changing your business code*. That's true. However, in the last decade, I've faced a lot of strange feature requests, but two of them never came up:

- Nobody ever asked me to change the programming language of an existing project.
- Nobody ever asked me to change the database engine under a running project.

So I'm not authentic to talk about it as an advantage. That being said, I don't think repositories are helpful in most situations. **In fact, Laravel has a better solution: custom query builders.**

Custom Query Builders

First, let's discuss what a query builder is. When you write something like that:

```
$query = Product::where('name', 'Domain-Driven Design with Laravel');
```

You're interacting with the `\Illuminate\Database\Eloquent\Builder` class. It contains your favorite Eloquent methods such as `where`, `firstWhere`, `latest`, etc.

But in Laravel, we don't write code like this:

```
Builder::where('description', 'LIKE', "%{$searchTerm}%");
```

We call the `where` method on the `Model` classes. So there has to be some connection between `Model` and `Builder`. This connection is the `newEloquentBuilder` in the base `Model` class:

```
/**
 * Create a new Eloquent query builder for the model.
 *
 * @param \Illuminate\Database\Query\Builder $query
 * @return \Illuminate\Database\Eloquent\Builder|static
 */
public function newEloquentBuilder($query)
{
    return new Builder($query);
}
```

It returns a new `Builder` instance, and every time you interact with your model, this instance will be used. So when you call `Product::where` the `newEloquentBuilder` method will be called, it returns a new `Builder`.

We can extend this base `Builder` class and can create a custom query builder:

```
class MailBuilder extends Builder
{
    public function whereOpened(): self
    {
        return $this->whereNotNull('opened_at');
    }
}
```

This is a custom builder for the `Mail` model. The `whereOpened` method can be used as a scope. In fact, model scopes are just syntactic sugar. So this method can be used like this:

```
Mail::whereOpened()->get();
```

In the builder we need to return `self` to be able to chain Eloquent methods:

```
Mail::whereOpened()->where('title', 'First Mail')->get();
```

However, you don't have to write scope-like methods. In a query builder, you can do (almost) anything:

```
class DividendPayoutBuilder extends Builder
{
    public function sumByDate(DateFilter $dates, User $user): float
    {
        return $this->whereBelongsTo($user)
            ->wherePayedBetween($dates)
            ->sum('amount');
    }
}

$dividendThisMonth = DividendPayout::sumByDate(DateFiler::thisMonth());
```

This method cannot be chained because it returns a float. The last piece of the puzzle is to instruct Laravel that we want to use our query builder. This can be done by overwriting the `newEloquentBuilder` method in your model:

```
class Mail extends Model
{
    public function newEloquentBuilder($query): MailBuilder
    {
        return new MailBuilder($query);
    }
}
```

And that's it! A **query builder is the Laravel equivalent of a repository if you think about it**. But in my opinion, it feels much more like Laravel. To sum it up:

- I prefer query builders over repositories.
- I usually write scopes and often-used methods in builders.
- But I don't put every query in builders.

This way, models remain very simple and thin. Later I will discuss where to put the single-use queries.

Services

A service class means a lot of different things for many developers. But in general: it's a class that holds some business logic. For example:

```
class TodoNotificationService
{
    public function sendDueTodayNotifications(): void
    {
        Todo::whereDueToday()
            ->get()
            ->each(fn (Todo $todo) => $todo->user->notify(
                new DueTodayNotification($todo)
            ));
    }
}
```

Services are often used to wrap some external services, such as Github, Twilio, and others. But they can be more low-level, something like this:

```
class TodoService
{
    public function create(TodoData $data): Todo
    {
        $todo = Todo::create($data->all());
        // ...
    }
}
```

- So services can be used as repositories? In theory, no, but in practice, yes. And a lot of developers use services instead of repositories.
- Can services be used together with repositories? Yes, but I don't recommend this approach.

When you have repositories and services as well in a project, this is the main idea:

- Each model has a repository.
- Repositories contain database queries.
- When needed, you create a service to a model (or a set of models, just as we discussed in the repository chapter). For example, the Todo class has more complex notification logic, so you might add a `TodoNotificationService`.

This approach is fine and can work, but here's the main problem:

- You end up with inconsistent classes. When you're working with the `Todo` model, you have a repository and a service. But when you're working on a `Project` related feature, you only have a repository because the project doesn't require a service class. And in the case of `Todo`, you won't be able to tell quickly if a method lives in a service or a repository.
- So, in general, your features are spread across services and repositories and a bit harder to reason about.

This isn't very objective, so it may work very well for you! As for me, I stick with actions.

Actions

What if we can combine repositories and services? We probably will have a big mess, like a fat controller that does everything, right? Yeah, it's true. But let's forget about technical responsibilities for a minute. In the first chapter, I wrote that technical code sucks in a business application. I think that's why I dislike repositories and services being used alongside each other.

But what if we specify the responsibility based on features instead of technical stuff? Now, we have a clean, reusable action class:

```
class CreateTodoAction
{
    public function execute(TodoData $data): Todo
    {
        $todo = Todo::create($data->all());

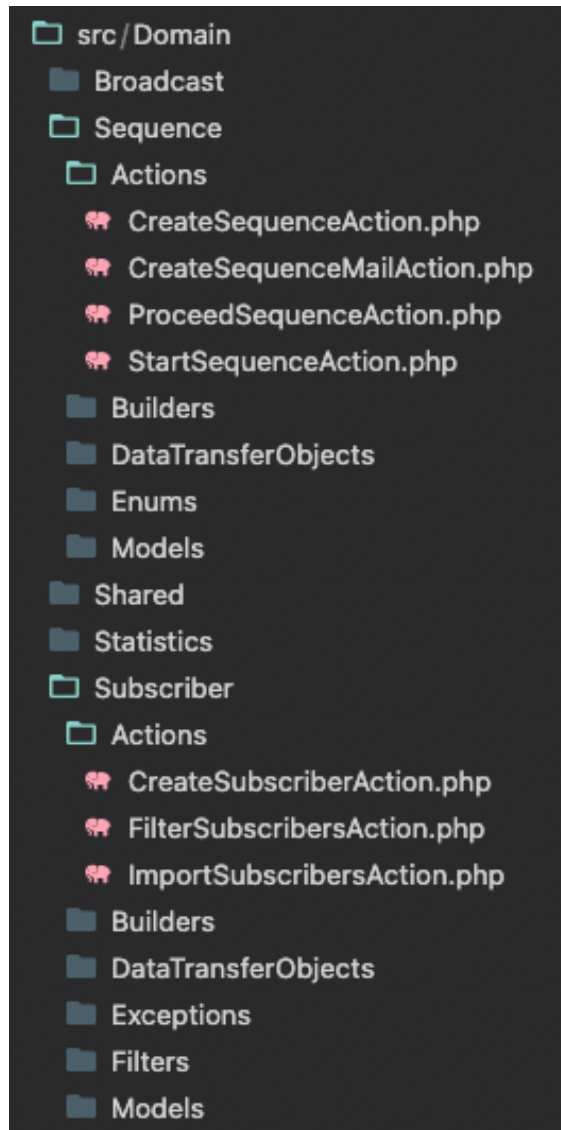
        if (!$todo->creator->is($todo->assignee)) {
            $todo->assignee->notify(new TodoAssignedNotification($todo));
        }

        $todo->watchers->each(fn (User $watcher) =>
            $watcher->notify(new TodoCreatedNotification($todo))
        );

        return $todo;
    }
}
```

Action classes have some significant advantages:

- Single responsibility. Each action takes care of precisely one thing, like creating a todo.
- Self-contained. An action completes a task from beginning to end.
- Nesting. An action can call any other action. If creating a todo is a complex task, maybe it's a good idea to extract the notification logic.
- Queueable. Not by default, but with the help of Spatie's [laravel-queueable-action](#) package.
- But here's the biggest one: **your actions describe your user stories**. So they bring your code closer to the business language. Just take a look at this:



If a new developer joins your team, he/she will know exactly what your application can do with subscribers:

- Create a subscriber
- Filter subscribers
- Import subscribers

There's no need to dig deep into controllers, models, or services. It's clean and straightforward. In my opinion, **it's a huge win**. As a bonus, actions are widely used in the Laravel community. By the way, these example come from the sample application we'll build throughout the book. As you can see, it's going to be a lot of action.

There's one more detail we can discuss. There are three ways to write functions in an action.

Non-static `execute` method

This is the most obvious choice, and you cannot go wrong with it. I used this in the previous examples.

Invokable class

In this case, you can write a class such as this:

```
class CreateTodoAction
{
    public function __invoke(TodoData $data): Todo
    {
        // ...
    }
}

class TodoController
{
    public function store(Request $request, CreateTodoAction $createTodoAction)
    {
        $todo = $createTodoAction(TodoData::from($request));
    }
}
```

As you can see, an action can be called just like a function. This is also a good option because actions look different from any other class, so you immediately know this is an action.

However, there are cases when you cannot inject the action into a method but only into a constructor. For example, when you're using an action in another one:

```
class CreateTodoAction
{
    public function __construct(
        private readonly NotifyUsersAction $notifyUsersAction
    ) {}

    public function __invoke(TodoData $data): Todo
    {
        $todo = Todo::create($data->toArray());
        ($this->notifyUsersAction)($todo);
    }
}
```

As you can see, you need to wrap the action in parentheses. And I don't particularly appreciate how it looks, so generally, this is not my go-to approach.

Static `execute` method

In this approach, all `execute` functions are static:

```
class CreateTodoAction
{
    public static function execute(TodoData $data): Todo
    {
        // ...
    }
}

class TodoController
{
    public function store(Request $request)
    {
        $todo = CreateTodoAction::execute(TodoData::from($request));
    }
}
```

This is my favorite one because it looks just neat! But if you want to write tests that mock actions, **it's a poor choice** because you cannot do that easily (or maybe it's impossible, I'm not sure). However, it's not a problem because I write API tests, so I don't mock actions. I only mock external dependencies, such as 3rd party APIs. In this book, I'll go with this approach, and you'll see how clean the code looks.

You can be wrong if you think actions are not part of DDD and are just some modern "Laravel magic." I first heard about actions from Robert C. Martin in his book "Agile Software Development," written in 2002 (10 years before Laravel was born). In this book, they were called "transactions." After that, I ran into them again in the C# world, where they were called "commands." By the way, this is the first letter in CQRS (more on that later). After all that, the MediatR package was created (2015), where actions were called "requests." As you can see, actions have a long history, and the Laravel community did not invent them. However, "action" is the coolest name I've ever heard to describe a class.

ViewModels

`ViewModel` is a very clever way to handle view-related data. But in this context, I'm not talking about Blade views exclusively. You can think of a view model as a data container responding to a specific request. They can be used in both SPAs (Inertia.js included) and full-stack MVC applications.

Let's say we work on a report page where we need to show revenue-related data, something like:

- Total revenue
- Total number of customer
- The average revenue per customer

This example can use a (highly simplified) view model like this:

```
class GetRevenueReportViewModel extends ViewModel
{
    public function totalRevenue(): int
    {
        return Order::sum('total');
    }

    public function totalNumberOfCustomers(): int
    {
        return Order::query()
            ->groupBy('customer_id')
            ->count('customer_id');
    }

    public function averageRevenuePerCustomer(): int
    {
        return $this->totalRevenue() / $this->totalNumberOfCustomers();
    }
}
```

A view model implements (or calls from a query builder) every query that the page or response needs. It can be used in controllers like this:

```
class RevenueReportController extends Controller
{
    public function index()
    {
        return new GetRevenueReportViewModel();
    }
}
```

But how can we produce a JSON response from a class with methods only? The base ViewModel class implements Laravel's `Arrayable` interface and will return an array like this:

```
[
    'total_revenue' => 24500,
    'total_number_of_customers' => 2311,
    'average_revenue_per_customer' => 10.60,
]
```

So a method called `totalRevenue` becomes a `total_revenue` array key, and Laravel will convert this array to JSON. Later I'll show the exact code that does the magic (5 lines using Reflection).

Wait a minute! Earlier, you said we're going to use DTOs as responses, didn't you? Yes, that's true, so in the sample application, instead of int values, I'll use DTOs. A quick example:

```

class GetDashboardViewModel extends ViewModel
{
    public function newSubscribersCount(): NewSubscribersCountData
    {
        return new NewSubscribersCountData(
            today: Subscriber::whereSubscribedBetween(
                DateFilter::today()
            )->count(),

            thisWeek: Subscriber::whereSubscribedBetween(
                DateFilter::thisWeek()
            )->count(),

            thisMonth: Subscriber::whereSubscribedBetween(
                DateFilter::thisMonth()
            )->count(),

            total: Subscriber::count(),
        );
    }
}

```

This is a dashboard page, and as you can see, I pack numbers together into a DTO, and I return this DTO from the ViewModel. The response looks like this:

```

{
    "new_subscribers_count": {
        "today": 4,
        "this_week": 39,
        "this_month": 104,
        "total": 341
    }
}

```

ViewModels can help your project in two ways:

- Your code is one step closer to the domain language. So when a product manager says "on the dashboard page," you immediately know that they talk about the `GetDashboardViewModel`.
- It can be an excellent addition to have the exact structure of your UI expressed as classes.

Later in the sample application, we will use Inertia but also APIs, so you will see how to write these classes for both situations (spoiler: the same way).

Now that we've learned about actions and view models, we can move on to the mysterious CQRS.

CQRS

CQRS is one of the most overcomplicated and misunderstood aspects of DDD, in my opinion, so let's clarify it! It means Command and Query Responsibility Segregation. It's a complicated name; let's see what those words mean:

- A command is a write operation such as creating a new product.
- A query is a read operation such as getting all products.
- Responsibility segregation means we need to separate these two types of functions.

So this simple class does not satisfy CQRS:

```
class ProductService
{
    public function create(ProductData $data): Product
    {
        // ...
    }

    public function getAll(): Collection
    {
        // ...
    }
}
```

I don't think there is anything wrong with this class; in fact, **it can be the superior solution** in many projects.

So how do we apply CQRS? If you think about the previous sections, we have similar concepts to commands and queries. The only difference is the name:

- Actions play the role of a command or a write operation.
- ViewModels play the role of a query or a read operation.

As you can see, it's very, very easy to apply CQRS by using these two classes. As it turned out, I was using CQRS years before I even knew what this thing was. So, I don't push the idea of CQRS; I like working with actions and view models. Also, I don't think it's necessary for every application, but it gives you some excellent benefits:

- Your codebase becomes more understandable.
- You have small, easier-to-maintain classes.
- It gives you a perfect separation of concern.
- Each class has one well-defined responsibility.

So that's what CQRS is all about. However, you'll find some very hard-to-understand articles and tutorials if you search for them. They often show you how CQRS is used with other more complicated concepts such as event sourcing, event stores, and separate read and write databases.

In this book, I won't talk about event sourcing for three reasons:

- I'm not using it, so I'm not authentic to write about it.
- I don't think it's necessary for most business applications.
- It requires a whole different architecture and perspective.

So it's doubtful that you will start to refactor your application to event sourcing after learning about it. However, it's much simpler to adapt any other concepts we'll use in this book.

CQRS can be a bit more complex (and usually is) than I described it. For example, in the C# world, developers use a so-called mediator. It's a way to implement in-process synchronous messaging. Basically, it's a layer that handles commands and queries. If you want to learn more about it, [check out this package](#).

Also, CQRS is a very different (and complex) animal in the microservices world. It means you have separate services and databases for reading and writing. In this context, it's almost required to use CQRS and event sourcing and event-driven async architecture together. If you're interested in this topic, [check out this video](#) (you can watch it without a master's degree in CS).

States And Transitions

A State is a class that represents the state of something. That something, in most cases, is an Eloquent model. So instead of a string, we have a dedicated class. What does a State class look like? Let's say we are working on an e-commerce application, so we have an Order class. An Order has a status like:

- Draft
- Pending
- Paid
- PaymentFailed

For the sake of simplicity, let's say the most critical business logic about the Order's state is whether it can be changed or not. A customer can modify a draft Order but cannot modify a Paid order.

First, let's create an abstract class:

```
abstract class OrderStatus
{
    public function __construct(protected Order $order)
    {
    }

    abstract public function canBeChanged(): bool;
}
```

Each state extends the OrderStatus parent class. Now we can create these concrete classes:

```
class DraftOrderStatus extends OrderStatus
{
    public function canBeChanged(): bool
    {
        return true;
    }
}
```

A draft order can be changed but a paid cannot be:

```
class PaidOrderStatus extends OrderStatus
{
    public function canBeChanged(): bool
    {
        return false;
    }
}
```

To make it even better, we can write an `Enum` that has a factory function and can create these status classes:

```
enum OrderStatuses: string
{
    case Draft = 'draft';
    case Pending = 'pending';
    case Paid = 'paid';
    case PaymentFailed = 'payment-failed';

    public function createOrderStatus(Order $order): OrderStatus
    {
        return match($this) {
            OrderStatuses::Draft => new DraftOrderStatus($order),
            OrderStatuses::Pending => new PendingOrderStatus($order),
            OrderStatuses::Paid => new PaidOrderStatus($order),
            OrderStatuses::PaymentFailed => new PaymentFailedOrderStatus($order),
        };
    }
}
```

As you can see, this enum acts like a factory function. This is one of the hidden features of PHP8.1 enums. In the Order model, we can leverage this with an attribute accessor:

```
class Order extends Model
{
    public function status(): Attribute
    {
        return new Attribute(
            get: fn (string $value) =>
                OrderStatuses::from($value) -> createOrderStatus($this),
        );
    }
}
```

This is the new Laravel 8 accessor syntax; it's equivalent to this one:

```
public function getStatusAttribute(string $value): OrderStatus
{
    return OrderStatuses::from($value) -> createOrderStatus($this)
}
```

First, I create an enum from the string value stored in the database; after that, I call the factory on the Enum. So anytime you access the status attribute of an order, you get an OrderStatus instance.

Now let's see how we can use these state classes:

```
class OrderController extends Controller
{
    public function update(UpdateOrderRequest $request, Order $order)
    {
        abort_if(!$order->status->canBeChanged(), 400);
    }
}
```

Or we can simply add a delegate method to the Order class:

```

class Order extends Model
{
    public function canBeChanged(): bool
    {
        return $this->status->canBeChanged();
    }
}

// Now we can use it like:
$order->canBeChanged();

// Instead of:
$order->status->canBeChanged();

```

This structure can give you some advantages:

- **Encapsulation:** Everything associated with a state is in one place.
- **Separation of concern:** Each state has its class, so you have an excellent separation.
- **More simple logic:** There is no need for nasty if-else or switch statements around a string attribute.

However, it can be a **significant overhead** if you have only a few states in your model and it's only used to check some basic behavior.

Moving on, we need to change the state from Pending to Paid at some point. I think we all wrote code similar to this in the past:

```

class OrderController extends Controller
{
    public function pay(PayOrderRequest $request, Order $order)
    {
        // Some logic here ...
        $order->status = 'paid';
        $order->save();
    }
}

```

Domain-Driven Design teaches us the following: we have to treat these transitions as first-class citizens. So let's put them in dedicated classes! First, we can create some kind of abstraction. In this case, we don't need a class, only an interface:

```
interface Transition
{
    /**
     * @throws Exception
     */
    public function execute(Order $order): Order;
}
```

A Transition can be executed. It takes an Order and returns an Order. If something goes wrong it throws an Exception. That's the contract.

This is a concrete Transition:

```
class DraftToPendingTransition implements Transition
{
    public function execute(Order $order): Order
    {
        if ($order->state::class !== DraftOrderStatus::class) {
            throw new Exception('Transition not allowed');
        }

        $order->status = PendingOrderStatus::class;
        $order->save();

        return $order;
    }
}
```

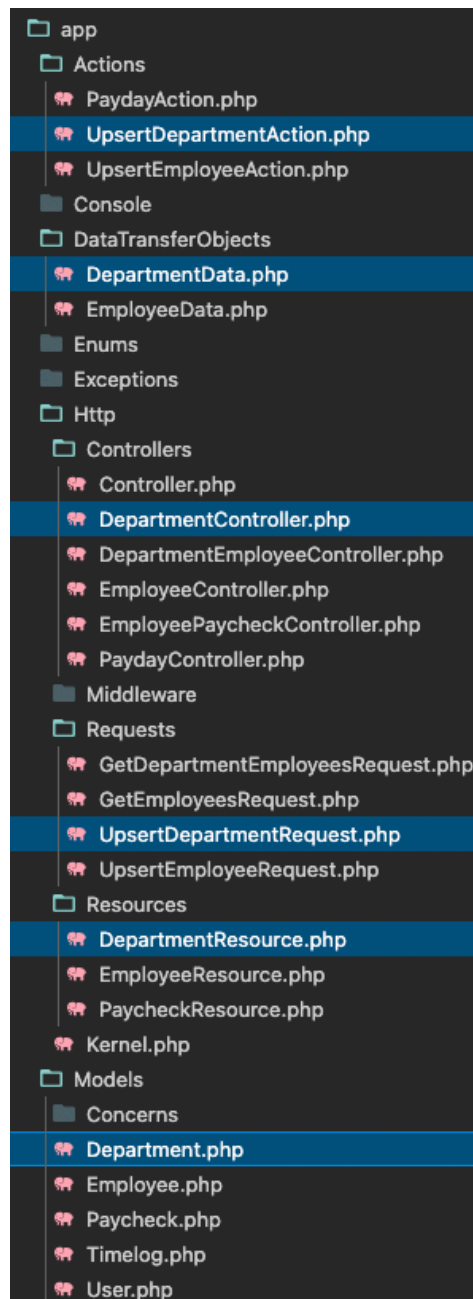
First, it makes sure that the current Order is Draft and updates it to Pending. Also, you can use PHP8 enum together with states and transitions.

As I said earlier, it can be beneficial if you have complicated logic based on states and transitions. However, it can be overengineering and introduce unwanted complexity in your codebase. For that reason, I only use them when I need to deal with really complicated state-related logic.

Domains And Applications

The name **Domain-Driven** Design implies that a domain is something that drives the development process. So it must be important, right? A domain is a “module” or a “container” that contains code that belongs together. But the main criteria of this grouping are not technical at all. It’s all about real-world problems and business language.

Laravel is a great framework and a good starting point for beginners. By default, it structures your code based on **technical details**. I’m talking about this:



This is a tiny application, but you can see the code that handles Departments is spread across **six different places**. It's grouped by technical attributes:

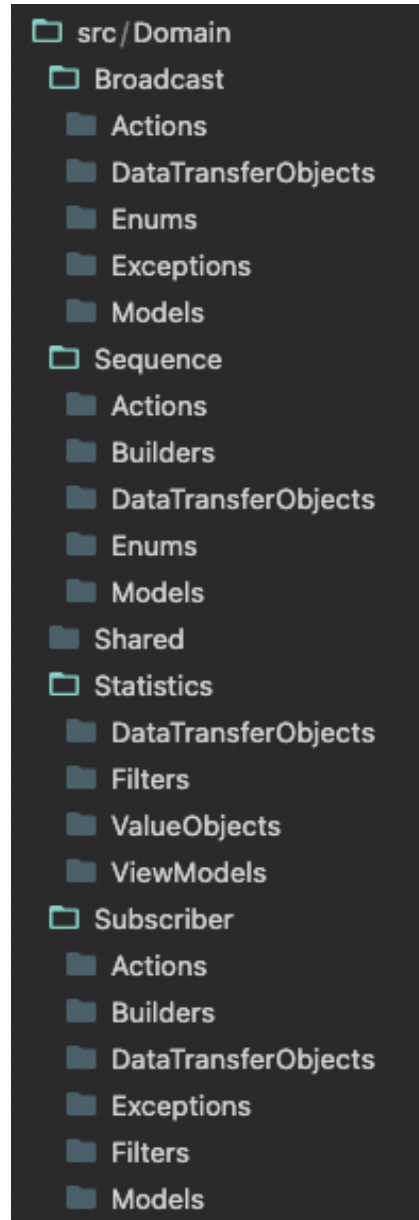
- Controllers
- Models
- Requests
- Resources
- Actions
- DTOs

Imagine a project with 200 models, 300 actions, 500 controllers, and 500+ routes. After a while, it becomes a nightmare. If you need to introduce a new attribute to the department model and accept it in your API, you probably have to change **five files in six different directories**.

By the way, have you ever heard a stakeholder saying something like: “Please work a little bit on the DepartmentController and add a new attribute to the DepartmentResource”? Of course not. They say things like: “On the department page, I need to see the number of employees within the department.” And from that information, you know that you probably need to tweak the DepartmentController and the DepartmentResource.

A developer is someone who translates the business language into code.

So, in my opinion, it is a good idea if the code is similar to the business language and follows the same structure. By using domains, we can achieve a much better structure:



The main benefit is that if you work on a broadcast-related feature, you go into the Broadcast folder, and everything is there. Meanwhile, your co-worker can work in the Subscriber domain. It also makes onboarding easier, in my opinion.

As you can see, there is no HTTP or Controllers folder inside a Domain. That's intentional. A domain contains only business code. The subscriber folder contains everything you can do with a subscriber. It does not have any application-related code. By application, I mean:

- API
- Web app
- Console commands

These environment-related classes live inside the app folder, just like in any Laravel app.

An important note: by using domains we don't need to change anything framework-related. So there's no need to tweak configs or bootstrap logic. Everything is untouched; Laravel upgrades won't be affected.

Advantages And Disadvantages

We've discussed most of the essential technical concepts in Domain-Driven Design. To wrap things up:

- Data
 - Value Objects
 - Data Transfer Objects
- Working with data
 - Repositories (or custom query builder in Laravel)
 - Services
 - CRQS
 - Actions
 - View models
 - States and transitions
- Code structure
 - Domains
 - Applications

If you think about them, these are straightforward ideas. Except for states and transitions, there's not even a "pattern" or a class structure behind these concepts. They take an idea (like structuring unstructured data with DTOs) and give us some classes that help us achieve the goal clean. This brings us to the most important advantage:

- **The technical aspects of DDD are easy.** We, developers, make it more complicated than it should be.

But these simple classes and ideas come with disadvantages as well:

- There are a lot of different concepts and jargon associated with DDD. This is why it's often misunderstood, and people make it more complicated than it should be.
- Also, at the code level, we're going to use a lot of different classes. It can be hard to get used to it if you're writing fat controllers and models.
- As a bonus, if you're browsing DDD articles or videos online, they are often written in a highly abstract way, so you need a Ph.D. just to understand them.

But here's the important thing:

- In my view, these concepts are not strict rules. So it's not like "you either use them 100% correct, or you're not even doing DDD." I don't think that's a good attitude.
- For example, if you're confused about DTOs and value objects, pick one of them and put everything under that folder. It's okay. You won't be outcasted from the developer community.

This book aims to help you write better software instead of blindly following some rules that don't fit in Laravel.

Important note: there are other DDD concepts such as aggregates and aggregate roots that we won't use in this book. The reason is simple: I don't think they are helpful with Laravel.

Designing an E-mail Marketing Software

Most DDD articles or tutorials give you some abstract ideas and overcomplicated concepts and then assume you're good to go. I want to do something different in this book. The best way to learn new ideas is to build projects. So in the upcoming chapters, we're going to build a functional, complex e-mail marketing system, just like ConvertKit or MailChimp. If you're not familiar with these applications, don't panic; in the following pages, I will explain every important feature.

Overview

At its core, an e-mail marketing application is software that stores your subscribers, and you can send e-mails to them. But it also comes with a great set of more complex features, such as:

- Managing subscribers.
- Tagging them. Tags are helpful to filter subscribers when you want to send e-mails.
- Sending broadcast (or one-time) e-mails. You write the content, set the filters, and hit the Send button.
- Creating sequences. A sequence is a collection of e-mails delivered to subscribers after a certain delay. For example, you write four e-mails, and you want to send these e-mails in four weeks. You can create a sequence that does precisely that automatically. And the great thing about sequences is that they handle new subscribers as well. So if you create this sequence in February and a person subscribes in May, they will still be added to it and get one e-mail per week.
- Adding subscribers to sequences based on specific criteria. We can also create different filters. For example, we want to exclude subscribers who bought a particular product or have a specific tag.
- Creating subscription forms where people can enter their e-mail addresses. You can build an HTML form with an e-mail input that can be embedded into your site. If someone submits this form, they will be added to your e-mail list.
- Tracking e-mail opens, and link clicks.
- Generating reports from these metrics.
- And much more, but these are the core features of ConvertKit.

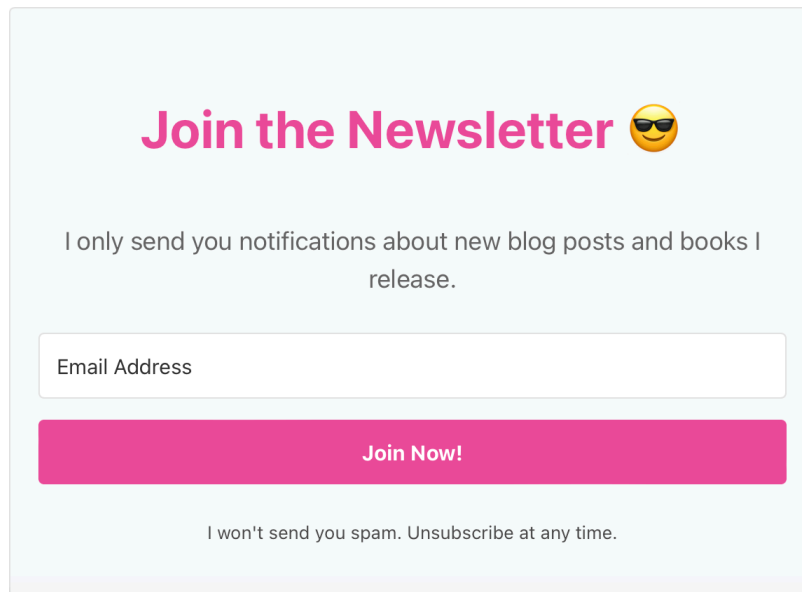
In the upcoming pages, I'll discuss these features in more detail, and after that, we're ready to write some user stories and design the domain model of our own ConvertKit Clone.

Subscribers

Subscribers represent people in an e-mail list. Usually, a subscriber has an e-mail address and other custom fields, like first name, last name, etc. A subscriber can be added to a list in several different ways:

- Manually
- By submitting a subscription form
- Via CSV import

We'll cover all of these use cases in our application; however, I won't implement a fully responsive works-on-every-device form builder because it's out of the scope of this book (and I'd probably lose the fight between me and CSS). By the way, a form looks like the one from my blog:



Join the Newsletter 😎

I only send you notifications about new blog posts and books I release.

Join Now!

I won't send you spam. Unsubscribe at any time.

Subscribers can also have tags. Tags can be used to filter out people from a particular e-mail. Let's say you have an e-mail list that contains people who haven't opened your e-mails in the last six months (you can get this statistic from ConvertKit). First, you query them and mark them with an inactive tag. Later, you can write an e-mail where you try to re-engage with these people. When you send this e-mail, you want to apply a filter with the inactive tag, so only those subscribers will get it.

Broadcasts

Broadcasts are e-mails you want to send one time to a given set of subscribers. First, you write the content and save it; after that, you either send it manually or schedule it for a later time.

Either way, you define which subscribers will get the mail. You can filter them by:

- Which tags do they have
- What form did they come from
- What products did they purchase

We won't implement products and purchases in the sample application, so our filters will work based on tags and forms.

Metrics like these can track the performance of a sent broadcast:

- How many people got the mail.
- Open rate. How many people opened the mail.
- Click rate. How many people clicked on a link inside the content (if there's a link).

These rates can be expressed as percentage values, for example:

- 1000 sent mails
- 49% open rate (490 were opened)
- 8% click rate (80 were clicked)

Sequences

Sequences are one of the most powerful features of ConvertKit. Imagine you want to create an e-mail course on Domain-Driven Design. You write eight e-mails with eight different concepts and ideas. If you use broadcasts to do this, you get the following results:

- On the 1st of June, you have 1000 subscribers, and you send out the first mail.
- From the 1st through the 8th of July, you get 100 new subscribers. They haven't received the first mail because they joined your newsletter a few days later.
- On the 8th of July, you have 1100 subscribers, and you send out the second mail.

The 100 new subscribers never got the first e-mail. They started your course on the 2nd e-mail. This is obviously not what we want. The first part of the solution is to create a sequence. The second part will be discussed in the next chapter.

A sequence is a set of e-mails grouped together. Using the DDD e-mail course as an example, you can create a sequence like this:

- Sequence name: Domain-Driven Design Course
 - 1st e-mail: What Is DDD?
 - 2nd e-mail: Using DTOs
 - 3rd e-mail: The Benefits of Value Objects

We want to send one e-mail per week but only on Wednesday. So we can specify delays between each e-mail:

- 1st e-mail will be sent on the first Wednesday after you publish the sequence.
- 2nd e-mail will be sent on the 2nd Wednesday.
- 3rd e-mail will be sent on the 3rd Wednesday.

We can implement this logic using a scheduled console command. ConvertKit even lets you create automation where you can link together multiple sequences using conditionals.

Each e-mail in the sequence can have custom subscriber filters; each subscriber has a different status when consuming a sequence. Let's take a look at an example.

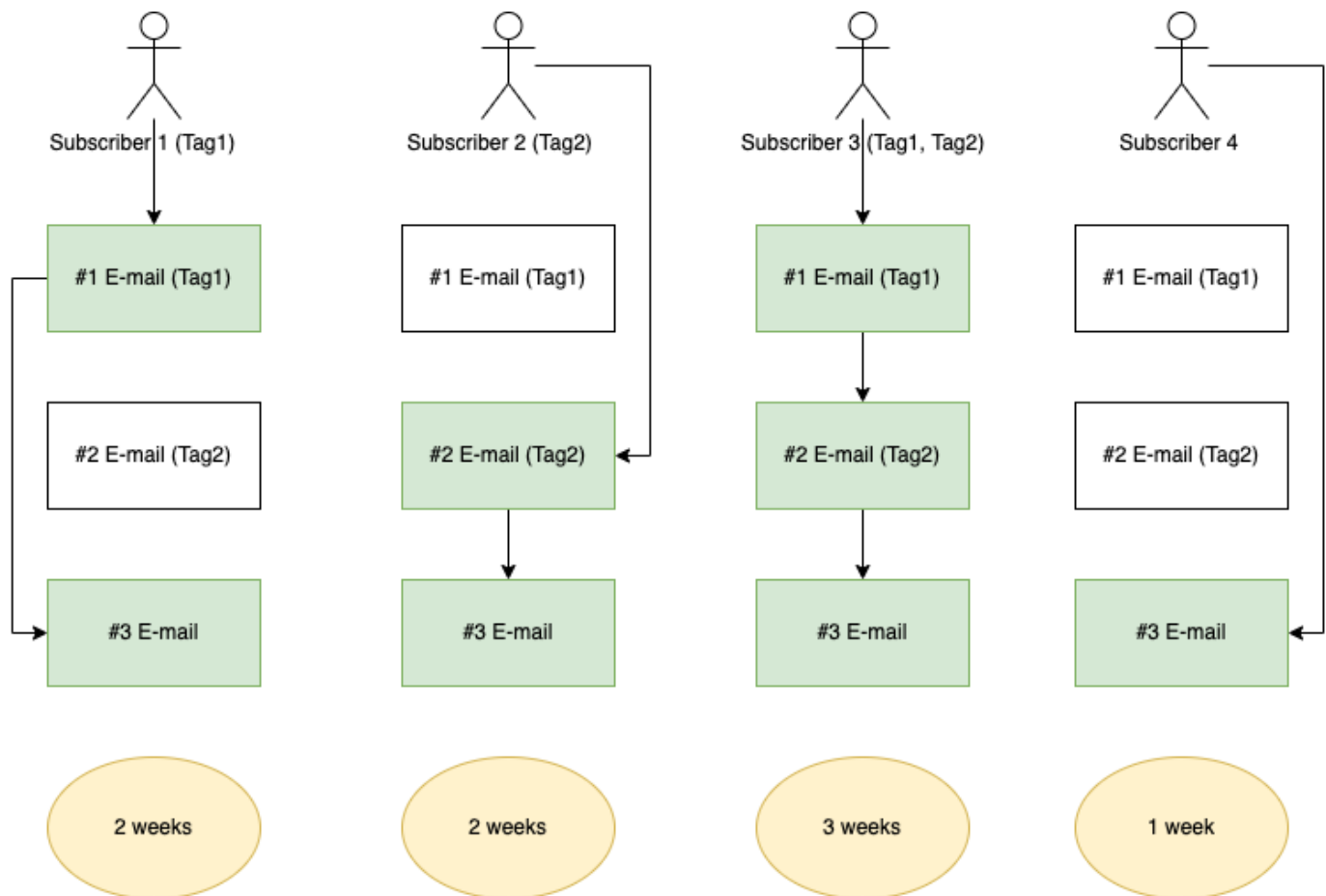
We have the following subscribers:

Subscriber Name	Tags
Subscriber 1	Tag 1
Subscriber 2	Tag 2
Subscriber 3	Tag 1, Tag 2
Subscriber 4	-

And this sequence:

E-mail Title	Filters (tags)
#1 E-mail	Tag 1
#2 E-mail	Tag 2
#3 E-mail	-

Each e-mail will be sent a week after the previous one. The results will be:



As you can see, each subscriber will get different e-mails, and it takes a different time to complete the sequence.

Automations

Let's say you have put together an excellent sequence from Laravel tips and other helpful content. It contains ten e-mails, and you send one e-mail every week. At the moment, you have 1000 subscribers, and you start your sequence. People love it, and you gain some more subscribers. Now, you have 1500 people on your list. But what happens with the new 500 subscribers? They all missed your content because they were not your subscribers when you started your sequence. Wouldn't it be great if we could come up with some solution?

ConvertKit solves this problem by using automation. Automation is a "flow" of events and actions, something like this:

*"When an **event** happens, *then* do this **action**."*

To give you an example:

"When a person subscribes to my e-mail list, add them to my awesome e-mail sequence."

This automation makes it possible that every new subscriber will get your important and valuable e-mails. As you can see from the examples, there are two essential parts of automation:

- Event. An event will happen, and it triggers an action.
- Action. This is what the app will execute as a consequence of the event.

In the demo app, we will handle the following events and actions:

- Event: when a subscriber subscribes to a form.
- Action: then add them to a sequence.
- Action: tag them.

We'll build them in an easy-to-extend way, so adding new events or actions will be straightforward. It's an exciting feature with a lot of fun stuff!

Other Features

Performance tracking. As I described earlier, we want to track three main performance metrics for each mail:

- Total e-mails sent out.
- Open rate.
- Click rate.

When you write an e-mail in ConvertKit, it will embed a 1px image in the content. When the image is loaded, ConvertKit knows that a particular subscriber opened the e-mail. When the e-mail has a link, and you click on it, it won't redirect you directly to the destination. First, it will open a ConvertKit URL and redirect you to the original destination. We won't implement these features, but we can simulate them with an API to track these metrics. From these metrics, we can craft some interesting reports. When it comes to sequences, there are two types of reports:

- For a particular e-mail in the sequence. In the example above, #3 E-mail was sent out to 4 subscribers; meanwhile, #1 E-mail was sent out to only two subscribers.
- For the whole sequence. For example, eight e-mails were sent all together in the example above:
 - Two instances of #1 E-mail
 - Two instances of #2 E-mail
 - Four instances of #3 E-mail

Tracking and reporting a broadcast's performance is the same as tracking a sequence mail's performance. We probably need some abstraction to handle this without code duplication.

Filters and Forms. We will have a Form model, but we won't implement a form builder. Filters can be created using forms and tags.

Why E-mail Marketing?

As you can see from these chapters, building e-mail marketing software can be pretty complicated! This is why I chose this topic:

- The domain model is perfect for introducing DDD. From these features, we can already identify some important domains: subscriber, broadcast, sequence, and report. Probably all of these can be a separate domain.
- The business logic is complicated. Just look at the figure above. Or think about sending 10000 e-mails or importing 50000 subscribers. We will face some challenges.
- We need to avoid code duplication. A broadcast and a sequence e-mail sound very similar to each other. We probably need some abstraction.
- Reports everywhere! Writing reports-related code can be pretty fun but often become a big mess.

It's interesting from the frontend perspective as well:

- I will use Inertia.js because it's a handy tool. If you're not familiar with it, don't panic. From the BE perspective, it's only one line of code in the controller.
- But we also need some API endpoints to track the open and click rates and create subscribers. These requests come outside of our application (from an embedded form or a script in the e-mail's content).
- Because of this, technically, we have three applications:
 - Web. We are building with Inertia, used by logged-in users.
 - API. We are building with Laravel, used by external parties.
 - Console. Scheduling sequences or importing subscribers.

For these reasons, I think ConvertKit is the perfect example to replicate and learn Domain-Driven Design!

User Stories

Now we are ready to write some user stories that we will implement later. However, I don't want to overload you with every tiny detail, so we're focusing only on the essential stories, the application's core.

User Story	Description
Subscribers	
Creating a subscriber	Needs an API endpoint.
Importing subscribers from CSV	We treat this as an "admin task" so it's gonna be a Console Command.
Filtering subscribers	It's not really a user story but we need it in multiple places.
Broadcasts	
Creating a broadcast	With filters and content.
Sending a broadcast	First it filters out the subscribers and then it queues up the e-mails.
Sequences	
Creating a sequence	With basic data such as title.
Adding a sequence mail	With filters and schedule (such as 3 days after the last mail).
Publishing a sequence	It's just a state transition from draft to published.
Proceeding a sequence	This will handle all the scheduling logic. It needs to be scheduled to run frequently. It will send the e-mails.
Tracking	
Tracking e-mail opens	It can be a broadcast or a sequence e-mail. Needs an API endpoint.
Tracking link clicks	It can be a broadcast or a sequence e-mail. Needs an API endpoint.
Automations	
Creating automations	Each automation has one event and any number of actions.
Running automations	When the specified event occurs we need to run the actions.

Reports	
Sequence progress	Such as how many subscribers completed it.
New subscribers	How many new subscribers are this week, this month, today, or in total
Daily new subscribers	How many new subscribers we had on a daily basis.
Performance for a sent mail	Either for a broadcast or a sequence mail (open rate, click rate).
Performance for a whole sequence	Aggregating the individual e-mails for a whole sequence.

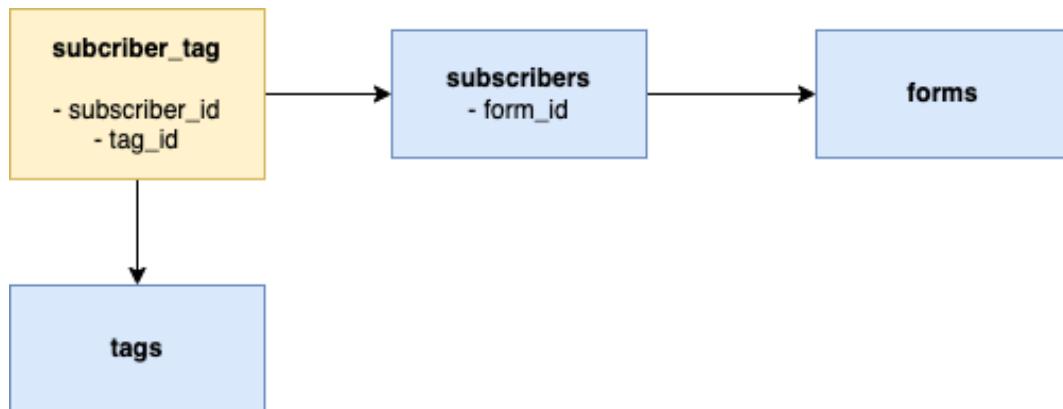
Data Modeling

This is one of the most crucial steps in every project, so let's nail it!

Subscribers

The easiest part is the subscriber-related tables, so let's start there. We know that subscribers have tags; this is a many-to-many relationship, so we need a pivot table. Subscribers also have a form where they come from. This is a one-to-many relationship.

So far, we have this:



Broadcasts

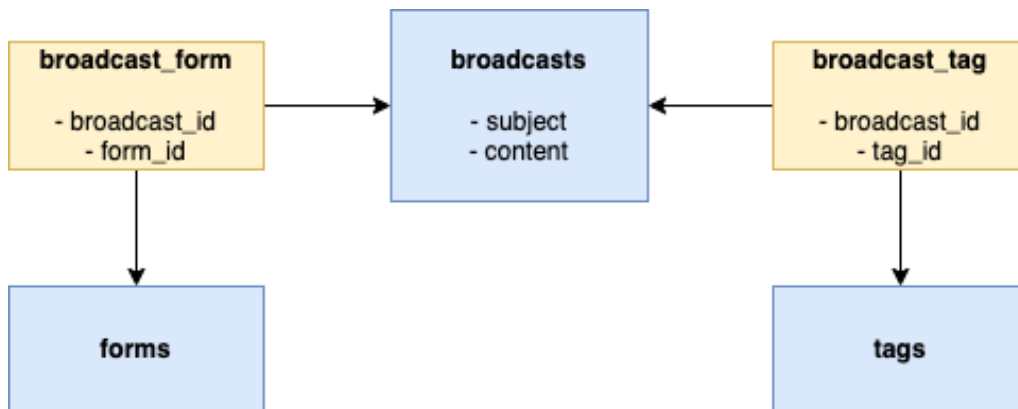
Now let's move on to broadcasts. A broadcast itself is straightforward; we only need to store some basic information such as the subject line or the content. But as you can see in the user stories, when we're sending a broadcast, we need to filter the subscribers based on the broadcast's filters. These filters can be:

- One or more forms
- One or more tags

This can be implemented in at least two ways.

Separate tables for the filters

This solution looks like this:



It's not a bad solution, but I see some problems that may occur:

- We need two extra tables and possibly six eloquent relationships to implement filtering by tags and forms.
- I don't like to predict the future, but filters are almost always something that will change over time. And being a ConvertKit user myself, I can say that a production e-mail marketing system also has filters like:
 - Landing pages (similar to forms)
 - Segments. It's a subset of subscribers based on some criteria.
 - Products and purchases. They also have e-commerce features.
 - Integration with other services. For example, Gumroad products.
- We need new tables and relationships for every new use case if we choose this structure.

So it can work, but I don't feel it's flexible enough to handle real-world use-cases.

Using a JSON column

Instead of using separate tables, we can add a JSON column to the broadcasts table, something like this:

```

{
  "form_ids": [1,2,3],
  "tag_ids": [12,4]
}
  
```

We can store a JSON array like this in a column. The data structure above describes a filter:

- Forms with the ID of 1,2,3
- Tags with the ID of 12,4

What are the advantages of this solution?

- Easy to implement.
- Easy to extend.
- Fast implementation. It can be an advantage in many projects and a disadvantage in others.
- Later, it's easy to convert it to separate tables if you change your mind.

What are the drawbacks of this solution?

- No data integrity. What if you delete the tag with the ID of 4? You need extra code to guarantee that none of the filters reference it anymore. However, in this situation, it's only partly true. Using `broadcast_form` and `broadcast_tag` pivot tables gives you data integrity. But we know that we need the same filters for sequences later. It means we are forced to use polymorph relationships, and instead of `broadcast_id` we have to go with something abstract such as `sendable_id`, which can reference either a broadcast or a sequence_mail. Since you cannot use foreign keys with morph relationships, you have no data integrity by using pivot tables.

Isn't a random JSON structure the same as having random associative arrays? Yes, but we can quickly solve this by casting this value to a DTO or value object.

Important note: I will use a JSON column. Partly because this is educational content, and by using JSON, I have an opportunity to show you how to deal with unstructured data and make it a joy to work with it. I think you can benefit from it if you're working with legacy systems.

SentMails

Moving on, what happens after a broadcast has been sent? We know from the user stories that we need to calculate reports like:

- Total e-mails sent
- Open rate
- Click rate

So we need something that represents a "sent e-mail." After sending a broadcast, it will send X e-mails and need to create X rows in a table, where X is the number of subscribers. It's also a requirement to track sequence e-mails, so we need to do the same with sequence e-mails. We need a table with a polymorph relationship. What does this table represent? As I said earlier, it stores e-mails that already have been sent, so my suggestion for the name is `sent_mails`:

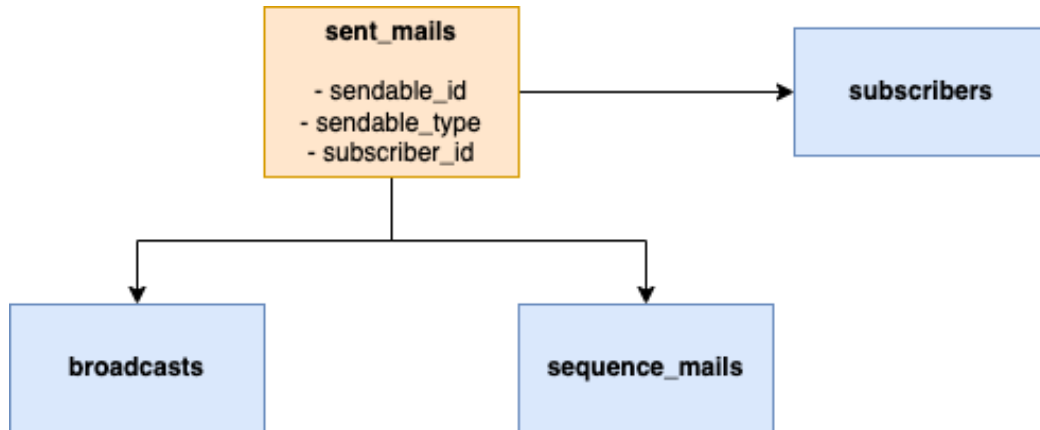
id	sendable_id	sendable_type	subscriber_id	sent_at	opened_at	clicked_at
1	1	Broadcast	101	2022-04-22 13:00:00	2022-04-22 13:11:00	2022-04-22 13:12:00
2	1	Broadcast	129	2022-04-22 13:00:00	2022-04-22 13:24:00	
3	4	SequenceMail	411	2022-04-25 09:13:00		

Since the table name is `sent_mails`, I've decided to use the `sendable` as the name for the morph relationship. The `sendable_type` will contain the fully-qualified class name of the models that I don't know precisely at the moment, so I just wrote the class names.

The tracking API will update the `opened_at` and `clicked_at` columns for a given subscriber and sendable. From this table, we can calculate the reports. For example, the performance of the broadcast:

- Total e-mails sent: 2
- Open rate: 100%
- Click rate: 50%

The database structure is very simple:



From this table, we can also query statistics for an individual subscriber, such as:

- How many e-mails have been sent to this particular subscriber?
- What is the average open rate for this specific subscriber?
- From the `sent_at` column, we can even query something like, on average, how many days passed between e-mails?

Once again: I don't want to predict the future, but reports are usually very "dynamic" by nature. This means that the management always wants more exotic reports. For me, this structure looks like a good starting point that supports several different statistics.

A Quick Note On Performance

Now let's play a little with numbers. First, let me make some assumptions:

- Our application will have around 100 000 users
- Users have an average list size (subscriber count) of 5 000
- They send an average of 1 e-mail per week

100 000 users * 5 000 e-mails * 52 weeks = 26 000 000 000 rows in `sent_mails` in one year. It seems incredibly high, right? Let's make some other assumptions:

- Acquiring 100 000 users will probably take YEARS. It depends on many factors, of course, but it's a very high number. ConvertKit has about 500 000 users (the number comes from their site), and it's an ~8-year-old business in a niche where the CEO is very well-known.
- 5 000 is an entirely made-up number; I couldn't find a reliable number online.

At this stage, I wouldn't worry about this kind of problem. Instead, I want to be ready for the first 100 or 1000 users. What can we do to make the DB effective?

- Indexing the `sent_at` column.
- Also the `opened_at` and `clicked_at` columns

The indexing itself can speed up your application. The next step is to write queries that can use these indexes, so 'never' write something like this:

```
select count(*)
from sent_mails
where year(sent_at) = "2022";
```

This query **won't** use your index on the `sent_at` column. Simply because MySQL cannot use indexes if you're using the `year` function. So instead of using a function, write queries like that:

```
select count(*)
from sent_mails
where sent_at between "2022-01-01 00:00:00" and "2022-12-31 23:59:59";
```

So you've created indexes and written good queries, but your reports are still slow. What else can you do?

- You can always look for N+1 query problems. It's a very common problem when you're working with ORM.
- You can optimize select * queries. Most of the time, it's unnecessary to query every column.

Your next step can be database partitioning. I'm talking about horizontal partitioning. It's a way to tell MySQL how to store your data physically. You can create multiple portions (partitions) from a table, and the engine will store these portions in different files. For example, you can define rules such as:

- `sent_mails` between 2022-01 and 2022-06 should be a partition.
- `sent_mails` between 2022-07 and 2022-12 should be another partition.

This way, you can write queries that only touch a small number of partitions (fewer rows to fetch).

If you're still having problems, you can always use Redis. These historical reports are static by nature. If you want, you can replicate the whole `sent_mails` in Redis and serve the requests from there. There are a dozen different ways to do that.

Sequences

Now we can move on to sequences. Once again, the idea is this:

- We have a sequence that contains multiple e-mails.
- Each of those e-mails has some scheduling logic.

We need to handle two different time units when scheduling sequence e-mails:

- Hours
- Days

So users can create schedules such as:

- Five days after the last e-mail.
- Two hours after the previous e-mail.

They also want to specify which days the given e-mail can be sent:

- Five days after the last e-mail, but only on Fridays.
- Two hours after the last e-mail on any day.

Before modeling the tables, let's discuss how to store these schedules:

delay	unit	allowed_days
5	days	{"monday": false, "tuesday": false, "wednesday": true, "thursday": false, "friday": false, "saturday": false, "sunday": false}
2	hours	{"monday": true, "tuesday": true, "wednesday": true, "thursday": true, "friday": true, "saturday": false, "sunday": false}

The attributes are the following:

- `delay` stores the number of days/hours we need to wait after the previous e-mail.
- `unit` is either days or hours. This column can be omitted if we store every delay in hours. In this case, five becomes 120. This can work, but we don't have to convert hours from days in the code if this table has a unit column. It's easier, in my opinion.
- `allowed_days` is an object that contains every day and a bool value associated with them. This tells us if the e-mail should be sent on a given day. In the first example, the e-mail will only be sent on Wednesday. In the second example, it'll be sent on any weekday.

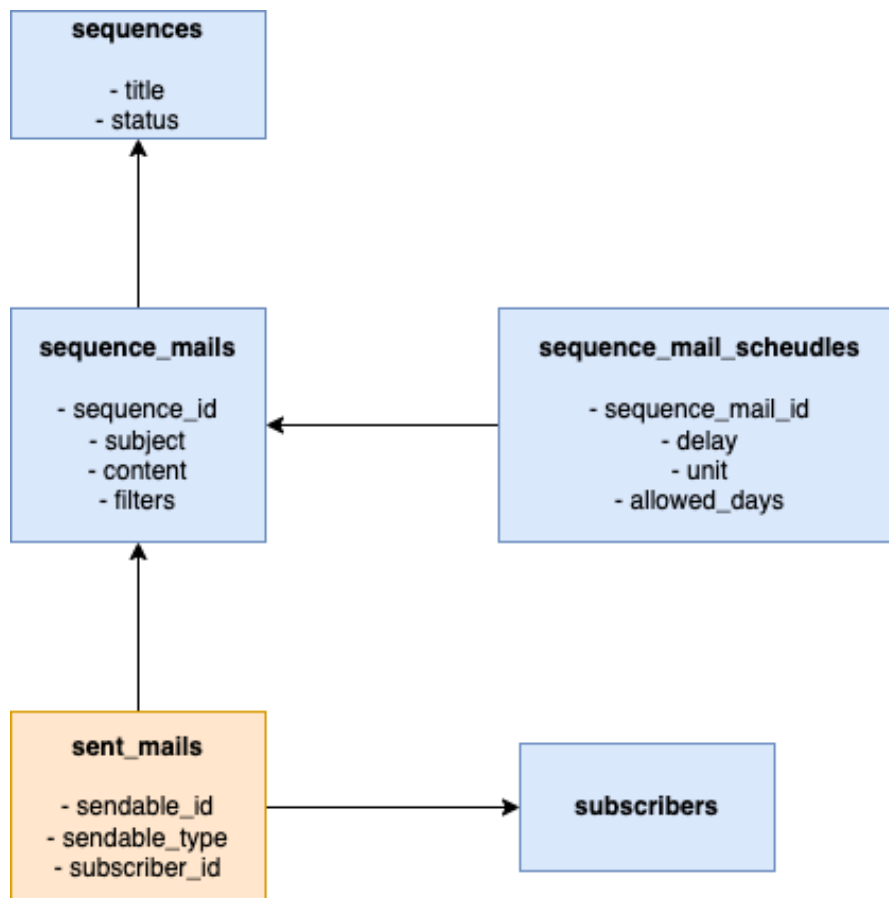
With this simple table, we can describe any scheduling logic we have right now.

Why did we create a new table for schedules but use a JSON column for filters before? This schedule can be also be stored as JSON, right? Yes, that's true. I have two reasons to go with a separate table:

- Believe it or not, scheduling logic is more straightforward and more "static" than filtering. This simple use case is the whole scheduling logic of ConvertKit! And I don't see too much change in that requirement.
- Since we used a JSON column earlier, I think it's good to go with a separate table this time. Now you can see the problems solved with DTOs and value objects. After implementing these features, you can

decide which is better for your current/future project.

Now we can construct the whole DB structure for sequences:



It's quite easy. A sequence mail is very similar to a broadcast; it also has:

- Subject line
- HTML content
- JSON filters

A sequence is just a "container" with a name and status. But it's important because only published sequences can send e-mails to subscribers. And as we've discussed earlier, each sequence mail becomes a row in the `sent_mails` table after it was sent.

Automations

The last important feature I want to model before moving on is automation. As I said earlier, we need to build flows like this one:

"When a person subscribes to my e-mail list, add them to my awesome e-mail sequence."

At the database level, it's straightforward; I think we only need two tables to represent such automation:

id	name
1	Add to newsletter

The more exciting part is the steps of this automation:

id	automation_id	type	name	value
1	1	event	subscribedToForm	{"form_id": 1}
2	1	action	addToSequence	{"sequence_id": 3}

The first row represents the event, so the *"When a person subscribes to my e-mail list"* part. While the second row is the action itself, or the *"add them to my awesome e-mail sequence"* part. The type and name columns are self-explaining. The value contains the ID of the "thing" that needs to be dealt with. Once again, I'm using a JSON column. However, these two examples can be done using a simple integer; what happens when there are more advanced, complicated steps? Such as "When a subscriber purchases a particular product, then wait for two days and send an upsell e-mil." In this case, we can benefit from a JSON column. On the other hand, we sacrifice nothing in the case of simple examples.

As you can see, it's a pretty simple database structure. However, we'll face some challenges in the code.



Before we move on, let me answer a question: if every automation has only one event, why not store it in the automation itself? Because this way, we have the flexibility to add multiple events to automation if needed later.

Domains

The last step is to plan the domains of the application. Maybe this is a new thing for you, but this is the easiest step. First, let's list all the tables we have come up with:

- subscribers
- tags
- subscriber_tag
- forms
- broadcasts
- sequences
- sequence_mails
- sequence_mail_schedules
- sent_mail
- automations
- automation_steps

The next step is to group these tables:

Table	Group
subscribers	Subscriber
tags	Subscriber
subscriber_tag	Subscriber
forms	Subscriber

These groups will be the domains of the application. Because subscribers have tags and forms, I've put tags and forms into the Subscriber group. If we had full form-builder functionality with dozens of other classes, Form would have been a separate group. But right now, they can be put into the Subscriber group.

Now let's see broadcasts and sequences:

Table	Group
broadcasts	Broadcast
sequences	Sequence
sequence_mails	Sequence
sequence_mail_schedules	Sequence
sent_mail	Broadcast or Sequence?

This is one way of doing it. We create a group for Broadcast and another one for Sequence. This approach has several benefits:

- Domains remain small.
- We don't mix up different concepts, for example, broadcasts and sequences.
- Sequences are complicated, so I feel it's a good thing to have a separate domain.

And several disadvantages:

- Maybe some domains will be too small? Such as the Broadcast. Right now, we only have one model in it.
- What about `sent_mails`? Both sequences and broadcasts are using it, so we cannot put it in one group. We can solve this using a Shared domain. It's kind of like a "joker" domain where you can put classes used by multiple domains. It's not the best solution, but it definitely works. And believe me, every project has a Shared or Common domain. Literally, every larger application has one of these.
- What about other shared classes between broadcasts and sequences? We are in the design phase, so we only know about one shared class. But I'm 100% absolutely sure that we need other classes. These all will be placed inside the Shared domain and will be used by sequences and broadcasts. And remember, right now, we only have tables and models. What about DTOs, value objects, actions, and so on? For example, both broadcasts and sequences have filters that require a DTO or a value object (or maybe both?). This will also live in the Shared domain.

I think this solution is not that bad, but broadcasts and sequences can be grouped in another way:

Table	Group
broadcasts	Mail
sequences	Mail
sequence_mails	Mail
sequence_mail_schedules	Mail
sent_mail	Mail

We can invent a new domain called Mail. What are the disadvantages?

- This domain can grow too large. With the current feature set, I'm sure it's going to be okay, but I also feel like sequences will be more complicated in the future and require more and more tables, classes, actions, and so on. So this is a valid risk.
- We mix up two concepts. It can be wrong but also good. For example, not having a Shared domain because of `sent_mails` is a good thing, in my opinion. So in this particular case, it's not a disadvantage.

Other than that, having a Mail domain is the superior option, in my opinion. So our domains look like this:

- Subscriber
- Mail
- Automation
- Shared

I listed Shared because I'm sure we're going to need it. Just a quick example: what if we need a base model for every other model? Or a percent value object? As I said earlier, I think it's okay to cheat a little bit with a Shared domain.

After all this planning, this is the whole database design:

```

subscribers:
  attributes:
    - email
    - first_name
    - last_name
  relationships:
    - form: A subscriber has one form
    - received_mails: A subscriber has many received mails. They can be
broadcasts and sequence mails
    - tags: A subscriber has many tags

tags:
  attributes:
    - title
  relationships:
    - subscribers: A tag has many subscribers

forms:
  attributes:
    - title
    - content: The HTML that can be embedded into websites
  relationships:
    - subscribers: A form has many subscribers

broadcasts:
  attributes:
    - title
  
```

- **content**: HTML content
- **filters**: JSON field
- **status**
- **sent_at**

relationships:

- **sent_mails**: A broadcast has many sent mails. One for every subscriber.

sequences:

attributes:

- **title**
- **status**: draft, published

relationships:

- **sequence_mails**: A sequence has many sequence mails

sequence_mails:

attributes:

- **subject**
- **status**: draft, published
- **content**: HTML content
- **filters**: JSON field

relationships:

- **sequence**: A sequence mail belongs to one sequence
- **schedule**: A sequence mail belongs to one schedule
- **sent_mails**: A sequence mail has many sent mails. One for every

subscriber.

sequence_mail_schedules:

description: Each sequence mail has a unique schedule such as 2 days after the last e-mail, only on Fridays

attributes:

- **delay**: For example, 2 days after
- **unit**: hour, day
- **allowed_days**: On which days the mail can be sent

sent_mails:

description: After sending a broadcast or a sequence mail it becomes a sent_mail record.

attributes:

- **sendable_id**: The ID of the broadcast or the sequence mail
- **sendable_type**: broadcast, sequence_mail
- subscriber_id
- sent_at
- opened_at
- clicked_at

automations:

description: It is like a folder for the actual steps

attributes:

- name

relationships:

- **steps**: Automation has many steps

automation_steps:

description: It contains a flow of an event and multiple actions for an automation

attributes:

- automation_id
- **type**: event or action
- name
- value

relationships:

- **automation**: Automation step belongs to one automation

Building an E-mail Marketing Software

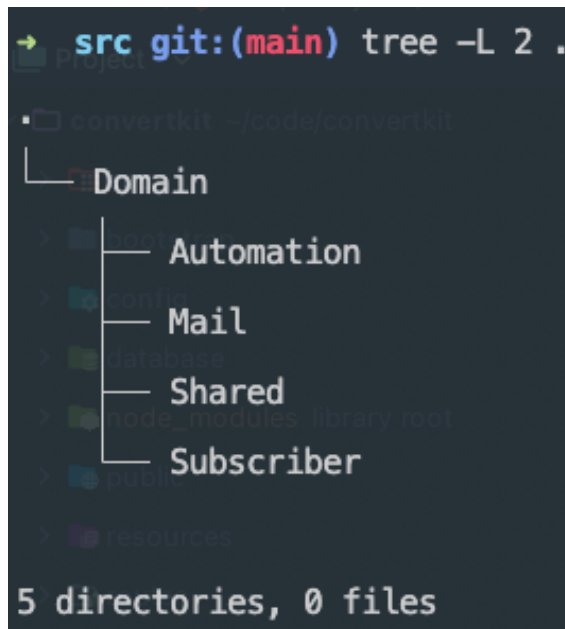
After all of these database designs and modeling let's write some code! First I want to show how to set up domains and applications, and after that, we can move on to subscribers.

Setting Up Domains and Applications

As I said earlier, domains are not modules, so they don't require tweaking config files and bootstrap processes. Only a few things need to be done, for example, autoloading in composer.json:

```
"autoload": {
    "psr-4": {
        "App\\": "app/",
        "Domain\\": "src/Domain/",
        "Database\\Factories\\": "database/factories/",
        "Database\\Seeders\\": "database/seeders/"
    }
},
```

The `App` namespace remains the same, and we add a new Domain namespace that references the `src/Domain` folder. This `src/Domain` folder contains all of the domains:



```
→ src git:(main) tree -L 2 .
├── Domain
│   ├── Automation
│   ├── Mail
│   ├── Shared
│   └── Subscriber
├── ...
└── ...

5 directories, 0 files
```

You absolutely don't need to use the Domain folder as the root; it's just an older habit of mine. Why the `src` directory? Because many tools will search for either the `app` or the `src` directory. I'm talking about tools like static analysis, some test helpers, and other developer tools. So by using the `src`, we can use most of these tools without any further configuration.

Each of those domain folders (such as the `Subscriber` or the `Automation`) will have a structure like this:

```

→ Subscriber git:(main) tree -L 1 .
Basic Concepts
├── Domain-Driven Design
├── Actions
├── Working With Data
├── Builders
├── Value Objects
├── DataTransferObjects
├── Exceptions
├── Filters
├── Query Builders
├── Services
├── Jobs
├── Actions
├── Models
├── ViewModels
├── ViewModels
├── CQRS
├── States And Transitions
8 directories, 0 files

```

This is your regular app structure. Each class type has a separate folder. Nothing unexpected so far.

Dealing with User model

If we use domains to structure our code, where to put the User model? Right into the Shared domain. Every single domain uses it, so it's shared in the application. To do that, we have to change Laravel's User provider configuration in the config/auth.php:

```

'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => \Domain\Shared\Models\User::class,
    ],
],

```

The *model* was changed from the default `App\Models\User::class`. After that, we can move the user into the `Shared\Models` namespace.

BaseModel and factories

In most applications, I use a `BaseModel` that extends Eloquent's model class and is extended by all models. It's also a class that can be placed inside the Shared domain. When using domains, you have to override a method on the model called `newFactory`. This method will return a new factory instance associated with the model. By default, Laravel tries to resolve the factory based on the model's namespace. But in our case, it tries to instantiate a class like `Database\Factories\Domain\Subscriber\Models\SubscriberFactory`. It's clearly not what we want.

First things first, I also namespace the `database/factories` folder by domain:

```

→ factories git:(main) tree -L 2 .
.
├── Actions
├── ViewModels
├── .
├── CQRS
├── Automation
│   ├── States And Transitions
│   │   ├── AutomationFactory.php
│   │   └── AutomationStepFactory.php
│   ├── Domains And Applications
│   └── Advantages And Disadvantages
├── Mail
│   ├── Designing an Email Marketing Software
│   │   ├── BroadcastFactory.php
│   │   ├── SentMailFactory.php
│   │   ├── Overview
│   │   │   ├── SequenceFactory.php
│   │   │   ├── SequenceMailFactory.php
│   │   │   └── SequenceMailScheduleFactory.php
│   ├── Subscribers
│   └── Broadcasts
├── Shared
│   └── Other
│       └── UserFactory.php
├── Subscriber
│   ├── User Stories
│   │   ├── FormFactory.php
│   │   └── Data Modeling
│   │       ├── SubscriberFactory.php
│   │       └── Subscribers
│   └── Broadcasts
│       └── TagFactory.php
├── SentMails
└── A Quick Note On

```

4 directories, 11 files

Why not put the factories inside the domain folder? Because they are not related to business logic:

- Factories are only used in tests and seeders, so the subscriber domain can do its job without the `SubscriberFactory`.
- I don't want yet another folder inside the `Domain\Subscriber`.

I think they should be left in the default `database/factories` directory for these reasons.

Back to the `BaseModel`, we need to tell Laravel how it should resolve these factories:

```
namespace Domain\Shared\Models;

abstract class BaseModel extends Model
{
    use HasFactory;

    protected static function newFactory()
    {
        $parts = str(get_called_class())->explode("\\");
        $domain = $parts[1];
        $model = $parts->last();

        return app("Database\\Factories\\{$domain}\\{$model}Factory");
    }
}
```

The `explode` will return a collection such as:

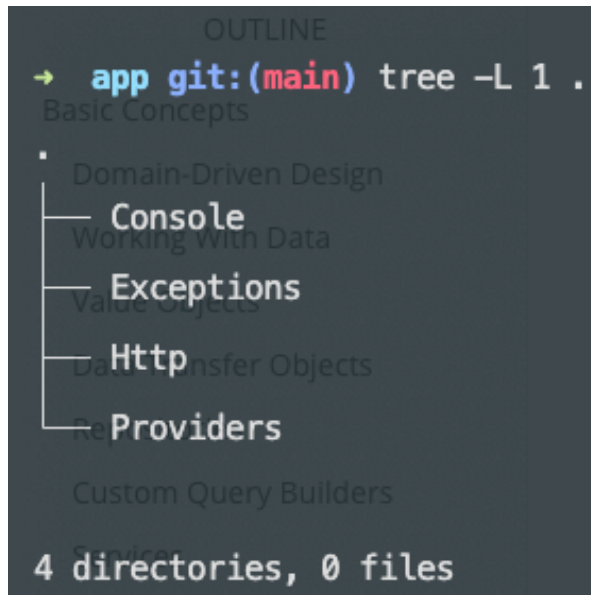
```
$parts = [
    'Domain',
    'Subscriber',
    'Models',
    'Subscriber',
];
```

From this array, we can construct the `Database\Factories\Subscriber\SubscriberFactory` class name. Also if you don't like to write `$fillable` arrays you can define `$protected $guarded = [ ];` in the `BaseModel`.

Applications

As I mentioned in the first section, I will use Inertia.js in the demo application. Please don't panic if you don't know it. The main difference between an Inertia.js and a classic Blade MVC application is one line on the Laravel side. Instead of returning a `view`, you have to return an Inertia specific class from the controller. On the frontend side, it's a Vue.js application, so later in this book, I'll share code snippets from good, old Vue.js components (but, of course, 95.13% of code snippets will show you PHP and Laravel code).

So our main app will be an Inertia.js app, but we also need some APIs (for example, tracking when a user opens an e-mail or clicks on a link). And, of course, we'll have some console commands (like every application has some). Now let's see what the app directory looks like:

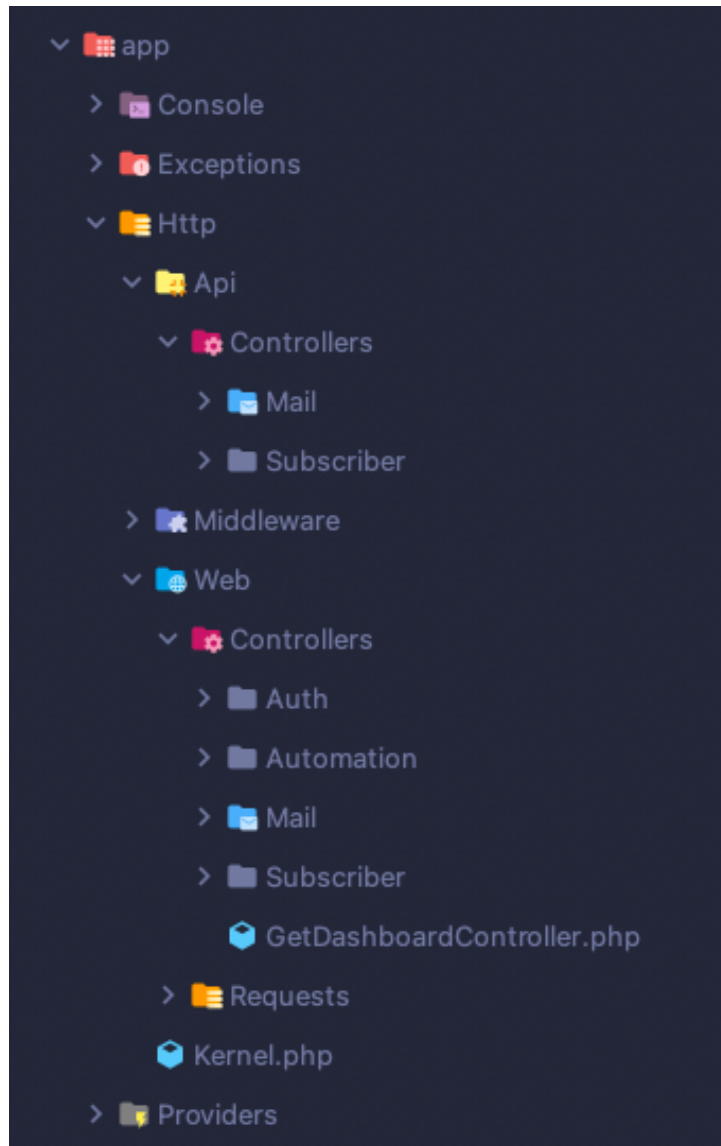


```

OUTLINE
→ app git:(main) tree -L 1 .
Basic Concepts
├── Domain-Driven Design
├── Console
├── Working With Data
├── Exceptions
├── Value Objects
├── Http
├── Data Transfer Objects
├── Providers
├── Custom Query Builders
└── Services
4 directories, 0 files
  
```

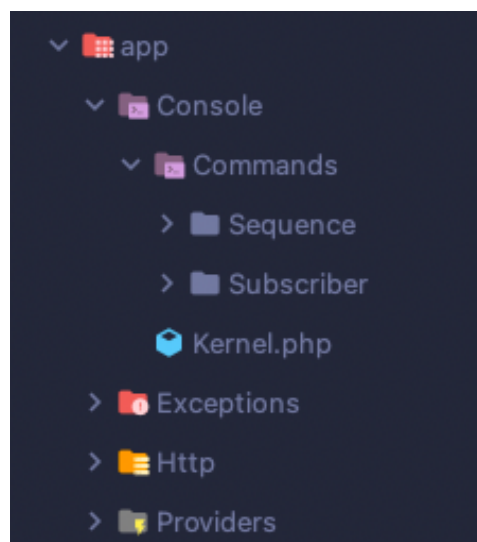
As you can see, it follows the default structure of a new Laravel app. In my opinion, it's important to stay close to the default stack so that Laravel upgrades will be as easy as possible.

If you take a closer look inside the Http directory, you can see there's an Api and a Web folder:



The Web folder contains the application-related backend logic for the Inertia.js app, while the Api folder contains the API-specific controllers. As you can see, each of those folders is namespaced by domains. Since both of these applications use middlewares, I left them in the Http folder. I find this setup very easy to understand and a joy to maintain. There's no unnecessary complexity.

By the way, the Console application is also namespaced by domains:



Shared domain

Now you probably have a better idea about the Shared domain. It contains classes that are shared across many (often every) domains. So far, we have put the `User` and the `BaseModel` classes inside the Shared domain. What other classes will it contain?

- Any base class like the `BaseModel`. For example, view models will have a parent class that lives inside the Shared domain (more on that later).
- General-purpose classes, such as a `Percentage` value object, a `DateFilter`, an Eloquent trait.
- Or even some actions can be placed inside this domain. For example, you need to read a CSV file and write an action for this. What if you need to use this class from many domains?

Users

I want to clarify one more thing before moving on. There are no teams, companies, or tenants in this application. Users register and can create their subscribers, broadcasts, automations, etc. So (almost) every table has a `user_id` column.

Subscribers

Creating A New Subscriber

Now that domains and applications have been set up let's write some code! I will start with subscribers because this is the most simple domain.

The subscriber form is pretty simple:

New Subscriber

The form is titled "New Subscriber". It contains the following fields and controls:

- FIRST NAME:** A text input field containing the value "Jane".
- LAST NAME:** A text input field containing the value "Doe".
- E-MAIL:** A text input field, currently empty.
- FORM:** A dropdown menu with a downward arrow, currently showing a hyphen "-" as the selected option.
- TAGS:** A list of tags: "Laravel", "Vue", "MySQL", "Inertia", and "DDD".
- Buttons:** A blue "Save" button and a "Cancel" button.

A subscriber can have only one form but many tags. Now let's take a look at the subscriber model:

```
class Subscriber extends BaseModel
{
    protected $fillable = [
        'email',
        'first_name',
        'last_name',
        'form_id',
        'user_id',
    ];

    public function tags(): BelongsToMany
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

```

public function form(): BelongsTo
{
    return $this->belongsTo(Form::class)
        ->withDefault();
}
}

```

At this point, there's nothing too interesting. A many-to-many tag and a one-to-many form relationship. In the form relationship, I use the `withDefault` method. By default, if a subscriber does not have a form (it's a nullable column), Laravel will return `null`, and it can cause some bugs and unnecessary `if` statements. Using the `withDefault` Laravel will return an empty Form object instead of `null`. It can be helpful in some situations.

Now, let's see the subscriber DTO. In code, I don't use the word 'DTO'. I call the namespace `DataTransferObjects`, and I suffix the classes with the term `Data`.

This is the `SubscriberData` class:

```

namespace Domain\Subscriber\DataTransferObjects;

use Spatie\LaravelData\Data;

class SubscriberData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $email,
        public readonly string $first_name,
        public readonly ?string $last_name,
        /** @var DataCollection<TagData> */
        public readonly ?DataCollection $tags,
        public readonly ?FormData $form,
    ) {}
}

```

All DTO will look similar to this one:

- The ID is always optional because there's no ID when a POST request comes in, and we transform it into a DTO. But when a PUT request comes in, there's an ID. And remember, this DTO will also be used when querying the subscribers and returning a response.

- Properties are in `$snake_case` format. By default, laravel-data will map the request or the model attributes with the DTO's properties. By using `snake_case` variables, there's no extra work to do. This also applies to plain DTO classes (when you're not using the laravel-data package). If you want your properties to be in `camelCase` you have to write the logic that transforms the `model_attributes` to `dtoAttributes`. I have done it in the past, but after a while, it gets messy. So nowadays, I'm using `snake_case` everywhere:
 - Models
 - DTOs
 - Request parameters
- `$form` is a nested property. As you can see, there's a `FormData` class that is a nested property of the `SubscriberData`. Just as the `Subscriber` model has a `Form` attribute. laravel-data helps us make this nesting very easy. We'll talk about it in more detail later.
- `$tags` is also a nested property, but a subscriber has many tags, and as you can see in DTOs, we can use the `DataCollection` class to do this mapping. It comes from the laravel-data package.
- The `$form` and `$tags` are nullable properties because not every subscriber has them.

I know it's probably a lot of new information but bare with me. Now let's see what the request and the validation rules look like:

```
public static function rules(): array
{
    return [
        'email' => [
            'required',
            'email',
            Rule::unique('subscribers', 'email')->ignore(request('subscriber')),
        ],
        'first_name' => ['required', 'string'],
        'last_name' => ['nullable', 'sometimes', 'string'],
        'tag_ids' => ['nullable', 'sometimes', 'array'],
        'form_id' => ['nullable', 'sometimes', 'exists:forms,id'],
    ];
}
```

There are some basic validation rules here, maybe the e-mail that is a bit more complicated, so let me explain it:

- The `email` has to be unique. Laravel will run a `select` query to check if there's a record in the `subscribers` table with the e-mail in the request.
- It works perfectly when creating a new subscriber.
- When it's an update, we need to ignore the currently updated subscriber's e-mail address. Otherwise, Laravel will throw an exception because the `subscribers` table already contains this e-mail address.

By the way, can you spot the two differences compared to a standard Laravel request `rules` function?

- This method is static.
- There's a `request` call in `ignore`.

What the hack is going on? Here's the thing: this method is in the `SubscriberData` class. As I said earlier, when using the `laravel-data` package, we have one class for:

- Request
- Resource
- DTO

And the package expects a static `rules` function. Since it's static, in the `ignore`, I cannot write something like `$this->id` as I would in the case of a Request class. So I get the subscriber ID from the request parameter. In the case of an update request, the URL looks like this: `subscribers/{subscriber}`, so the `request('subscriber')` will return the `Subscriber` model that is being updated.

How can we use this class? Let's see the controller:

```
public function store(
    SubscriberData $data,
    Request $request
): RedirectResponse {
    UpsertSubscriberAction::execute($data, $request->user());

    return Redirect::route('subscribers.index');
}
```

I will talk about the action later, but now let's focus on the DTO. As you can see, we can inject the `SubscriberData` class into the method, and the package will automatically create a new instance from the request. But I also inject the `Request` itself to get the currently logged-in user.

I could put a user property in the `SubscriberData` but remember we will also use this class to return as a response, and I don't want to include sensitive user data in the responses if it's not necessary. And in most cases, the user is not needed on the frontend. Remember, when users log in to the application, they will only see their information. So there's no need to display to creator user's name or anything like that. For that reason, I left out the user property from the DTO.

By default, `laravel-data` will do a one-to-one mapping from the request to the data object. For each property within the data object, a value with the same key will be searched within the request values. This default behavior works in most cases. But if you take a look at the data class, it has properties like `tags` and `form`; meanwhile, in request, we expect keys like `tag_ids` and `form_id`. So how can we override the default mapping?

We need to define a `fromRequest` method in the data class:

```

public static function fromRequest(Request $request): self
{
    return self::from([
        ... $request->all(),
        'tags' => TagData::collection(
            Tag::whereIn('id', $request->collect('tag_ids'))->get()
        ),
        'form' => FormData::from(Form::findOrNew($request->form_id)),
    ]);
}

```

In a moment, I'll show you the `TagData` and the `FormData` classes. A `from` function is available in every data class; it's a simple factory function that creates a new DTO from an array. We can use this and override the array keys we want to transform:

- `form` will be a `FormData` instance that wraps the query's result.
- `tags` will be a collection of `TagData` that wraps the query's result.

It's very similar to Laravel resource classes. Before we move on, let's summarize how the whole flow works:

- The `SubscriberData` is type-hinted in the constructor.
- `laravel-data` will run the validation rules against the request.
- The package will create a `SubscriberData` instance from the request using the `fromRequest` factory function.
- This function will create a `FormData` instance from the `form_id`.
- And a collection of `TagData` from the `tag_ids`.

As a result, from the following request:

```

{
    "id": null,
    "first_name": "Test",
    "last_name": "Subscriber",
    "email": "test@subscriber.io",
    "form_id": 2,
    "tag_ids": [1, 2]
}

```

We got this DTO:

```

Domain\Subscriber\DataTransferObjects\SubscriberData {#1548 ▼
  #includes: []
  #excludes: []
  #inclusionTree: null
  #exclusionTree: null
  #additional: []
  +id: null
  +email: "test@subscriber.io"
  +first_name: "Test"
  +last_name: "Subscriber"
  +full_name: null
  +subscribed_at: null
  +tags: Spatie\...\DataCollection {#1513 ▼
    -through: null
    -filter: null
    -items: Illuminate\...\Collection {#1905 ▼
      #items: array:2 [▼
        0 => Domain\...\TagData {#1911 ▼
          #includes: []
          #excludes: []
          #inclusionTree: null
          #exclusionTree: null
          #additional: []
          +id: 1
          +title: "Laravel"
          +subscribers: null
        }
        1 => Domain\...\TagData {#1553 ▼
          #includes: []
          #excludes: []
          #inclusionTree: null
          #exclusionTree: null
          #additional: []
          +id: 2
          +title: "Vue"
          +subscribers: null
        }
      ]
      #escapeWhenCastingToString: false
    }
    -dataClass: "Domain\Subscriber\DataTransferObjects\TagData"
    #includes: []
    #excludes: []
    #inclusionTree: null
    #exclusionTree: null
  }
  +form: Domain\...\FormData {#1887 ▼
    #includes: []
    #excludes: []
    #inclusionTree: null
    #exclusionTree: null
    #additional: []
    +id: 2
    +title: "Join The Newsletter"
    +content: "<html><head><title>Ullam magnam iste et possimus saepe
  }
}

```

Now let's see the `FormData` class:

```
namespace Domain\Subscriber\DataTransferObjects;

use Spatie\LaravelData\Data;

class FormData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $title,
        public readonly string $content,
    ) {}
}
```

It only has a title and content property (the HTML of the form itself). The `TagData` is even more simple:

```
namespace Domain\Subscriber\DataTransferObjects;

use Spatie\LaravelData\Data;

class TagData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $title,
    ) {}
}
```

It only has a title. As you can see, every DTO has an optional ID property. Now that you have seen the basic data structure, let's move on to the action that creates a new subscriber:

```

namespace Domain\Subscriber\Actions;

class UpsertSubscriberAction
{
    public static function execute(
        SubscriberData $data,
        User $user
    ): Subscriber {
        $subscriber = Subscriber::updateOrCreate(
            [
                'id' => $data->id,
            ],
            [
                ... $data->all(),
                'form_id' => $data->form?->id,
                'user_id' => $user->id,
            ],
        );

        $subscriber->tags()->sync($data->tags->toCollection()->pluck('id'));

        return $subscriber->load('tags', 'form');
    }
}

```

There are two common patterns:

- I use the name `upsert` for almost all of my actions. It can be used to create or update a subscriber.
- In these kinds of actions, I always use Eloquent's `updateOrCreate` method. It takes two arrays:
 - The first one is used in a `select` query. If a record is found with these values, it will run an `update` query. If it's not found, it will run an `insert` query.
 - The second one contains the attributes that will be inserted or updated.

Every DTO has an `all` method that returns the properties as an array. This is a handy method and can be used with Eloquent, as in this example. But when you have nested properties, the array looks like this:

```
[
    'first_name' => 'John',
    'last_name' => 'Doe',
    'form' => [
        'id' => 1,
        'title' => 'Some Form',
        'content' => ' ... ',
    ],
]
```

Obviously, we can't save the whole form array; instead, we only want to store the `id`. This is why the second parameter of `updateOrCreate` is constructed like this:

```
[
    ... $data->all(),
    'form_id' => $data->form?->id,
    'user_id' => $user->id,
];
```

Since PHP 8, the `...` operator can be used with associative arrays, so the example above is equivalent to this:

```
[
    array_merge(
        $data->all(),
        [
            'form_id' => $data->form?->id,
            'user_id' => $user->id,
        ],
    );
];
```

Using `...`, we have a much cleaner solution. I will use this technique a lot. There are three other important things in this action:

- The `id` of the `SubscriberData` (and any other DTO) is nullable. So when it contains a new subscriber, the `updateOrCreate` will get `null` as the ID, and it will run an `insert` query.
- The `$form` is a nullable property in the `SubscriberData` class, so I use the `?` operator. If the form is null, it won't throw an exception but instead, use a `null` value. It's also a PHP8 feature.

And the third important thing is this line:

```
$subscriber->tags()->sync(  
    $data->tags->toCollection()->pluck('id')  
);
```

If you remember, the `$tags` property in the `SubscriberData` class is a `DataCollection`. This class comes from the `laravel-data` package, and it can be converted into a Laravel collection by calling the `toCollection` method. Since `tags` is a `belongsToMany` relationship in the `Subscriber` model, we can use the `sync` method:

- It will attach every tag from the given collection.
- And detach every other tag that the subscriber had earlier (in case of an update).

Updating A Subscriber

Now let's see how the `update` method looks in the `SubscriberController`:

```
public function update(
    SubscriberData $data,
    Request $request
): RedirectResponse {
    UpsertSubscriberAction::execute($data, $request->user());

    return Redirect::route('subscribers.index');
}
```

As you can see, it's the same as the `store`. That's because the action takes care of both actions. Why doesn't it have a `Subscriber $subscriber` argument from a route binding?

- The frontend will send the ID in the request.
- The `SubscriberData` loads this ID alongside the other attributes.
- The `updateOrCreate` in the action will run an `update` query.

So it doesn't need a `Subscriber` parameter; the DTO takes care of everything.

View Models

To better understand creating and updating a subscriber, we need to take a closer look at view models. As I said in the introduction, these classes contain data that the frontend needs. The only place they are used in the backend is the controller:

```
namespace App\Http\Web\Controllers\Subscriber;

use Inertia\Response;

class SubscriberController
{
    public function create(): Response
    {
        return Inertia::render('Subscriber/Form', [
            'model' => new UpsertSubscriberViewModel(),
        ]);
    }

    public function edit(Subscriber $subscriber): Response
    {
        return Inertia::render('Subscriber/Form', [
            'model' => new UpsertSubscriberViewModel($subscriber),
        ]);
    }
}
```

If you remember from the good old Blade days, `create` and `edit` renders the create and edit page for a subscriber. But instead of rendering a Blade view, they are returning an `Inertia\Response` instance. This is the only difference when using inertia. The `Subscriber/Form` is the path of a Vue component that contains the subscriber form. The second argument is the data we want to pass to this component as its props. In both cases, I give an instance of the `UpsertSubscriberViewModel`. Here's what this class looks like:

```

namespace Domain\Subscriber\ViewModels;

use Domain\Shared\ViewModels\ViewModel;

class UpsertSubscriberViewModel extends ViewModel
{
    public function __construct(public readonly ?Subscriber $subscriber = null)
    {
    }

    public function subscriber(): ?SubscriberData
    {
        if (!$this->subscriber) {
            return null;
        }

        return SubscriberData::from(
            $this->subscriber->load('tags', 'form')
        );
    }

    /**
     * @return Collection<TagData>
     */
    public function tags(): Collection
    {
        return Tag::all()->map(fn (Tag $tag) => TagData::from($tag));
    }

    /**
     * @return Collection<FormData>
     */
    public function forms(): Collection
    {
        return Form::all()->map(fn (Form $form) => FormData::from($form));
    }
}

```

```
}
```

The view model queries every tag and form from the database because we need to display them in a dropdown so users can choose from them.

Four essential things apply to most view models:

- Since this class handles both the create and the edit page, the subscriber can be `null`. When creating a new one, it will be null, so the input fields on the form will all be empty. When updating an existing one, it will have a value, so the input values on the form will be populated.
- Every method returns a data class. Now you have a better understanding of DTOs:
 - They are being used when handling incoming requests.
 - They are being used when returning responses.
 - Basically, DTOs are the "single source of truth" when it comes to data.
- Every method we want to access on the frontend as a data property (more on that in a minute) must be public.
- There is no `get` prefix in the method names. Two reasons for this:
 - **A view model only has getters**, so there's no reason to use a prefix.
 - These methods will be converted into an array, and they become properties of the Vue component.

When returning a view model from a controller, we want an array such as:

```
[
  'subscriber' => [
    'first_name' => 'John',
    'last_name' => 'Doe',
    'form' => [
      'id' => 1,
      'title' => 'Join the Newsletter',
      'content' => ' ... ',
    ],
    'tags' => [
      [
        'id' => 1,
        'title' => 'Laravel',
      ],
      [
        'id' => 2,
        'title' => 'Vue',
      ],
    ],
  ],
  'forms' => [
    // Every form from the database for the dropdown
  ],
  'tags' => [
    // Every tag from the database for the dropdown
  ],
];
```

In a moment, I will show you how we can do this. For now, imagine we have this array from the view model. When we use this in the controller by this line:

```
return Inertia::render('Subscriber/Form', [
  'model' => new UpsertSubscriberViewModel(),
]);
```

Inertia will pass this array to the Vue component, where we can accept it as a prop:

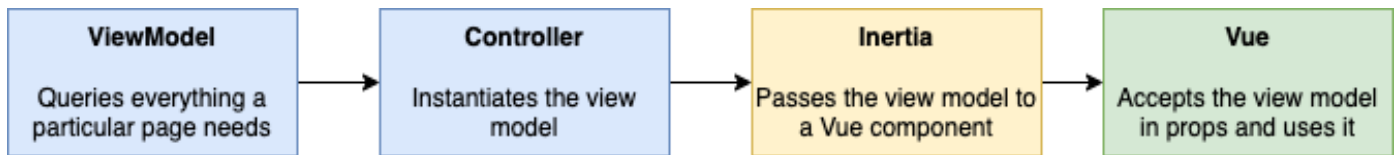
```
export default {
  props: {
    model: {
      type: Object,
      required: true,
    },
  },
}
```

And the result:



This screenshot was taken on an edit form (the ID is not null). Of course, you don't have to use the root level `model` key, but I find it very useful.

Here's a fancy figure to help you understand the whole flow:



From this flow, you can also see that Inertia is not a framework. It's a library that glues Laravel and Vue together.

Now, let's see the magic that creates an array from methods:

```

namespace Domain\Shared\ViewModels;

abstract class ViewModel implements Arrayable
{
    public function toArray(): array
    {
        return collect((new ReflectionClass($this))>getMethods())
            ->reject(fn (ReflectionMethod $method) =>
                in_array($method->getName(), ['__construct', 'toArray'])
            )
            ->filter(fn (ReflectionMethod $method) =>
                in_array(
                    'public',
                    Reflection::getModifierNames($method->getModifiers())
                )
            )
            ->mapWithKeys(fn (ReflectionMethod $method) => [
                Str::snake($method->getName()) => $this->{$method->getName()}()
            ])
            ->toArray();
    }
}
  
```

This method is defined in the parent `ViewModel` class that every view model extends. We only need to implement the `Arrayable` interface, and Laravel will take care of the rest. I don't want to go into too many details about how the reflection API works, but the main logic is this:

- We want every method from the view model except `__construct` and `toArray`.
- We also want to reject every `private` or `protected` method. Only `public` methods represent data properties.
- We want the array keys to be in `snake_case`. So a method called `automationSteps` will become `automation_steps` in the array and the Vue component.

This is how the view model gets converted into an array and passed to the Vue component as a property.

Vue Component

This book is about backend, but I'll also show you some Vue examples. So let's see how the component handles the view model.

First, it accepts it as a prop:

```
export default {  
  props: {  
    model: {  
      type: Object,  
      required: true,  
    },  
  },  
}
```

Then I define a `form` key in the data object:

```
data() {  
  return {  
    form: {  
      id: null,  
      first_name: null,  
      last_name: null,  
      email: null,  
      form_id: null,  
      tag_ids: [],  
    },  
  };  
}
```

I set everything to null, and in the `created` hook, I initialize these properties:

```

created() {
  if (!this.model.subscriber) {
    return;
  }

  this.form = {
    id: this.model.subscriber.id,
    first_name: this.model.subscriber.first_name,
    last_name: this.model.subscriber.last_name,
    email: this.model.subscriber.email,
    form_id: this.model.subscriber.form.id,
    tag_ids: this.model.subscriber.tags.map(t => t.id),
  };
}

```

I do this in the `created` method because the `model.subscriber` can be null, so, unfortunately, it won't work in the `data`.

I use the `form` object for data bindings:

```

<div>
  <label for="first_name">
    First Name
  </label>
  <input v-model="form.first_name" id="first_name" type="text"
placeholder="Jane">
</div>

```

The last piece of the puzzle is the `submit` function:

```

methods: {
  submit() {
    if (this.model.subscriber) {
      this.$inertia.put(
        `/subscribers/${this.model.subscriber.id}`,
        this.form
      )
    } else {
      this.$inertia.post('/subscribers', this.form)
    }
  },
}

```

If there's a subscriber, it will call the PUT endpoint (the `update` method in the controller); otherwise, it calls the POST endpoint (the `store` method in the controller). The `this.$inertia.post` is very similar to `await axios.post` but it will do a page reload after getting the response. This is why it's synchronous. And it's perfect for the use case because after submitting the subscriber, I want to redirect the user back to the index page. You can see this in the controller:

```

use Illuminate\Http\RedirectResponse;

public function store(
    SubscriberData $data,
    Request $request
): RedirectResponse {
    UpsertSubscriberAction::execute($data, $request->user());

    return Redirect::route('subscribers.index');
}

```

It returns a `RedirectResponse` with the `subscribers.index` route, the subscriber list.

I highly recommend you to go through the last few chapters one more time because it'll be a recurring theme of this book, so it's essential to understand how DTOs and view models work.

Get Subscribers

Let's create a list of subscribers! First, the controller:

```
public function index(Request $request): Response
{
    return Inertia::render('Subscriber/List', [
        'model' => new GetSubscribersViewModel($request->get('page', 1)),
    ]);
}
```

There's no DTO involved here, only a view model. A typical e-mail list can contain subscribers, ranging from a few thousand to tens of thousands of people, so pagination is crucial. This is why you see the `$request->get('page', 1)` is passed to the view model.

There's bad news about pagination: since we want to return DTOs from the view model, we cannot just write something like:

```
return Subscriber::paginate();
```

Because this will return a pagination object, and we cannot manipulate the models inside it. But luckily, it's very easy to use Laravel's pagination features without using the `paginate` method. First of all, the frontend won't list available pages. It only shows:

- Total number of subscribers
- Next button
- Prev button

This is called simple pagination in Laravel. It's easy to achieve this behavior:

```

namespace Domain\Subscriber\ViewModels;

class GetSubscribersViewModel extends ViewModel
{
    private const PER_PAGE = 20;

    public function __construct(private readonly int $currentPage)
    {
    }

    public function subscribers(): Paginator
    {
        /** @var Collection $items */
        $items = Subscriber::with(['form', 'tags'])
            →orderBy('first_name')
            →get()
            →map(fn (Subscriber $subscriber) ⇒
                SubscriberData::from($subscriber)
            );

        $items = $items→slice(self::PER_PAGE * ($this→currentPage - 1));

        return new Paginator(
            $items,
            self::PER_PAGE,
            $this→currentPage,
            [
                'path' ⇒ route('subscribers.index'),
            ],
        );
    }

    public function total(): int
    {
        return Subscriber::count();
    }
}

```

```
}
```

Unfortunately, when creating paginators manually, the Laravel documentation says [here](#), we need to query every item and slice the results before passing them to the `Paginator` class. You can easily fix this behavior by:

- Using `offset` and `limit` in the query.
- Creating a custom pagination data class that contains information such as current page, next page, and so on.

However, I don't want to recreate existing logic, and usually, a user has a few thousand or ten thousand subscribers, so we're not querying millions of records. This is highly subjective, but I would only optimize this when some performance issues occur.

On the frontend, we can create a simple `Pagination` component:

```
<div>
  <div>
    Total: {{ this.total }}
  </div>
  <div>
    <button @click="$emit('paginatedPrev')" :disabled="this.current_page === 1">
      Prev
    </button>
    <button @click="$emit('paginatedNext')">
      Next
    </button>
  </div>
</div>
```

Which can be used as:

```
<Pagination
  :total="model.total"
  :current_page="model.subscribers.current_page"
  @paginated-prev="prevPage()"
  @paginated-next="nextPage()"
></Pagination>
```

And we can use the links given by the backend:

```
methods: {
  nextPage() {
    if (!this.model.subscribers.next_page_url) {
      return;
    }

    this.$inertia.get(this.model.subscribers.next_page_url);
  },
  prevPage() {
    if (!this.model.subscribers.prev_page_url) {
      return;
    }

    this.$inertia.get(this.model.subscribers.prev_page_url);
  },
}
```

Where is the user ID?

This is the query in the `GetSubscribersViewModel`:

```
Subscriber::with(['form', 'tags'])
    →orderBy('first_name')
    →get()
    →map(fn (Subscriber $subscriber) => SubscriberData::from($subscriber));
```

You can see there's no user id in this query. That's because I'm using a global scope that adds a where condition to every query:

```

namespace Domain\Shared\Models\Scopes;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Scope;

class UserScope implements Scope
{
    public function apply(Builder $builder, Model $model)
    {
        if ($user = request()->user()) {
            $builder->whereBelongsTo($user);
        }
    }
}

```

It retrieves the currently logged-in user from the request and adds a where condition to the query. It's being used in a trait called `HasUser`:

```

namespace Domain\Shared\Models\Concerns;

use Domain\Shared\Models\Scopes\UserScope;
use Domain\Shared\Models\User;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

trait HasUser
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    protected static function booted()
    {
        static::addGlobalScope(new UserScope());
    }
}

```

```
}
```

And this trait is being used in the `Subscriber` model:

```
class Subscriber extends BaseModel
{
    use HasUser;
}
```

Now anytime you write a subscriber query, a `where user_id = ?` will be added. Let's discuss the advantages and disadvantages of this solution. First, the disadvantages:

- It's hidden. It's magic. Yes, it is.
- It's harder to understand by newcomers. Yes, it is. However, in my experience, if the project has such a critical logic hidden by a scope (or something similar), it will come up in the first hour of the first day.

Now, the advantages:

- If you don't use this scope, you have to write a where statement in **EVERY QUERY** for the next X years. What happens if you forget it? Users will see other users' subscribers. This is a clear violation of the GDPR and possibly some other laws. Please note that there is an \$877 million GDPR fine because of a poorly implemented cookie contest or a \$255 million one caused by a poor explanation of a data processing practice. \$255 million for a wrong description! And, of course, newcomers will forget to filter by user, and you need to catch these bugs while reviewing their code. Will you catch them all?

For me, the luxury of knowing that my code won't send me to jail is absolutely 100% worth the price of having a little bit of magic in the project and explaining it to new developers.

How does a DTO get created from a model?

In the `GetSubscribersViewModel`, there's this query:

```
Subscriber::with(['form', 'tags'])
    →orderBy('first_name')
    →get()
    →map(fn (Subscriber $subscriber) => SubscriberData::from($subscriber));
```

As you can see, the last line creates a `SubscriberData` from the `subscriber` model by using the `from` method. The `from` method will work with requests and models by default. In both cases, it converts the object to an array (by calling the `all` or the `toArray` method) and tries to populate the DTO's properties from this array. However, we can override this default behavior by defining a `fromModel` method on the `SubscriberData` class:

```

public static function fromModel(Subscriber $subscriber): self
{
    return self::from([
        ... $subscriber->toArray(),
        'tags' => Lazy::whenLoaded(
            'tags',
            $subscriber,
            fn () => TagData::collection($subscriber->tags)
        ),
        'form' => Lazy::whenLoaded(
            'form',
            $subscriber,
            fn () => FormData::from($subscriber->form)
        ),
    ]);
}

```

What is this `Lazy` thing? It's basically the same as the `whenLoaded` in HTTP resources. For example, the `$form` property will only be evaluated if the `form` relationship is loaded in the given `$subscriber` instance. Only then the callback of the `whenLoaded` will be executed. This helps us to avoid N+1 queries. Otherwise, when querying the subscribers and mapping each one to a data object, it would run two extra queries (loading the form and the tags) **for every subscriber**. This is why the `Lazy::whenLoaded` is such a valuable helper. To use this, we need to make one more change:

```

class SubscriberData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $email,
        public readonly string $first_name,
        public readonly ?string $last_name,
        /** @var DataCollection<TagData> */
        public readonly null|Lazy|DataCollection $tags,
        public readonly null|Lazy|FormData $form,
    ) {}
}

```

The `$tags` and the `$form` can be either a `Lazy` or the original type. Remember, when we're creating a DTO from a request, we always have a `FormData` or `DataCollection`:

```
public static function fromRequest(Request $request): self
{
    return self::from([
        ... $request->all(),
        'tags' => TagData::collection(
            Tag::whereIn('id', $request->collect('tag_ids'))->get()
        ),
        'form' => FormData::from(Form::findOrNew($request->form_id)),
    ]);
}
```

Here I'm not using the `Lazy` because this method is always called in a context where I only have a single subscriber, such as:

- Create
- Update
- Delete

In these cases, N+1 query is not a problem. For this reason, I use multi-type properties

`Lazy|DataCollection` or `Lazy|FormData`. If you're not using the `laravel-data` package N+1 queries are still a problem that you need to solve.

DTOs and model attribute accessors

One more challenge we need to solve when it comes to models and DTOs is attribute accessors. First, let's define a `full_name` accessor in the `Subscriber` model:

```
public function fullName(): Attribute
{
    return new Attribute(
        get: fn () => "{$this->first_name} {$this->last_name}",
    );
}
```

It only needs a getter that constructs the full name from the first and last names. If you call the `toArray` method on a model (just as we did in the `fromModel`), you can see there's no `full_name` attribute:

```
=> [
    "id" => 1,
    "user_id" => 1,
    "email" => "frank37@example.net",
    "first_name" => "Lance",
    "last_name" => "Leannon",
    "form_id" => 3,
    "subscribed_at" => "2022-02-23 18:57:43",
    "created_at" => "2022-02-28T18:57:43.000000Z",
    "updated_at" => "2022-02-28T18:57:43.000000Z",
]
```

So when we're calling the `toArray` in the `fromModel` method, the `full_name` attribute won't be included:

```
public static function fromModel(Subscriber $subscriber): self
{
    return self::from([
        ... $subscriber->toArray(),
    ]);
}
```

All we need to do is to include a `full_name` key directly in the array:

```
public static function fromModel(Subscriber $subscriber): self
{
    return self::from([
        ... $subscriber->toArray(),
        'full_name' => $subscriber->full_name,
    ]);
}
```

Now the DTO will have a `full_name` property. This issue is not strictly related to laravel-data; you also need to solve this with your own DTOs (spoiler: the same as we did here).

DTOs and Carbon dates

The last DTO-related question is about Carbon attributes. I have a `subscribed_at` column in the `Subscriber` model. You haven't seen this column in the `UpsertSubscriberAction` because I use the `useCurrent` method in the migration:

```
public function up()
{
    Schema::create('subscribers', function (Blueprint $table) {
        $table->id();
        $table->foreignIdFor(User::class)->constrained()->cascadeOnDelete();
        $table->string('email');
        $table->string('first_name');
        $table->string('last_name')->nullable();
        $table->foreignId('form_id')->nullable()->constrained()->nullOnDelete();
        $table->dateTime('subscribed_at')->useCurrent();
        $table->timestamps();

        $table->unique(['user_id', 'email']);
    });
}
```

This means the `subscribed_at` column will have a default `NOW()` value every time you insert a record. Of course if you override it when calling the `Subscriber::create()` or the `Subscriber::updateOrCreate()` the given value will be inserted.

The good news is `Carbon` cast is included by default; all we need to do is type-hint the property:

```
class SubscriberData extends Data
{
    public function __construct(
        public readonly ?Carbon $subscribed_at,
    ) {}
}
```

Now, every time you pass a string, a Carbon, or a DateTime it will be converted to a Carbon instance. You can also define the date format you want to use in the package's config located in `config/data.php`.

It's important to note that the `subscribed_at` property is nullable because when creating a DTO from a request, there's no `subscribed_at` in the request, so that it will be null. But when creating it from a model, there will be a value. And once again, when you're calling the `$subscriber->toArray()`, the `subscribed_at` will be a string. The `Carbon` type-hint in the `SubscriberData` makes it possible to parse this string and cast it to a Carbon instance.

This date and carbon casting is also a problem when using plain DTOs without any package. By now, I hope you can see why a package like `laravel-data` is so useful to have.

Wrapping it up

I know this chapter was exhausting, but we've talked about the basics, and it's very important to understand them. After all, this is what the `SubscriberData` looks like:

```
namespace Domain\Subscriber\DataTransferObjects;

class SubscriberData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $email,
        public readonly string $first_name,
        public readonly ?string $last_name,
        public readonly ?string $full_name,
        public readonly ?Carbon $subscribed_at,
        /** @var DataCollection<TagData> */
        public readonly null|Lazy|DataCollection $tags,
        public readonly null|Lazy|FormData $form,
    ) {}

    public static function fromRequest(Request $request): self
    {
        return self::from([
            ... $request->all(),
            'tags' => TagData::collection(
                Tag::whereIn('id', $request->collect('tag_ids'))->get()
            ),
            'form' => FormData::from(
                Form::findOrNew($request->form_id)
            ),
        ]);
    }
}
```

```

}

public static function fromModel(Subscriber $subscriber): self
{
    return self::from([
        ... $subscriber->toArray(),
        'tags' => Lazy::whenLoaded(
            'tags',
            $subscriber,
            fn () => TagData::collection($subscriber->tags)
        ),
        'form' => Lazy::whenLoaded(
            'form',
            $subscriber,
            fn () => FormData::from($subscriber->form)
        ),
        'full_name' => $subscriber->full_name,
    ]);
}

public static function rules(): array
{
    return [
        'email' => [
            'required',
            'email',
            Rule::unique('subscribers', 'email')->ignore(request('subscriber')),
        ],
        'first_name' => ['required', 'string'],
        'last_name' => ['nullable', 'sometimes', 'string'],
        'tags' => ['nullable', 'sometimes', 'array'],
        'form_id' => ['nullable', 'sometimes', 'exists:forms,id'],
    ];
}
}

```

The Real Power of DTOs and Actions

The real power of DTOs and actions comes from having multiple components or applications that require the same logic to be reusable. In this application, we have two features that can significantly benefit from actions and DTOs:

- Subscriber import that can be used via the web app and from a console command
- An API for creating a new subscriber

Subscriber API

Both of these features are required in an e-mail marketing application. Let's start with the API. First of all, there's a common misunderstanding that you cannot have APIs when using Inertia, and you cannot have multiple applications such as a web and a mobile app. That's not true, and to be honest, I have no idea why people think that's the case.

First, let's define an API route:

```
use App\Http\Api\Controllers\Subscriber\CreateSubscriberController;

Route::middleware(['api', 'auth:sanctum'])->group(function () {
    Route::post('subscribers', CreateSubscriberController::class);
});
```

As you can see from the namespace, the API lives under the `App\Http\Api` namespace, which is our API application. The controller is straightforward:

```
namespace App\Http\Api\Controllers\Subscriber;

class CreateSubscriberController
{
    public function __invoke(
        SubscriberData $data,
        Request $request
    ): SubscriberData {
        $subscriber = UpsertSubscriberAction::execute($data, $request->user());

        return $subscriber->getData();
    }
}
```

This is very simple; it's almost identical to the `SubscriberController::store` method. It's an invokable controller, which means it only has one method; there's no store, update, create, or anything like that. The whole controller is responsible for creating a new subscriber and nothing else. I use this type of controller because the API doesn't need any other functionality. So this is a perfect opportunity to use an invokable controller.

What is the `$subscriber->getData()` method? In the previous chapters, you have seen this snippet multiple times:

```
Subscriber::with(['form', 'tags'])
    ->orderBy('first_name')
    ->get()
    ->map(fn (Subscriber $subscriber) => SubscriberData::from($subscriber));
```

Here we map through the subscribers and create a `SubscriberData` inside the callback. We need to do this so frequently that laravel-data provides the `getData` helper function to achieve the same. We need to tell the package what data class belongs to a model:

```
namespace Domain\Subscriber\Models;

use Domain\Subscriber\DataTransferObjects\SubscriberData;

class Subscriber extends BaseModel
{
    use WithData;
    use HasUser;

    protected $dataClass = SubscriberData::class;
}
```

After that, we can use the `getData` on any `Subscriber` instance, and it will return a `SubscriberData` instance.

Basically, that's it; we have an API that can be called with the same request as the web route:

```
{
  "email": "test@test.io",
  "first_name": "Test",
  "last_name": "User",
  "tag_ids": [1,2],
  "form_id": 1
}
```

And you get a `SubscriberData` as the response:

```
{
  "id": 214,
  "email": "test@test.io",
  "first_name": "Test",
  "last_name": "User",
  "full_name": "Test User",
  "subscribed_at": "2022-05-05 15:50:34",
  "tags": [
    {
      "id": 1,
      "title": "Laravel"
    },
    {
      "id": 2,
      "title": "Vue"
    }
  ],
  "form": {
    "id": 1,
    "title": "Waiting List",
    "content": "<html> ... </html>"
  }
}
```

Importing subscribers from CSV

The CSV import is the next feature where DTOs and actions are beneficial. We want to import a CSV like this:

email	first_name	last_name	tags
john@doe.io	John	Doe	Laravel,Vue,nodejs
jane@doe.io	Jane	Doe	PHP,Inertia
james@doe.io	James	Doe	

It's a really simple CSV that contains a comma-separated list of tags. Of course, we need a new action, the `ImportSubscribersAction`:

```
namespace Domain\Subscriber\Actions;

use Domain\Shared\Actions\ReadCsvAction;

class ImportSubscribersAction
{
    public static function execute(string $path, User $user): void
    {
        ReadCsvAction::execute($path)
            →each(function (array $row) use ($user) {
                $parsed = [
                    ... $row,
                    'tags' => self::parseTags($row, $user),
                ];

                $data = SubscriberData::from($parsed);

                if (self::isSubscriberExist($data, $user)) {
                    return;
                }

                UpsertSubscriberAction::execute($data, $user);
            });
    }
}
```

Let's see what's happening step-by-step:

- The `ReadCsvAction` reads the given CSV file into a Collection of arrays. It returns a Collection that contains arrays. Every array represents a row from the CSV, so it has the following shape:

```
[
  'email' => 'john@doe.io',
  'first_name' => 'John',
  'last_name' => 'Doe',
  'tags' => 'Laravel,Vue,nodejs',
]
```

- I won't list the `ReadCsvAction` here. It's a very basic script that uses PHP's native `fgetcsv` function and a `while` loop to read the file's contents.
- After reading the CSV, the action goes through every item and parses the tags. It means:
 - Reading the individual tags from the comma-separated string.
 - Querying the existing tags.
 - Creating non-existing tags.
 - So the `tags` key will contain actual `Tag` models.
 - In a minute, I'll show you the implementation.
- The line `SubscriberData::from($parsed)` will create a `SubscriberData` from the given array. Remember, if the argument of the `from` method is an array, laravel-data tries to map every array key to a DTO property. This is why we created `Tag` models from the strings.
- The `isSubscriberExist` method checks if a subscriber exists in the database or not. We don't want to import existing subscribers.
- And finally, we call the `UpsertSubscriberAction`.

Here's how the `parseTags` method works:

```

/**
 * @param string[] $row
 * @param User $user
 * @return Tag[]
 */
private static function parseTags(array $row, User $user): array
{
    $tags = collect(explode(',', $row['tags']))
        ->filter()
        ->toArray();

    return self::getOrCreateTags($tags, $user);
}

```

It explodes the string by commas, so we get an array like this: `['Laravel', 'vue']`. If you call the `filter` without any callback, it will filter out `null` values from the collection, which is very helpful in this case. After that, we are ready to query existing tags and insert new ones:

```

/**
 * @param string[] $tags
 * @param User $user
 * @return Tag[]
 */
private static function getOrCreateTags(array $tags, User $user): array
{
    return collect($tags)
        ->map(fn (string $title) => Tag::firstOrCreate([
            'title' => $title,
            'user_id' => $user->id,
        ]))
        ->toArray();
}

```

The `firstOrCreate` will run a `select` query if a record is found by the given attributes, or it will run an `insert` query if it's not found. This action will run from the command line where we don't have a request, so this is why the array has a `user_id` key (if you remember, the `UserScope` works only if there's a request).

Finally, the `isSubscriberExist` function is quite simple:

```
private static function isSubscriberExist(SubscriberData $data, User $user):
bool
{
    return Subscriber::query()
        ->whereEmail($data->email)
        ->whereBelongsTo($user)
        ->exists();
}
```

Now that the `ImportSubscriberAction` is ready, we can use it from a command:

```
namespace App\Console\Commands\Subscriber;

use Domain\Subscriber\Jobs\ImportSubscribersJob;

class ImportSubscribersCommand extends Command
{
    protected $signature = 'subscriber:import {user? : The ID of the user}';
    protected $description = 'Import subscribers from csv';

    public function handle()
    {
        $userId = $this->argument('user') ?? $this->ask('User ID');

        ImportSubscribersJob::dispatch(
            storage_path('subscribers/subscribers.csv'),
            User::findOrFail($userId),
        );

        $this->info("Subscribers are being imported ...");

        return self::SUCCESS;
    }
}
```

There are three things worth mentioning here:

- I like to use only optional arguments in commands. If it's `null`, I call the `ask` method to ask for input. This way, you can call any command without thinking about arguments. The command will ask them.
- There's an `ImportSubscribersJob`. The import is being queued because it can be a heavy and long-running task in the case of a large CSV file.
- The command is located in the `App\Console` namespace where the console application lives.

The `ImportSubscriberJob` itself is very simple:

```
namespace Domain\Subscriber\Jobs;

use Domain\Subscriber\Actions\ImportSubscribersAction;

class ImportSubscribersJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(
        private readonly string $path,
        private readonly User $user,
    ) {}

    public function handle()
    {
        ImportSubscribersAction::execute($this->path, $this->user);
    }
}
```

It calls the import action, and that's it. This class is required only to be able to queue the import process. It's inconvenient to have one more class, but this is how Laravel works, so there's not much to do.

Now let's see how we can use this import action from the web application. As I showed you earlier, I used a resource controller for the subscribers. Now I need to add an import functionality that is not part of the standard resource methods. For these features, I like to use invokable controllers. So the routes look like this in the `web.php`:

```

use App\Http\Web\Controllers\Subscriber\ImportSubscribersController;
use App\Http\Web\Controllers\Subscriber\SubscriberController;

Route::middleware(['auth', 'verified'])->group(function () {
    Route::resource('subscribers', SubscriberController::class);
    Route::post('subscribers/import', ImportSubscribersController::class);
});

```

I find this technique suitable for most of my projects. Of course, there's the disadvantage of having both resource and invokable controllers simultaneously, which can be annoying. What are the alternatives?

- You only have invokable controllers. Personally, I don't like this approach too much. One of the disadvantages of using DDD is that you have many classes. If you're using invokable controllers, you have even more of them. And controllers, actions, commands, and jobs will have very similar names, which can be hard to navigate.
- You only have resource controllers, and you write custom methods. So you have a `SubscriberController` with an `import` method. This works fine for the first few custom methods. After that, controllers will grow very large very soon. So I try to avoid this approach. A resource controller has seven methods by default. In my opinion, that's enough.

So this is why I'm using both resource and invokable controllers. As you can expect, the `ImportSubscribersController` is very simple:

```

namespace App\Http\Web\Controllers\Subscriber;

use Domain\Subscriber\Jobs\ImportSubscribersJob;

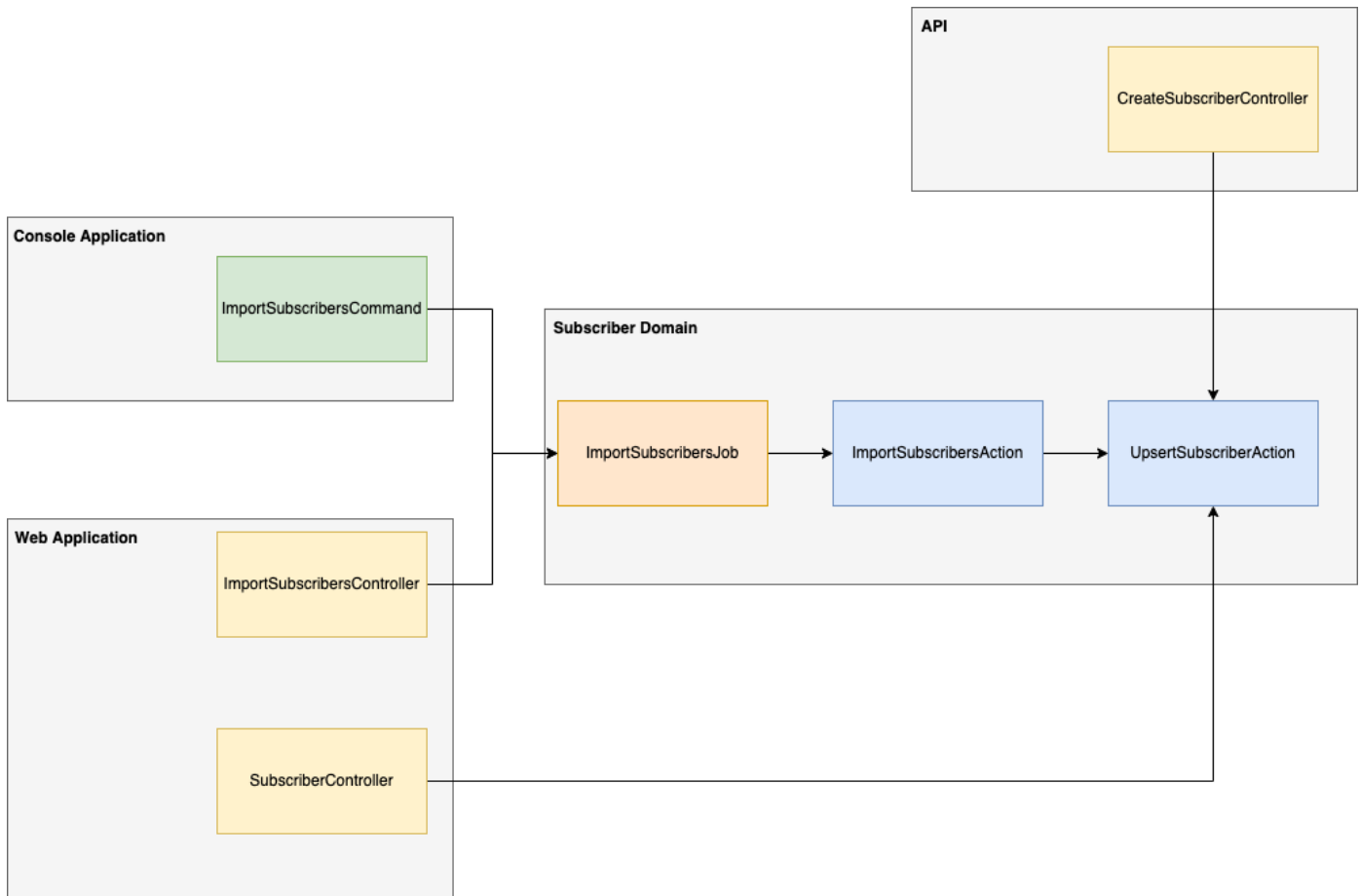
class ImportSubscribersController
{
    public function __invoke(Request $request): Response
    {
        ImportSubscribersJob::dispatch(
            storage_path('subscribers/subscribers.csv'),
            $request->user(),
        );

        return response('', Response::HTTP_ACCEPTED);
    }
}

```

What does it do? It dispatches the job. What does the job do? It calls the action. I think you got the point. Actions are great. DTOs are great. Put them together, and you have a superpower.

A lot happened here, so let's recap what is going on:



Conclusion

In this chapter, we've used the most important DDD concepts:

- Data transfer objects to avoid using unstructured arrays.
- View models to separate the queries from the commands and encapsulate a single page or view of the application.
- Actions to write reusable classes that encapsulate a single user story.

But we haven't used the following classes:

- Requests
- Resources
- Services
- Repositories

We've solved these problems a bit differently. To be more specific:

Other possible solution	Our solution
Requests	DTOs
Resources	DTOs
Services	View models / Actions
Repositories	View models / Actions

You might be thinking this structure has a significant disadvantage: right now, there's no way to reuse single queries. For example, what do we do if we need to query all the subscribers? Just like we did for the list, but this time it's going to be used in export, for example. Right now, if you want to do something like that, you need to instantiate the whole `GetSubscribersViewModel` and call the `subscribers` method. It's not optimal. But even worse, it will return a `Paginator` instead of a `Collection`. Don't worry; we will write reusable queries using custom Query Builders. But for now, they were not needed. By the way, I like the idea that the `subscribers` function of the `GetSubscribersViewModel` returns a `Paginator`. This class is specific to this very page, and it needs pagination.

Broadcasts

As a quick reminder, broadcasts and sequences live under the Mail domain. In this case, I'll use sub-namespaces, for example:

```
Domain
  Mail
    Models
      Broadcast
        Broadcast.php
      Sequence
        Sequence.php
        SequenceMail.php
```

The Broadcast DTO

Now that the app can manage subscribers, it's time to move on to broadcasts. They are the most straightforward type of e-mail. A broadcast can be sent at any time to any subset of subscribers (using filters). First, let's see the `BroadcastData` class:

```
namespace Domain\Mail\DataTransferObjects\Broadcast;

use Domain\Mail\Enums\Broadcast\BroadcastStatus;
use Domain\Mail\DataTransferObjects\FilterData;
use Spatie\LaravelData\Attributes\WithCast;
use Spatie\LaravelData\Casts\EnumCast;

class BroadcastData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $subject,
        public readonly string $content,
        public readonly ?FilterData $filters,
        public readonly ?Carbon $sent_at,
        #[WithCast(EnumCast::class)]
        public readonly ?BroadcastStatus $status = BroadcastStatus::Draft,
    ) {}
```

```
}
```

This is the whole class; there's no `fromRequest` or `fromModel` or `rules`. It's quite simple, but there are some things worth discussing:

- Every broadcast can have filters. The target audience can be narrowed by specifying forms or tags. For example, if you want to send a broadcast to subscribers who come from the 'Waiting List' form, you can do that with a simple form filter. I'll show you the `FilterData` class in a minute.
- The `withCast` attribute can be used to cast properties. In this example, the `EnumCast` is used. It tries to create a `BroadcastStatus` enum from a string. Consider the following example:

```
$data = new BroadcastData(
    subject: 'My Awesome Broadcast',
    content: '<html>Content</html>',
    status: 'draft',
);
```

- In this case, the `EnumCast` tries to create an enum from the string `draft`. Of course, this will only work with backed enums, and a `case` with the value `draft` must exist.
- The `status` property has a default `draft` value. There's no `status` input field on the create form, so we need a default value. Every broadcast is a draft by default. You can only "change" the status by sending the broadcast to subscribers. The status will be set to `sent` when you do that.

There are no validation rules. It's because a broadcast has only three rules:

- The subject is required.
- The content is required.
- Status must be a valid `BroadcastStatus` value.

If you think about it, all of these rules are expressed implicitly in the constructor: there is no `?` before `$subject` or `$content`, and there's a `BroadcastStatus` type-hint. Fortunately, laravel-data provides us with these default, implicit validations. So if you try to submit a broadcast without a title, it won't throw you a type error (null given instead of string) but a valid response with the validation errors. The same applies to the status.

Speaking of the status, this is the enum:

```
namespace Domain\Mail\Enums\Broadcast;

enum BroadcastStatus: string
{
    case Draft = 'draft';
    case Sent = 'sent';
}
```

I like to use `PascalCase` instead of `YELLING_CASE`. It feels better for me.

Handling Filters

The next interesting thing is the `FilterData`:

```
namespace Domain\Mail\DataTransferObjects;

class FilterData extends Data
{
    public function __construct(
        public readonly array $form_ids = [],
        public readonly array $tag_ids = [],
    ) {}
}
```

As you can see, it accepts tag IDs and form IDs so the request will look like this:

```
{
    "subject": "My Awesome Broadcast",
    "filters": {
        "form_ids": [1],
        "tag_ids": [4,9]
    }
}
```

As discussed earlier, the `filters` column's type is JSON, so the whole object with the two arrays will be saved. This can cause some problems, for example:

```
$broadcast = Broadcast::first();  
echo $broadcast->filters;  
  
// Output is this string:  
'{"tag_ids": [1], "form_ids": [4,9]}'
```

Of course, we can use an `array` cast in the `Broadcast` model to have an array:

```
[  
    "tag_ids" => [1],  
    "form_ids" => [4,9],  
]
```

But the whole point of using DTOs (or at least partly) is to avoid working with unstructured arrays. Fortunately, in Laravel, we can write our own cast. This is the `FiltersCast`:

```

namespace Domain\Mail\Models\Casts;

class FiltersCast implements CastsAttributes
{
    public function get($model, string $key, $value, array $attributes):
    FilterData
    {
        $filterArray = json_decode($value, true);

        return $filterArray
            ? FilterData::from($filterArray)
            : FilterData::from(FilterData::empty());
    }

    /**
     * @param FilterData $value
     */
    public function set($model, string $key, $value, array $attributes)
    {
        return [
            'filters' => json_encode($value),
        ];
    }
}

```

It has two methods:

- `get` will run when we access the property. It decodes the raw JSON data and returns a new `FilterData` instance. I'll explain what is the `FilterData::empty` in just a minute.
- `set` will run when we set the property. As you can see in the doc block, the `$value` is a `FilterData` in this case.

Now that we have the cast, we need to use it in the `Broadcast` model:

```

namespace Domain\Mail\Models\Broadcast;

use Domain\Mail\DataTransferObjects\Broadcast\BroadcastData;
use Domain\Mail\Models\Casts\FiltersCast;
use Domain\Mail\Enums\Broadcast\BroadcastStatus;
use Domain\Mail\DataTransferObjects\FilterData;

class Broadcast extends BaseModel
{
    use WithData;
    use HasUser;

    protected $dataClass = BroadcastData::class;

    protected $casts = [
        'filters' => FiltersCast::class,
        'status' => BroadcastStatus::class,
    ];

    protected $attributes = [
        'status' => BroadcastStatus::Draft,
    ];
}

```

By using this cast, we can do this:

```

$broadcast->filters = FilterData::from($request->filters);
echo $broadcast->filters;

// Output will be a FilterData instance

```

You might be thinking: "Why to bother with a separate cast class? The same can be done using an attribute accessor, right?". Yes, it's true. It's possible to use an accessor something like:

```

class Broadcast extends BaseModel
{
    public function filters(): Attribute
    {
        return new Attribute(
            get: fn (string $value) => FilterData::from(json_decode($value, true)),
            set: fn (FilterData $value) => json_encode($value),
        );
    }
}

```

It works the same way, just as the custom cast. Although I like the attribute better, we know that an e-mail in a sequence will also have filters. So by moving this logic into a cast class, we can reuse it later in the sequence mail model. And in any other future class that needs filters.

One last note before moving on. As you can see, the `FilterData` has no ID property. This is obvious since a filter is not an "entity" or a model. It's just a container for tag and form IDs. In the introduction, I said that the main difference between a value object and a DTO is that a DTO always has an ID, but a value object never has one. I also said that I would violate this rule. Why did I do that?

- In the context of Larave and laravel-data, using a `Data` class is so much easier to handle filters.
- Nesting the `FiltersData` inside the `BroadcastData` works automatically.
- laravel-data will handle the casting automatically. We can just do this:
`BroadcastData::from($request->all())` and it will work.
- We can inject the data class in a controller method (as we did earlier), and it will work.
- Validation is handled out-of-the-box if we need some.

So in this particular example, I'll go with the practical solution instead of blindly following a "DDD rule". As a bonus: if you need to refactor the filters and use a `filters` table instead of a single column (as discussed in the design chapter), it will be easier if you already have a `FilterData` class.

Upserting A Broadcast

Now, let's do something with that DTO! The broadcast form looks like this:

New Broadcast

SUBJECT

My Awesome Broadcast

CONTENT

HTML content

FILTERS

FORMS

Free Tutorial
Join The Newsletter
Waiting List

TAGS

Laravel
Vue
MySQL
Inertia
DDD

Save Cancel

It has three attributes:

- Subject. It's going to be the subject line in the e-mail.
- Content. This is the actual HTML content.
- Filters. You can filter the audience using forms and tags.

The controller is easy as usual:

```

namespace App\Http\Web\Controllers\Mail\Broadcast;

use Domain\Mail\Actions\Broadcast\UpsertBroadcastAction;

class BroadcastController
{
    public function store(
        BroadcastData $data,
        Request $request
    ): RedirectResponse {
        UpsertBroadcastAction::execute($data, $request->user());

        return Redirect::route('broadcasts.index');
    }

    public function update(
        BroadcastData $data,
        Request $request
    ): RedirectResponse {
        UpsertBroadcastAction::execute($data, $request->user());

        return Redirect::route('broadcasts.index');
    }
}

```

This should be familiar by now. Every controller follows this pattern:

- Injecting a DTO in the method.
- Calling an action.
- Returning some response.

The `UpsertBroadcastAction` is one of the most simple actions:

```

namespace Domain\Mail\Actions\Broadcast;

use Domain\Mail\DataTransferObjects\Broadcast\BroadcastData;
use Domain\Mail\Models\Broadcast\Broadcast;
use Domain\Shared\Models\User;

class UpsertBroadcastAction
{
    public static function execute(BroadcastData $data, User $user): Broadcast
    {
        return Broadcast::updateOrCreate(
            [
                'id' => $data->id,
            ],
            [
                ...$data->all(),
                'user_id' => $user->id,
            ],
        );
    }
}

```

The same logic as earlier:

- Determining if a broadcast exists with the ID in the DTO.
- Updating or inserting the properties.
- Saving the `user_id` explicitly.
- Returning the upserted broadcast.

These controller actions (store and update) don't require a view model. They just take care of inserting or updating a broadcast. However, create and edit need a view model (just as earlier with subscribers):

```

class BroadcastController
{
    public function create(): Response
    {
        return Inertia::render('Broadcast/Form', [
            'model' => new UpsertBroadcastViewModel(),
        ]);
    }

    public function edit(Broadcast $broadcast): Response
    {
        return Inertia::render('Broadcast/Form', [
            'model' => new UpsertBroadcastViewModel($broadcast),
        ]);
    }
}

```

The view model is very simple:

```

class UpsertBroadcastViewModel extends ViewModel
{
    use HasTags;
    use HasForms;

    public function __construct(public readonly ?Broadcast $broadcast = null)
    {
    }

    public function broadcast(): ?BroadcastData
    {
        if (!$this->broadcast) {
            return null;
        }

        return $this->broadcast->getData();
    }
}

```

```
}
```

There are two new things here:

- HasTags
- HasForms

Since a lot of view models will need to query tags and forms, I extracted these methods into two traits:

```
namespace Domain\Shared\ViewModels\Concerns;

trait HasTags
{
    /**
     * @return Collection<TagData>
     */
    public function tags(): Collection
    {
        return Tag::all() ->map->getData();
    }
}

trait HasForms
{
    /**
     * @return Collection<FormData>
     */
    public function forms(): Collection
    {
        return Form::all() ->map->getData();
    }
}
```

These traits live inside the shared domain, under the `Domain\Shared\ViewModels\Concerns` namespace. They are also being used in subscriber-related view models.

Filtering Subscribers

The next few chapters are going to be fun. Here are some of the techniques we'll use:

- A `Pipeline` that takes care of filtering subscribers.
- An `Enum` that can be used as a factory.
- Some polymorphic classes that implement the strategy design pattern.
- A `Mailable` class that can send any HTML content.
- Queue jobs to handle e-mail sending.

Without further hype, let's jump into the code. To send broadcasts, first, we need to filter the subscribers based on the filters in the broadcast. Let's define the relationship between filters. If you have the following setup:

- Form IDs: 1
- Tag IDs: 10,12

then the following query will run:

```
where
    subscribers.form_id IN (1)
and
    subscriber_tag.tag_id IN (10,12)
```

Or in other words:

- There's an `AND` relation between the form and the tag filters
- There's an `OR` relation between the IDs. Writing `tag_id IN (1,2)` is the same as writing `tag_id = 1 OR tag_id = 2`

Filters are a topic that can require many changes in the future. Just think about some of these features:

- What if the app has some e-commerce features? For example, creators can publish products on the platform, and subscribers can buy them if they're interested. Now we have filters like: who purchased a product, who used a discount code, how much a subscriber paid for products, etc.
- The app does not have this feature right now, but what about custom fields for subscribers? In this case, users can create any fields they want and populate via some automation rules, by hand, or by CSV import. All of these fields can be used in filters.
- Geolocation based on IP addresses?
- What if we need to integrate with 3rd parties? Maybe these integrations also bring some new interesting filter opportunities.

We clearly need a solution that can handle new use-cases easily. In my opinion, the strategy pattern is our best choice here. In this pattern, every filter has a dedicated class, and there's some kind of abstraction. Either an abstract class or an interface. As of now, we need three classes:

- `Filter`: This is the abstract class or interface that defines the contract of a concrete filter class.
- `TagFilter`: This class takes care of the tag filters.
- `FormFilter`: And finally, a class for form filters.

These filters can be expressed as one query, so it seems logical to me that both of these classes will append `where` clauses to a base query. Something like that:

```
public function filterSubscribers(Broadcast $broadcast)
{
    // $query is a basic select * from subscribers
    $query = Subscriber::query();

    /*
     * At this point $query contains something like:
     * select * from subscribers
     * where subscribers.form_id IN (1)
     */
    (new FormFilter($broadcast->filters))>filter($query);

    // Finally TagFilter will join the subscriber_tag table and add another
    where clause
    (new TagFilter($broadcast->filters))>filter($query);
}
```

It's just pseudo-code, but let's think about the idea. Can this work in other use cases?

Custom subscriber fields

I think this can be implemented with two extra tables:

- `fields`: it contains every user-defined field with types (such as string, number, JSON) and other meta information.
- `subscriber_field`: a pivot table where every row represents the value of a custom field for a specific subscriber.

If you think about it, this is very similar to tags. It is also implemented with two tables: `tags` and `subscriber_tag`. So if we can handle tags with this pattern, I think custom fields can also be handled.

Geolocation

In this situation, we probably will have some new columns in the `subscribers` table, such as:

- IP address
- Country
- Region
- City

Or maybe a new table to handle multiple IP addresses, multiple locations. But the point is: it's pretty easy.

3rd party integrations

This is very hard to predict, but basically, we have two choices:

- Calling the 3rd party API every time before sending a broadcast. This is clearly a very poor implementation. Imagine calling an API 5000 times before sending a broadcast to 5000 subscribers.
- Storing the "external" data in our database. For example, integrating with an e-commerce provider and we want to store every product purchase made by our subscribers. In this case, it's just another table with a `subscriber_id` that can be joined and filtered with a `where` clause.

E-commerce features

It's also tough to predict, but if we're talking about products and purchases, it's essentially the same as storing the purchases given by a 3rd party API.

In my opinion, all of those cases can be implemented by adding a new `Filter` class such as the `FormFilter` or the `TagFilter`. The main difference will be the data. In the case of forms or tags, we'll only use IDs, but in other cases (such as custom fields or geolocation), a more sophisticated data structure will be needed. But you can always invent some more generic DTO class to handle such challenges.

Now, back to reality! Let's start with the `FormFilter`, the easiest of all:

```
namespace Domain\Subscriber\Filters;

use Illuminate\Database\Eloquent\Builder;

class FormFilter
{
    public function __construct(protected readonly array $ids)
    {
    }

    public function filter(Builder $subscribers): Builder
    {
        if (count($this->ids) === 0) {
            return $subscribers;
        }

        return $subscribers->whereIn('form_id', $this->ids);
    }
}
```

Two important things:

- There's no abstraction yet. We'll get there later. At this point, I have no idea what's the abstraction is, so I like to start with one of the concrete implementations.
- This class lives inside the `Subscriber` domain. I know you're reading the "Broadcasts" chapter, but these filter classes will filter subscribers.

The `filter` method is pretty simple, but there are two critical things. First of all, the `if` statement is very important. Without it, the following query will run:

```
select * from subscribers
where form_id in ()
```

Since it's an empty array, every subscriber will be returned. It's clearly not what we want. If the `$ids` array is empty, we don't need to do anything. There's no form filter; there's nothing to handle.

Now, let's see the `TagFilter`:

```
namespace Domain\Subscriber\Filters;

use Illuminate\Database\Eloquent\Builder;

class TagFilter
{
    public function __construct(protected readonly array $ids)
    {
    }

    public function filter(Builder $subscribers): Builder
    {
        if (count($this->ids) === 0) {
            return $subscribers;
        }

        return $subscribers->whereHas('tags', fn (Builder $tags) =>
            $tags->whereIn('id', $this->ids)
        );
    }
}
```

It queries subscribers where they have tags, but only those tags specified in the `$ids` array. Now we can see the abstraction:

- A method that takes a `Builder` and returns another one.
- A `__construct` with an array of integers.

It's time to write the `Filter` base class:

```
namespace Domain\Subscriber\Filters;

use Domain\Mail\DataTransferObjects\FilterData;
use Illuminate\Database\Eloquent\Builder;

abstract class Filter
{
    public function __construct(protected readonly array $ids)
    {
    }

    abstract public function filter(Builder $subscribers): Builder;
}
```

The constructors are removed from the filter classes, and we have a contract. When I use the strategy pattern and have multiple classes that implement the same contract, I usually want some "dynamic" behavior. Let me illustrate it with some made-up code:

```
public function filterSubscribers(Broadcast $broadcast)
{
    /*
     * $broadcast->filters is a FilterData that can be converted to an array:
     * [
     *   'tag_ids' => [1,2],
     *   'form_ids' => [1],
     * ];
     */

    $query = Subscriber::query();

    foreach ($broadcast->filters->toArray() as $type => $ids)
```

```

{
    // $type is 'tag_ids' and $ids is [1,2]
    $filter = $this->createFilter($type, $ids);
    $query = $filter->filter($query);
}
}

```

So by "dynamic," I mean I don't want to use `TagFilter` or `FormFilter` manually. I want something that loops through the broadcast's filters and gets things done. One of the key components of this "something" is the `$this->createFilter()` method. This is a factory method. It creates the appropriate `Filter` instance from the `tag_ids` or `form_ids` strings. Luckily it can be achieved with an `Enum`:

```

namespace Domain\Subscriber\Enums;

use Domain\Subscriber\Filters\Filter;
use Domain\Subscriber\Filters\FormFilter;
use Domain\Subscriber\Filters\TagFilter;

enum Filters: string
{
    case Tags = 'tag_ids';
    case Forms = 'form_ids';

    public function createFilter(array $ids): Filter
    {
        return match ($this) {
            self::Tags => new TagFilter($ids),
            self::Forms => new FormFilter($ids),
        };
    }
}

```

This single enum comes with several benefits:

- It's a single source of truth for every possible filter.
- It helps us be consistent with the string `tag_ids` and `form_ids`.
- It can be used as a factory that creates a `Filter` instance from an enum value.

The `createFilter` can be used as:

```
$filter = Filters::from('tag_ids')->createFilter($ids);
```

The `from` function creates an enum instance from a scalar value (only works with backed enums), and we can call methods on this instance.

Pipelines

The whole process of filtering can be imagined as a "pipeline" where we **send** the initial query **through** the `Filter` instances, and each filter modifies the query. Maybe you don't know about this, but Laravel has a concept exactly like that:

```
use Illuminate\Pipeline\Pipeline;

$subscribers = app(Pipeline::class)
    ->send(Subscriber::query())
    ->through($arrayOfFilters)
    ->thenReturn()
    ->get();
```

Let's see what's happening here step by step:

- An instance of `Pipeline` is resolved from the container.
- The initial query (`select * from subscribers`) is passed to the `send` method.
- We can specify an array of "pipes" in the `through` method. This array will contain the actual `Filter` instances.
- The `thenReturn` will return the result after each "pipe" is executed. It's the final query with the appropriate `where` clauses.
- Since it's a `Builder` instance, we need to call the `get` method to get the actual subscribers.

So Laravel will send the initial query through the `Filter` instances where both `TagFilter` and `FormFilter` will append a `where` clause (and a join) to the query builder.

To use the `Pipeline`, we have to make a small change in the `Filter` classes:

```

class TagFilter extends Filter
{
    public function handle(Builder $subscribers, Closure $next): Builder
    {
        if (count($this->ids) === 0) {
            return $next($subscribers);
        }

        $subscribers->whereHas('tags', fn (Builder $tags) =>
            $tags->whereIn('id', $this->ids)
        );

        return $next($subscribers);
    }
}

```

There are three differences compared to the previous version:

- The function is called `handle`. This is because `Pipeline` will look for a `handle` method in each pipe. It can be overwritten, but I decided to go with the default.
- There's a second argument. It's called `next` and is a type of `closure`. Each pipe will get the next pipe as an argument. So when the first `Filter` is being executed, let's say the `FormFilter`, the `$next` argument will be the second filter, so the `TagFilter` instance.
- We call this `$next` variable with the query builder instance on the last line.

Of course, we also need to change the `TagFilter`:

```

class FormFilter extends Filter
{
    public function handle(Builder $subscribers, Closure $next): Builder
    {
        if (count($this->ids) === 0) {
            return $next($subscribers);
        }

        $subscribers->whereIn('form_id', $this->ids);

        return $next($subscribers);
    }
}

```

And the abstract `Filter` as well:

```

abstract class Filter
{
    public function __construct(protected readonly array $ids)
    {
    }

    abstract public function handle(Builder $subscribers, Closure $next):
    Builder;
}

```

You absolutely don't need to use this approach, but it fits the current use case very well. After all of this, let's write a class that will actually use these classes and returns the subscribers for a broadcast:

```

namespace Domain\Subscriber\Actions;

class FilterSubscribersAction
{
    /**
     * @return Collection<Subscriber>
     */
    public static function execute(Broadcast $broadcast): Collection
    {
        return app(Pipeline::class)
            →send(Subscriber::query())
            →through(self::filters($broadcast))
            →thenReturn()
            →get();
    }

    /**
     * @return array<Filter>
     */
    public static function filters(Broadcast $broadcast): array
    {
        return collect($broadcast→filters→toArray())
            →map(fn (array $ids, string $key) =>
                Filters::from($key)→createFilter($ids)
            )
            →values()
            →all();
    }
}

```

Let's focus on the `filters` method first. `$broadcast→filters` returns a `FilterData` that can be converted into an array by calling the `toArray`. It has the following shape:

```
[
  'tag_ids' => [1,2],
  'form_ids' => [3,4],
]
```

So in the `map` `$key` is either `tag_ids` or `form_ids` and `$ids` contain the actual IDs. As we discussed earlier, this line would create a `TagFilter`:

```
Filters::from('tag_ids')->createFilter([1,2]);
```

Since the initial collection is associative, we need to call the `values` method to get a classic array with integer indexes. This array can be used with the `Pipeline`.

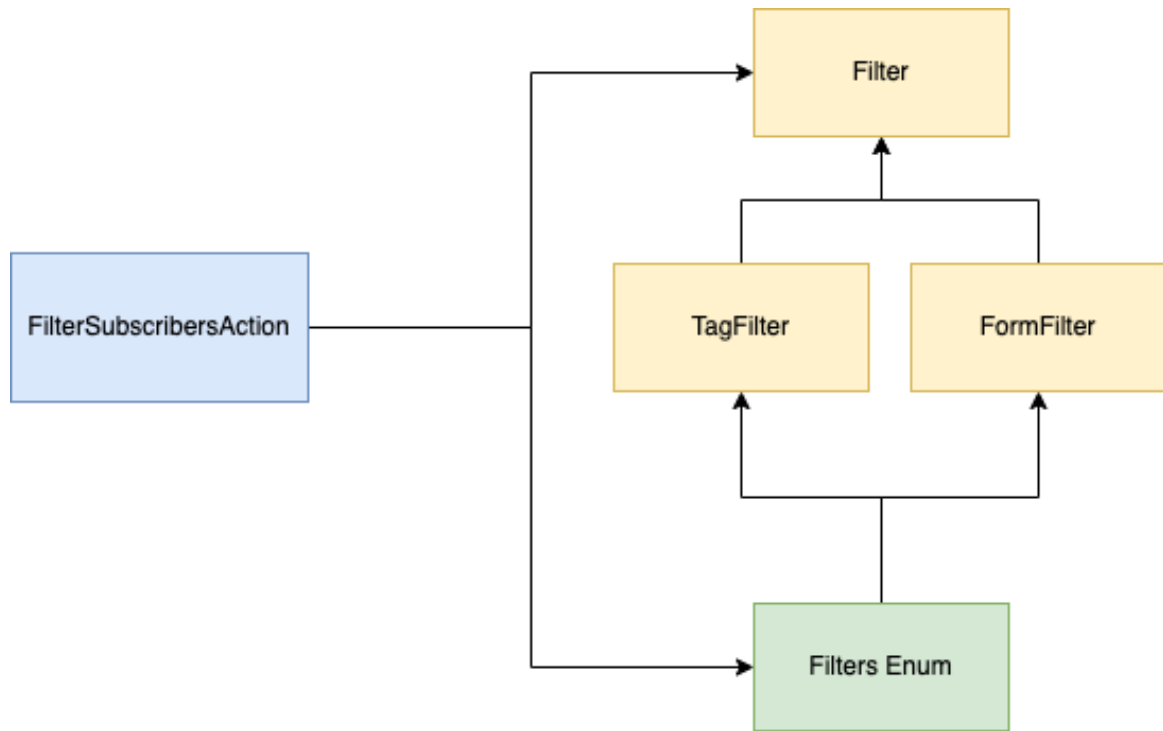
As you can see, this class is an action that returns a collection without side effects. So there are no inserts updates or any other operation. This logic can be moved into a query builder, for example, into the `BroadcastBuilder` (which doesn't exist yet). But here's the thing: I know that sequences will use the same logic to filter subscribers. And what if we want to use this piece of logic in other places as well? Such as:

- Filters on the subscriber list
- Segments. A segment is basically a saved subset of subscribers. So first, I apply some filters, and then I save it to the DB to use later.

So there are at least three classes that this logic can belong to:

- Broadcast
- Sequence Mail
- Subscriber

Later, when we implement sequences, we'll run into some issues with this action that raises some interesting questions and refactoring ideas. All right, it was a long chapter, so here's a context diagram to wrap things up:



Sending A Broadcast

The reason `FilterSubscribersAction` exists is to be able to send out broadcasts. Of course, an action class will send a broadcast, the `SendBroadcastAction` to be more specific:

```
class SendBroadcastAction
{
    public static function execute(Broadcast $broadcast): int
    {
        if (!$broadcast->status->canSend()) {
            throw CannotSendBroadcast::because(
                "Broadcast already sent at {$broadcast->sent_at}"
            );
        }

        $subscribers = FilterSubscribersAction::execute($broadcast)
            ->each(fn (Subscriber $subscriber) =>
                Mail::to($subscriber)->queue(new EchoMail($broadcast))
            );

        $broadcast->markAsSent();

        return $subscribers->each(fn (Subscriber $subscriber) =>
            $broadcast->sent_mails()->create([
                'subscriber_id' => $subscriber->id,
                'user_id' => $broadcast->user->id,
            ])
        )->count();
    }
}
```

There are four parts of this function; let's go through them one by one.

Rejecting a broadcast

There's an important rule: users cannot send a broadcast already sent. In this case, the method will throw an exception. If you remember, the `Broadcast` model has an enum cast:

```
class Broadcast extends BaseModel
{
    use WithData;
    use HasUser;

    protected $casts = [
        'filters' => FiltersCast::class,
        'status' => BroadcastStatus::class,
    ];
}
```

The status column has a `BroadcastStatus` enum cast. This enum has a method called `canSend`:

```
namespace Domain\Mail\Enums\Broadcast;

enum BroadcastStatus: string
{
    case Draft = 'draft';
    case Sent = 'sent';

    public function canSend(): bool
    {
        return match ($this) {
            self::Draft => true,
            self::Sent => false,
        };
    }
}
```

In the introduction, I talked about states and transitions where each state has a dedicated class. I also said that it could be overkill for simple problems. Now broadcast has only two statuses and one behavior that depends on the status. In these cases, you can move these status-dependent features into the enum. As far as it's simple like this one, it's perfectly okay, in my opinion.

So if the status of the broadcast is sent, we throw an exception. I often use a factory function called `because` that looks like:

```
namespace Domain\Mail\Exceptions\Broadcast;

class CannotSendBroadcast extends Exception
{
    public static function because(string $message): self
    {
        return new self($message);
    }
}

// Can be used like this:
throw CannotSendBroadcast::because(
    "Broadcast already sent at {$broadcast->sent_at}"
);
```

It's just a small helper function, but it feels natural to me. Another approach is to write a factory function for each reason:

```
namespace Domain\Mail\Exceptions\Broadcast;

class CannotSendBroadcast extends Exception
{
    public static function broadcastAlreadySent(Carbon $date): self
    {
        return new self("Broadcast already sent at {$date}");
    }
}

// Can be used like this:
throw CannotSendBroadcast::broadcastAlreadySent($broadcast->sent_at);
```

It's also an excellent approach.

Sending The E-mails

The next part of the action is filtering the subscribers and actually sending the e-mails:

```
$subscribers = FilterSubscribersAction::execute($broadcast)
    →each(fn (Subscriber $subscriber) ⇒
        Mail::to($subscriber)→queue(new EchoMail($broadcast))
    );
```

This method will probably send 1000s of e-mails, so obviously we need to queue them. Now, what is the `EchoMail`? Since we cannot have a dedicated mail for every broadcast and sequence, we need some 'generic' mail class and mail template. So we need a mail class and template that accepts a broadcast and just uses the `$broadcast->title` and the `$broadcast->subject`. This is what the `EchoMail` looks like:

```
namespace Domain\Mail\Mails;

class EchoMail extends Mailable implements ShouldQueue
{
    use Queueable, SerializesModels;

    public function __construct(public readonly Broadcast $broadcast)
    {
    }

    public function build()
    {
        return $this
            →subject($this→broadcast→subject)
            →view('emails.echo');
    }
}
```

And here's the `echo.blade.php` template:

```
{!! $broadcast→content !!}
```

It's as easy as that. The `!!` makes sure that the content is rendered as HTML. Now, this class and Blade template can be used for every broadcast. Of course, if we want to use them to handle sequences (in the future), we need to change the parameter type for something more generic.

If you noticed, I use this in the action:

```
Mail::to($subscriber)
```

To send e-mails to a `Subscriber`, we need to use the `Notifiable` trait.

```
class Subscriber extends BaseModel
{
    use Notifiable;
    use WithData;
    use HasUser;
}
```

Behind the scene, this trait will look for an `email` column in the model to send e-mails.

Changing the status

The next part is changing the broadcast's status from `draft` to `sent`:

```
$broadcast->markAsSent();
```

This method comes from the `BroadcastBuilder`, which is our first query builder:

```
namespace Domain\Mail\Builders\Broadcast;

use Domain\Mail\Enums\Broadcast\BroadcastStatus;
use Illuminate\Database\Eloquent\Builder;

class BroadcastBuilder extends Builder
{
    public function markAsSent(): void
    {
        $this->model->status = BroadcastStatus::Sent;
        $this->model->sent_at = now();
        $this->model->save();
    }
}
```

A builder class works very similarly to a standard model. You can write two kinds of methods:

- One that uses an actual model instance such as this one.
- One that works like a scope. So there's no model instance, only building a query.

In a case of a `Builder`, we need to use the `$this->model` to access the actual broadcast instance. Later on, we'll write scope-like methods.

After that, we need to override the `newEloquentBuilder` method in the `Broadcast` model:

```
class Broadcast extends BaseModel
{
    use WithData;
    use HasUser;

    public function newEloquentBuilder($query): BroadcastBuilder
    {
        return new BroadcastBuilder($query);
    }
}
```

Now, anytime we do something like this:

```
$broadcast->markAsReady();

// This is only a made-up function
Broadcast::whereSent();
```

We're interacting with the `BroadcastBuilder` class. I created `markAsReady` method for one main reason:

- The `status` and the `sent_at` date need to change together.

Of course, this function does not guarantee that this rule will be obeyed, but it's a good start to have a dedicated method that takes care of this rule.

Creating sent mails

The last part of the action is to create `sent_mail` records.

```
return $subscribers->each(fn (Subscriber $subscriber) =>
    $broadcast->sent_mails()->create([
        'subscriber_id' => $subscriber->id,
        'user_id' => $broadcast->user->id,
    ])
)->count();
```

If you remember from the introduction, every broadcast or sequence mail will become a row in the `sent_mails` table so we can query reports such as:

- Total e-mails sent
- Overall open rate %
- Overall click rate %
- Or, if you want to charge users based on their volume, this table will be the fundamental pillar of the pricing

The `sent_mails` table has the following properties:

sendable_id	sendable_type	subscriber_id	user_id	opened_at	clicked_at
1	Domain\Mail\Models\Broadcast\Broadcast	1	1	NULL	NULL
1	Domain\Mail\Models\Broadcast\Broadcast	2	1	2022-03-09 20:50:11	NULL
1	Domain\Mail\Models\Broadcast\Broadcast	3	1	2022-03-09 20:54:23	2022-03-09 20:56:03

It's a polymorph table where the related model can be either a broadcast or a sequence mail (later on that). I decided to go with the `sendable` name to avoid annoying conflicts with Laravel's `Mailable`. So when we're sending a broadcast to 1500 subscribers, we need to create 1500 rows in this table. This is exactly what the last part of the action does.

Since it's a polymorph relation, we need to use the `morphMany` method in the `Broadcast` model:

```
public function sent_mails(): MorphMany
{
    return $this->morphMany(SentMail::class, 'sendable');
}
```

Queueing the whole action

Now we can send broadcasts to subscribers! In the action we're only using a queue to send the e-mails. But the rest of the action is also performance heavy:

- Filtering potentially 1000s of subscribers.
- Inserting 1000s of records into the `sent_mails` table.

Because of that, the action needs to run in a queue. For that we need a job:

```
namespace Domain\Mail\Jobs\Broadcast;

use Domain\Mail\Actions\Broadcast\SendBroadcastAction;

class SendBroadcastJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(private readonly Broadcast $broadcast)
    {
    }

    public function handle()
    {
        SendBroadcastAction::execute($this->broadcast);
    }
}
```

It's very similar to the `ImportSubscribersJob`; it only calls the action. Now we only need a controller to dispatch this job:

```
namespace App\Http\Web\Controllers\Mail\Broadcast;

use Domain\Mail\Jobs\Broadcast\SendBroadcastJob;

class SendBroadcastController
{
    public function __invoke(Broadcast $broadcast): Response
    {
        SendBroadcastJob::dispatch($broadcast);

        return response('', Response::HTTP_ACCEPTED);
    }
}
```

The route for this action:

```
Route::patch('broadcasts/{broadcast}/send', SendBroadcastController::class);
```

Calculating The Performance Of A Broadcast

If you remember from the design chapter, we need to report the performance metrics of a broadcast. Namely, these three metrics:

- Total e-mails sent
- Open rate %
- Click rate %

Fortunately, with the `sent_mails` table, these metrics are easy to calculate:

- Total e-mails sent is the number of records in the `sent_mails` table with a given broadcast's ID.
- Open rate is the number of records where the `opened_at` is not null, divided by the total number of records.
- Finally, the click rate is the same but with the `clicked_at` column.

Using this example from the previous chapter:

sendable_id	sendable_type	subscriber_id	user_id	opened_at	clicked_at
1	Domain\Mail\Models\Broadcast\Broadcast	1	1	NULL	NULL
1	Domain\Mail\Models\Broadcast\Broadcast	2	1	2022-03-09 20:50:11	NULL
1	Domain\Mail\Models\Broadcast\Broadcast	3	1	2022-03-09 20:54:23	2022-03-09 20:56:03

We get the following stats:

- Total e-mails sent: 3
- Open rate: $2/3 = 66.67\%$
- Click rate: $1/3 = 33.33\%$

So it's not complicated at all. Let's start with some queries in the `SentMailBuilder`:

```
namespace Domain\Mail\Builders\SentMail;

use Illuminate\Database\Eloquent\Builder;

class SentMailBuilder extends Builder
{
    public function countOf(Broadcast $broadcast): int
    {
        return $this
            ->where('sendable_id', $broadcast->id)
            ->where('sendable_type', Broadcast::class)
            ->count();
    }
}
```

Now, if you think about it for a minute, we discussed in the design chapter that these stats would be needed in the case of:

- Broadcasts
- Sequences
- Individual sequence mails

For a minute, let's forget about sequences and focus on broadcasts and sequence mails. We'll also need to query the total number of `sent_mails` in case of sequence mails. That query would look like this:

```
public function countOf(SequenceMail $mail): int
{
    return $this
        ->where('sendable_id', $mail->id)
        ->where('sendable_type', SequenceMail::class)
        ->count();
}
```

This is essentially the same as a broadcast; the only difference is the class name itself. Why don't we create a `Sendable` interface that abstracts away this tiny difference?

```
namespace Domain\Mail\Contracts;

interface Sendable
{
    public function id(): int;
    public function type(): string;
}
```

The implementation is very simple:

```

namespace Domain\Mail\Models\Broadcast;

use Domain\Mail\Contracts\Sendable;

class Broadcast extends BaseModel implements Sendable
{
    use WithData;
    use HasUser;

    public function id(): int
    {
        return $this->id;
    }

    public function type(): string
    {
        return $this::class;
    }
}

```

Now we can accept a `Sendable` type in the query:

```

namespace Domain\Mail\Builders\SentMail;

use Illuminate\Database\Eloquent\Builder;

class SentMailBuilder extends Builder
{
    public function countOf(Sendable $sendable): int
    {
        return $this
            ->where('sendable_id', $sendable->id())
            ->where('sendable_type', $sendable->type())
            ->count();
    }
}

```

So we have the total number of e-mails sent. Let's move on to open rate:

```
public function openRate(Sendable $sendable, int $total): float
{
    $openedCount = $this
        ->where('sendable_id', $sendable->id())
        ->where('sendable_type', $sendable->type())
        ->whereNotNull('opened_at')
        ->count();

    return $openedCount / $count;
}
```

This method accepts a `Sendable` and an `integer`. The `integer` is the total number of e-mails sent, and so is the result of the previous query. As you can see, there is some code duplication between the two methods. And possible, the `clickRate` will be very similar to the `openRate`. We can extract this query to a scope:

```
public function whereSendable(Sendable $sendable): self
{
    return $this
        ->where('sendable_id', $sendable->id())
        ->where('sendable_type', $sendable->type());
}

public function whereOpened(): self
{
    return $this->whereNotNull('opened_at');
}

public function whereClicked(): self
{
    return $this->whereNotNull('clicked_at');
}
```

This is the equivalent of an Eloquent scope, but we don't need to use the `scope` prefix here. Now we can refactor the queries like the following:

```

public function countOf(Sendable $sendable): int
{
    return $this->whereSendable($sendable)->count();
}

public function openRate(Sendable $sendable, int $total): float
{
    return $this
        ->whereSendable($sendable)
        ->whereOpened()
        ->count() / $total;
}

public function clickRate(Sendable $sendable, int $total): float
{
    return $this
        ->whereSendable($sendable)
        ->whereClicked()
        ->count() / $total;
}

```

I think it's very clear and readable. These methods can be used as:

```

public function index(Broadcast $broadcast)
{
    $total = SentMail::countOf($broadcast);
    $openRate = SentMail::openRate($broadcast);
    $clickRate = SentMail::clickRate($broadcast);

    /**
     * $total = 3
     * $openRate = 0.6667
     * $clickRate = 0.3333
     */
}

```

As you can see, the open rate and click rate will be floats. I think we can do better than that. Expressing a percentage is the perfect opportunity to use a value object. So instead of a float, we want something like that:

```
public function openRate(Sendable $sendable, int $total): Percent
{
    $openedCount = $this
        ->whereSendable($sendable)
        ->whereOpened()
        ->count();

    return Percent::from($openedCount, $total);
}
```

A percent is a very generic value, so I have these kinds of value objects inside the Shared domain:

```
namespace Domain\Shared\ValueObjects;

class Percent
{
    public readonly float $value;
    public readonly string $formatted;

    public function __construct(float $value)
    {
        $this->value = $value;
        $this->formatted = number_format($this->value * 100, 1) . '%';
    }

    public static function from(float $numerator, float $denominator)
    {
        if ($denominator === 0.0) {
            return new self(0);
        }

        return new self($numerator / $denominator);
    }
}
```

```
}
```

We accept both the numerator and the denominator in the `from` method. It has two benefits:

- It's a little bit easier to use this class.
- More importantly, it encapsulates how the application deals with a 0 in the denominator.

Without this factory function, we need to check for a 0 value every place we want to create a new percentage. This object contains both the raw and the formatted value. We don't really need this right now, but this is the way I used to write these classes.

It's time to use the `Percent` class:

```
public function openRate(Sendable $sendable, int $total): Percent
{
    $openedCount = $this
        ->whereSendable($sendable)
        ->whereOpened()
        ->count();

    return Percent::from($openedCount, $total);
}

public function clickRate(Sendable $sendable, int $total): Percent
{
    $clickedCount = $this
        ->whereSendable($sendable)
        ->whereClicked()
        ->count();

    return Percent::from($clickedCount, $total);
}
```

Now we can write a class that calls these queries and put together a DTO that can be used in controllers. Just as before with `FilterSubscribersAction`, it's going to be an action class:

```

namespace Domain\Mail\Actions;

use Domain\Mail\DataTransferObjects\PerformanceData;

class GetPerformanceAction
{
    public static function execute(Sendable $sendable): PerformanceData
    {
        return new PerformanceData(
            total: SentMail::countOf($sendable),
            open_rate: SentMail::openRate($sendable, $total),
            click_rate: SentMail::clickRate($sendable, $total),
        );
    }
}

```

I created this class because performance is needed for three different models, and the app will probably have many views and reports where these metrics need to be shown. By writing an action, we can handle **every possible use case**. The `PerformanceData` class only contains three properties:

```

namespace Domain\Mail\DataTransferObjects;

use Domain\Shared\ValueObjects\Percent;
use Spatie\LaravelData\Data;

class PerformanceData extends Data
{
    public function __construct(
        public readonly int $total,
        public readonly Percent $open_rate,
        public readonly Percent $click_rate,
    ) {}
}

```

I'll use this action and DTO in a minute, but first, let's take a breath and add another cool feature.

Later, when we calculate the performance of sequences, we'll run into some issues with the `GetPerformanceAction` that raise some interesting questions and refactoring ideas.

Previewing A Broadcast

Wouldn't it be great if we could preview a broadcast before sending it? Luckily it's easy with Laravel. First, let's create a route:

```
Route::get('broadcasts/{broadcast}/preview',
    PreviewBroadcastController::class);
```

Then a controller:

```
namespace App\Http\Web\Controllers\Mail\Broadcast;

use Domain\Mail\Mails\EchoMail;
use Domain\Mail\Models\Broadcast\Broadcast;
use Domain\Mail\ViewModels\Broadcast\PreviewBroadcastViewModel;

class PreviewBroadcastController
{
    public function __invoke(Broadcast $broadcast): Response
    {
        return Inertia::render('Broadcast/Preview', [
            'model' => new PreviewBroadcastViewModel(new EchoMail($broadcast)),
        ]);
    }
}
```

So the controller creates a new `EchoMail` instance with the `$broadcast` from the route. Here's the view model:

```

namespace Domain\Mail\ViewModels\Broadcast;

use Domain\Mail\Mails\EchoMail;
use Domain\Shared\ViewModels\ViewModel;

class PreviewBroadcastViewModel extends ViewModel
{
    public function __construct(private readonly EchoMail $mail)
    {
    }

    public function subject(): string
    {
        return $this->mail->mail->subject();
    }

    public function content(): string
    {
        return $this->mail->mail->content();
    }
}

```

As you can see, there's a `mail` property available in every `Mailable` class. In this `mail` property, you can access:

- The subject of the e-mail
- The raw HTML content

That's exactly what we need. The last thing we need is the `Broadcast/Preview` Vue component:

```
<template>
  <Head title="Broadcast Preview" />

  <BreezeAuthenticatedLayout>
    <template #header>
      <h2 class="font-semibold text-xl text-gray-800 leading-tight">
        {{ model.subject }}
      </h2>
    </template>

    <div v-html="model.content" class="py-12 max-w-7xl mx-auto"></div>
  </BreezeAuthenticatedLayout>
</template>
```

Vue will render the raw HTML content given to the `v-html`. The result is a page with the content of the e-mail.

Get Broadcasts

The last broadcast-related feature is the list. If a broadcast is sent, we need to show the performance too.

The controller is easy as usual:

```
namespace App\Http\Web\Controllers\Mail\Broadcast;

use Domain\Mail\ViewModels\Broadcast\GetBroadcastsViewModel;

class BroadcastController
{
    public function index(): Response
    {
        return Inertia::render('Broadcast/List', [
            'model' => new GetBroadcastsViewModel(),
        ]);
    }
}
```

And here's the view model:

```

namespace Domain\Mail\ViewModels\Broadcast;

use Domain\Mail\Actions\GetPerformanceAction;
use Domain\Mail\DataTransferObjects\Broadcast\BroadcastData;
use Domain\Mail\Models\Broadcast\Broadcast;
use Domain\Shared\ViewModels\ViewModel;

class GetBroadcastsViewModel extends ViewModel
{
    /**
     * @return Collection<BroadcastData>
     */
    public function broadcasts(): Collection
    {
        return Broadcast::latest()→get()→map→getData();
    }

    /**
     * @return Collection<int, PerformanceData>
     */
    public function performances(): Collection
    {
        return Broadcast::all()
            →mapWithKeys(fn (Broadcast $broadcast) ⇒ [
                $broadcast→id ⇒ GetPerformanceAction::execute($broadcast),
            ]);
    }
}

```

This is the class that uses the `GetPerformanceAction`.

You have a clear benefit if you're using a Trait instead of the `GetPerformanceAction` here. You don't need to loop through every broadcast and store the performances as an array. The trait itself can define an attribute accessor that loads the performance into a `performance` attribute. And you can just use this attribute on the frontend. It won't cause performance issues because attribute accessors are evaluated lazily. This means that they won't be called until you actually access them. This also means that a simple `Broadcast::all()` won't load them, and the FE won't be able to show the performances. You can solve this issue quickly by using the `append` method:

```
Broadcast::all()>each->append('performance');
```

Now back to the actual feature. Since it lists every broadcast, we need the performance for every broadcast, so the resulting collection looks like this:

```
[
  12 => [
    'total' => 43,
    'open_rate' => [
      'value' => 0.494,
      'formatted' => '49.4%',
    ],
    'click_rate' => [
      'value' => 0.121,
      'formatted' => '12.1%',
    ],
  ],
],
]
```

Where index 12 is a broadcast ID. We need to display these values in a table like this:

Broadcasts

[New Broadcast](#)

SUBJECT	STATUS	PERFORMANCE	
Best Laravel Eloquent Features	sent	200 Recipients • 49.5% Open rate • 9.0% Click rate	Preview Remove
Domain-Driven Design with Laravel Is Out!	draft	-	Preview Send Remove

And we also need to display it on the broadcast page:

Best Laravel Eloquent Features Sent

200 Recipients • 49.5% Open rate • 9.0% Click rate

SUBJECT

Best Laravel Eloquent Features

CONTENT

Since we will also need something very similar for sequences and sequence mails, it's a good idea to create a Vue component called `PerformanceLine`:

```
<script>
export default {
  name: 'PerformanceLine',
  props: {
    performance: {
      type: Object,
      required: true,
    },
  },
}
</script>

<template>
  <div class="text-sm text-gray-900">
    {{ performance.total }} Recipients •
    {{ performance.open_rate.formatted }} Open rate •
    {{ performance.click_rate.formatted }} Click rate
  </div>
</template>
```

It displays the performance as a text, as a line. It can be used like this:

```

<tr v-for="broadcast in model.broadcasts" :key="broadcast.id" class="hover:bg-
gray-100">
  <td class="px-6 py-4 hover:cursor-pointer" @click="edit(broadcast)">
    <div class="text-sm text-gray-900">{{ broadcast.subject }}</div>
  </td>
  <td class="px-6 py-4">
    <div class="text-sm text-gray-900">{{ broadcast.status }}</div>
  </td>
  <td class="px-6 py-4">
    <PerformanceLine
      v-if="broadcast.status !== 'draft'"
      :performance="model.performances[broadcast.id]">
    </PerformanceLine>
    <div v-else></div>
  </td>
</tr>

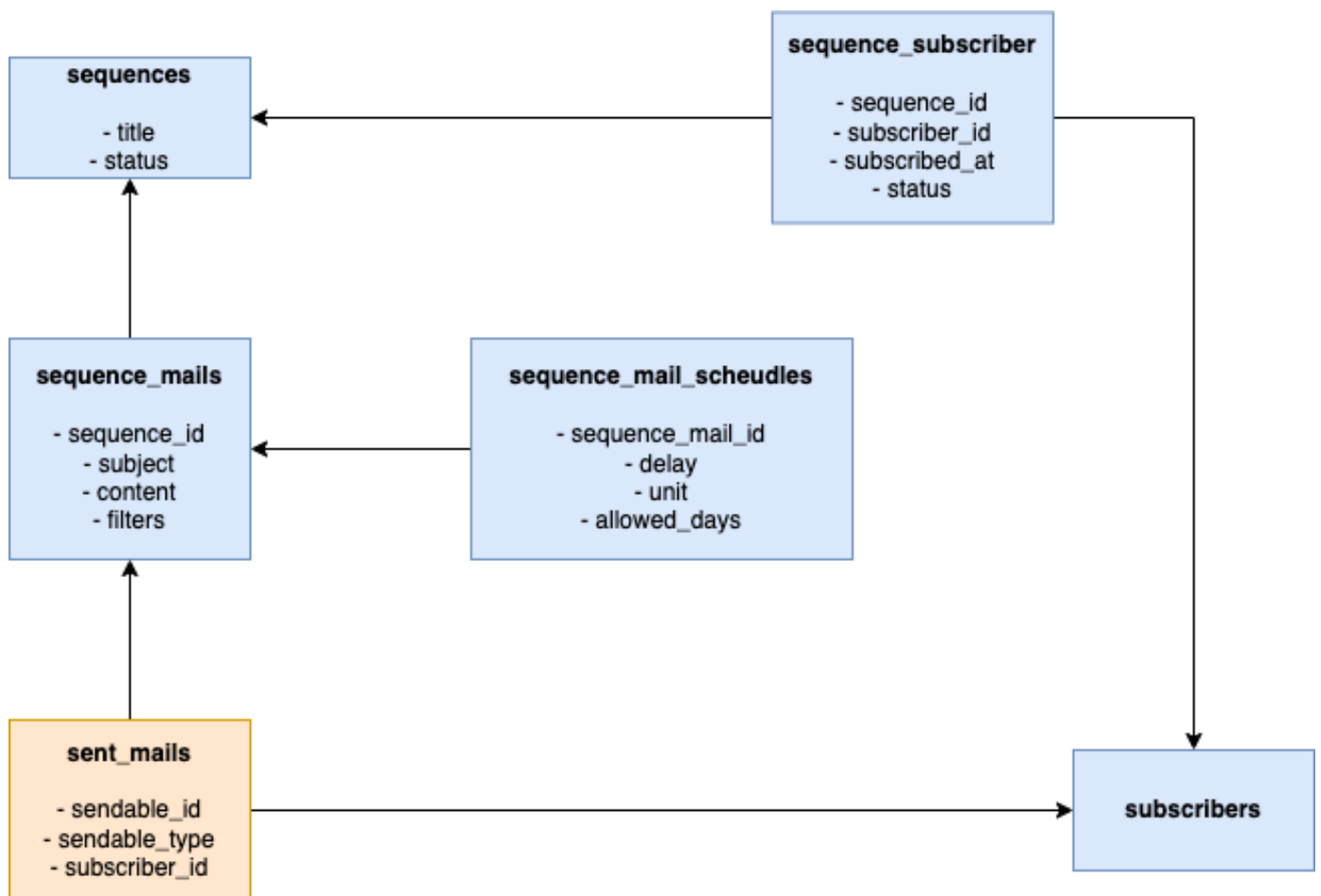
```

Sequences

Congratulations! At this point, you have learned the basics:

- Creating models
- Updating models
- Combine these two actions using upsert
- Listing models
- Using DTOs in these scenarios
- How view models work
- A lot of actions

So in this chapter, I try to avoid the basics and focus on the more exciting stuff. First, let's recap the database structure:



Everything is discussed in the design chapter except one part, the `sequence_subscriber`. When we are dealing with sequence reports, we need numbers like these:

My Awesome Sequence

49 Subscribers • 0.0% Open rate • 0.0% Click rate

Total Subscribers
49In Progress
1Completed
48

SUBJECT	SENDS	AVERAGE OPEN RATE	AVERAGE CLICK RATE
First E-mail	1	0.0%	0.0%
Second E-mail	48	0.0%	0.0%

What do these numbers mean?

- Total subscribers: each subscriber who received at least one e-mail from this sequence is considered to be subscribed to the sequence.
- In Progress: each subscriber is currently receiving e-mails from this sequence.
- Completed: each subscriber who finished the sequence. So they will not receive any more e-mails from this sequence.

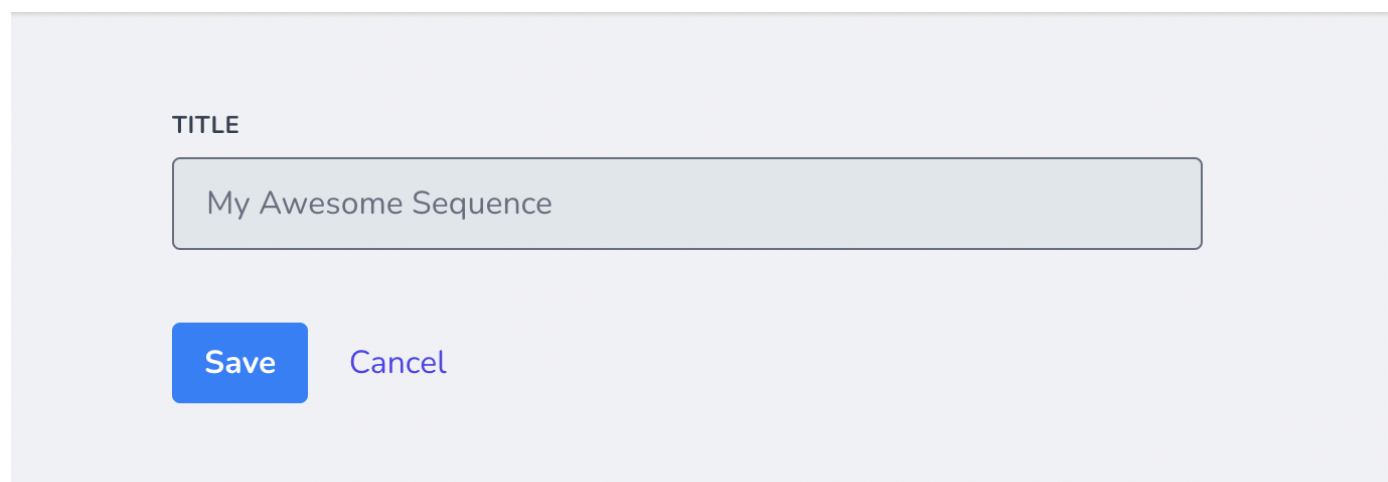
We need the `sequence_subscriber` pivot table with a `status` column to calculate these numbers. The app will attach every subscriber to the sequence when creating a new one. So if there are 100 subscribers in the database when creating a new sequence, 100 rows will be inserted in the `sequence_subscriber` table with a default `null` status. `null` means that the subscriber hasn't received any e-mail from the sequence yet.

A scheduled command will send the e-mails in the sequence. When this command is running, it will update this status. This is how can keep track of the subscriber statuses.

What happens with new subscribers? You create a sequence with 100 subscribers, and after a day, you have ten new subscribers. Now they're not in the `sequence_subscriber` table, so will they miss the sequence? Of course not. We'll take care of them later.

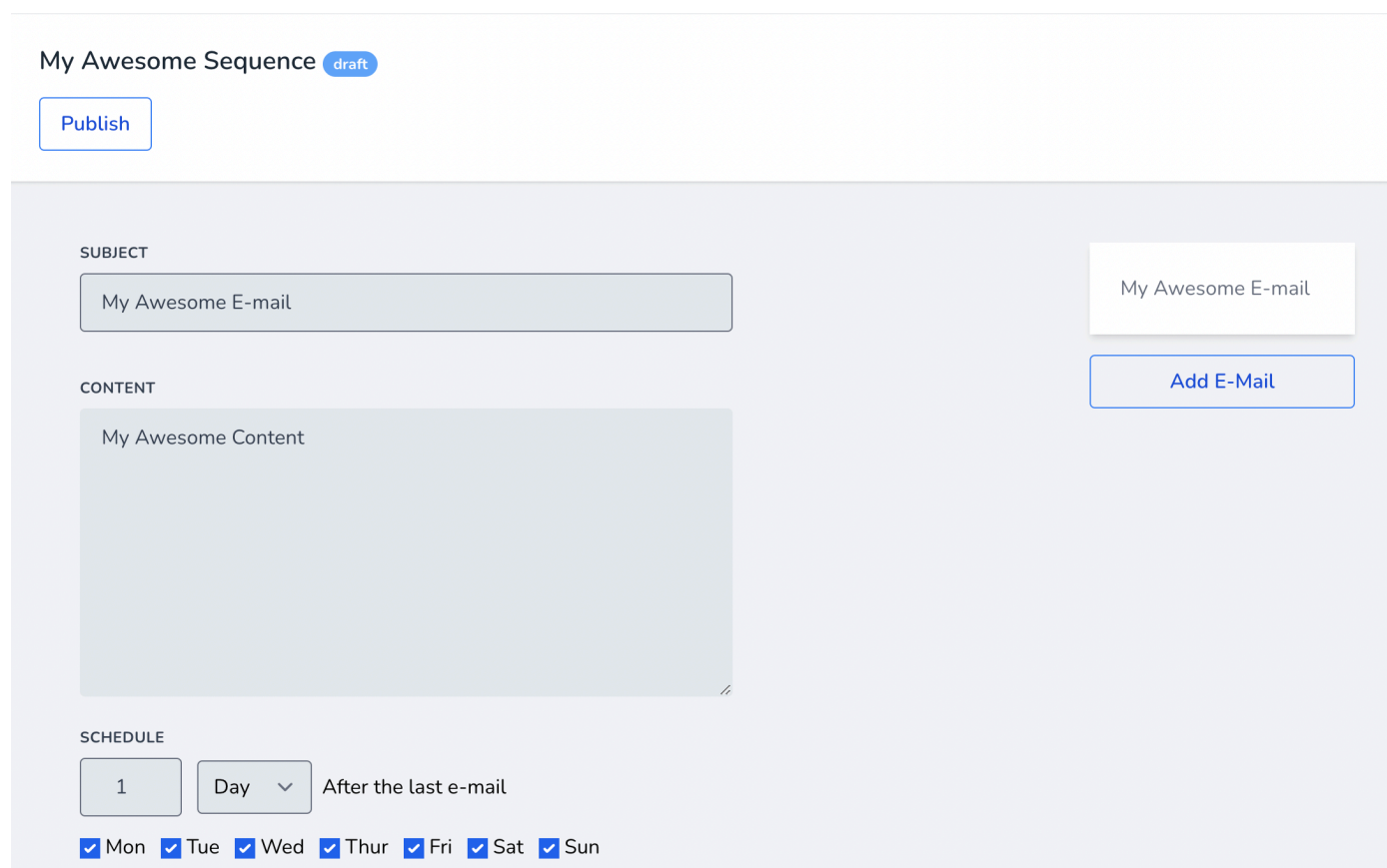
Creating A Sequence

Creating a sequence is a bit different from creating a broadcast, for example. I recommend you try out the demo application. If you create a new sequence, you'll only need to provide a name:



A screenshot of a web form for creating a new sequence. The form has a title label 'TITLE' above a text input field containing 'My Awesome Sequence'. Below the input field are two buttons: a blue 'Save' button and a grey 'Cancel' button.

After you provide a name, you are redirected to a page where you can add/edit/remove mails in the sequence:



A screenshot of a web page for editing a sequence named 'My Awesome Sequence', which is currently in 'draft' status. The page features a 'Publish' button. Below this, there are three main sections: 'SUBJECT' with a text input field containing 'My Awesome E-mail'; 'CONTENT' with a large text area containing 'My Awesome Content'; and 'SCHEDULE' with a numeric input field set to '1', a dropdown menu set to 'Day', and the text 'After the last e-mail'. At the bottom of the schedule section, there are seven checkboxes for the days of the week, all of which are checked: Mon, Tue, Wed, Thur, Fri, Sat, and Sun. On the right side of the page, there is a preview box showing 'My Awesome E-mail' and an 'Add E-Mail' button.

One interesting thing: when you create a new sequence, the app will automatically create the first sequence mail with default data that you can see on the screenshot.

Here's the way it's implemented:

```

namespace Domain\Mail\Actions\Sequence;

use Domain\Mail\DataTransferObjects\Sequence\SequenceData;
use Domain\Mail\DataTransferObjects\Sequence\SequenceMailData;
use Domain\Mail\Models\Sequence\Sequence;
use Domain\Shared\Models\User;
use Domain\Subscriber\Models\Subscriber;

class CreateSequenceAction
{
    public static function execute(SequenceData $data, User $user): Sequence
    {
        return DB::transaction(function () use ($data, $user) {
            $sequence = Sequence::create([
                ... $data->all(),
                'user_id' => $user->id,
            ]);

            UpsertSequenceMailAction::execute(
                SequenceMailData::dummy(),
                $sequence,
                $user
            );

            $sequence->subscribers()->sync(
                Subscriber::select('id')->pluck('id')
            );

            return $sequence;
        });
    }
}

```

As discussed earlier, the last step of this action inserts every subscriber into the `sequence_subscriber` table without a status.

The `UpsertSequenceMailAction` contains a standard `updateOrCreate` Eloquent call. This action can be used to add another e-mail to the sequence.

You can see I'm using the `SequenceMailData::dummy` method, which is a factory function that returns a new `SequenceMailData` with some default data:

```
public static function dummy(): self
{
    return self::from([
        'subject' => 'My Awesome E-mail',
        'content' => 'My Awesome Content',
        'status' => SequenceMailStatus::Draft,
        'filters' => FilterData::empty(),
        'schedule' => [
            'delay' => 1,
            'unit' => 'day',
            'allowed_days' => SequenceMailScheduleAllowedDaysData::empty(),
        ]
    ]);
}
```

This is how a 'dummy' mail is created. I extracted this function because later, we'll reuse it.

There's a method called `empty` in every `Data` class provided by laravel-data. It returns an array with the shape of your DTO, but every attribute will have a `null` value. It's very useful in some situations. So the `SequenceMailScheduleAllowedDaysData::empty` will return this array:

```
[
    'monday' => null,
    'tuesday' => null,
    'wednesday' => null,
    'thursday' => null,
    'friday' => null,
    'saturday' => null,
    'sunday' => null,
]
```

But in this case, we want every value to be `true`:

```
[
    'monday' => true,
    'tuesday' => true,
    'wednesday' => true,
    'thursday' => true,
    'friday' => true,
    'saturday' => true,
    'sunday' => true,
]
```

To achieve this, the `empty` method can be simply overwritten:

```
namespace Domain\Mail\DataTransferObjects\Sequence;

use Spatie\LaravelData\Data;

class SequenceMailScheduleAllowedDaysData extends Data
{
    public function __construct(
        public readonly bool $monday,
        public readonly bool $tuesday,
        public readonly bool $wednesday,
        public readonly bool $thursday,
        public readonly bool $friday,
        public readonly bool $saturday,
        public readonly bool $sunday,
    ) {}

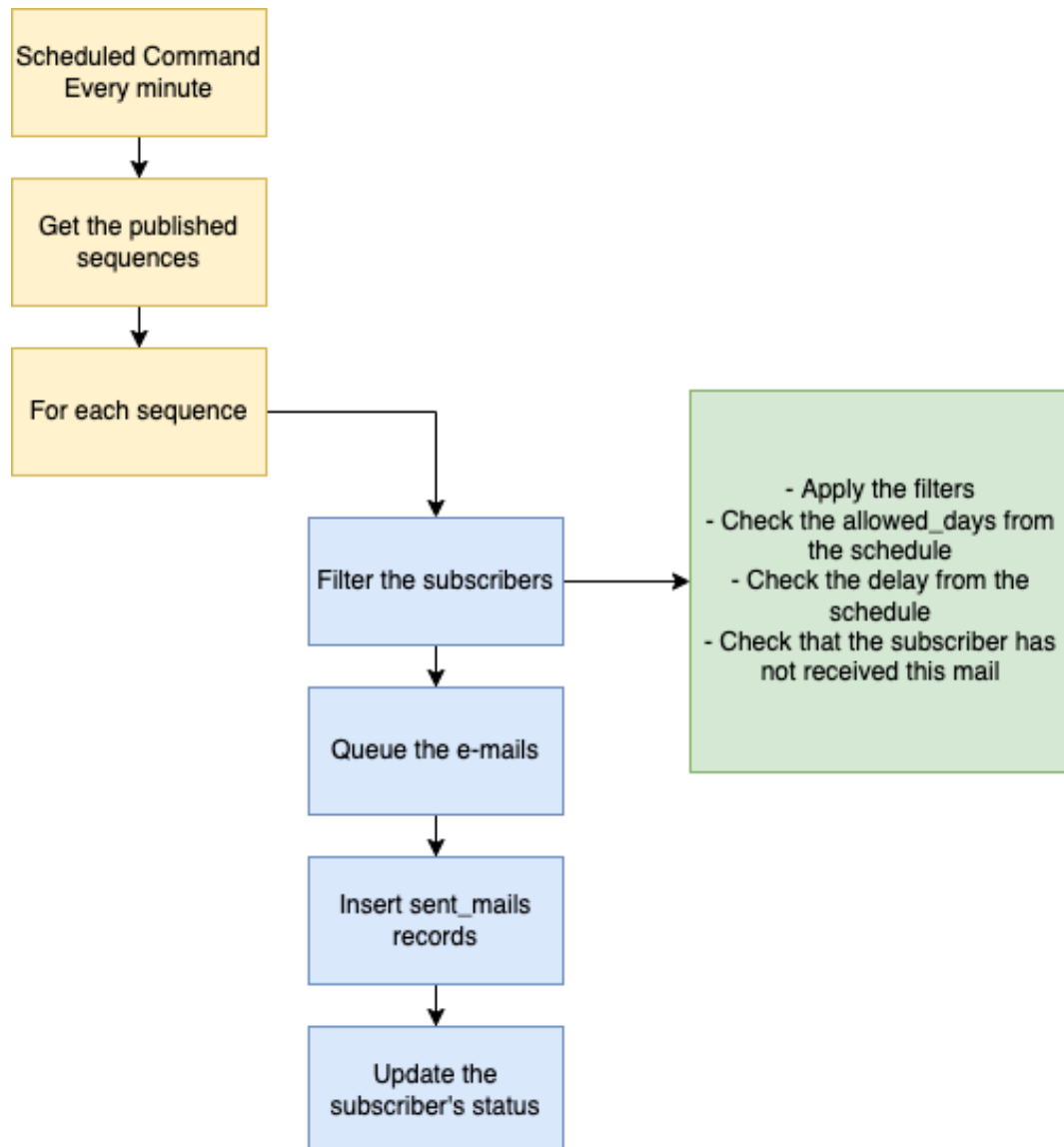
    public static function empty($extra = []): array
    {
        return (new self(true, true, true, true, true, true, true))->toArray();
    }
}
```

In this case, where the DTO has only bool properties, this is a more reasonable default instead of null values. As promised, I don't think we need to spend much more time on CRUD actions or anything like that (by the way, you can always check the source code for more details). The interesting stuff is how the app will start and proceed in a sequence.

Proceeding A Sequence

First, let's go through what it means to start and proceed with a sequence step by step:

- A scheduled job will run every minute (or so) and initiate the whole process for every published sequence. From here on, everything is meant for one sequence.
- Query the published mails.
- Determine which subscribers will receive this mail:
 - Apply the filters set up for the mail. The `FilterSubscribersAction` will be used for this.
 - Check that the mail should be sent based on the schedule. For example, if the `allowed_days` contains only weekdays and the current day is Saturday, we don't need to send this mail.
 - Check if it's not too early for the mail to be sent. The schedule contains a `delay` attribute that says something like: "this mail can only be sent after two days since the last mail".
 - Finally, we need to check that the given subscriber has not already received this mail.
- After all these, we can queue the actual e-mails.
- And create the `sent_mails` records.
- As a final step, this action will also update the `sequence_subscriber` status column to either in progress or completed.



I'll cover the scheduled command later. Now, let's focus on the code that deals with a single sequence and its mails. This logic will live inside an action, and a good name can be `ProceedSequenceAction`. But first, let's implement the logic inside the "filter the subscribers" box.

Should send today

Let's write a simple method that determines if the `SequenceMail` should be sent at the current date based on its schedule:

```
class SequenceMail extends BaseModel implements Sendable
{
    public function shouldSendToday(): bool
    {
        $dayName = Str::lower(now() -> dayName);
        return $this -> schedule -> allowed_days -> {$dayName};
    }
}
```

I don't put this method inside a query builder simply because it's not a query. It's in the `SequenceMail` model. I'm not a big fan of "fat models," but they can still have some small methods.

The schedule is a `SequenceMailSchedule` instance, and in the `allowed_days` column, there's a JSON like `{"monday": true}` as we have seen earlier. So the line that calls the `$dayName` on the `allowed_days` does something like this:

```
return $this -> schedule -> allowed_days -> monday;
```

By default, if you're using the `array` cast provided by Laravel, the `allowed_days` would be an array. And this line would throw a huge exception in your face. To be able to use it as an object, I have a custom cast that casts the attribute into a DTO:

```

namespace Domain\Mail\Models\Casts\Sequence;

use
Domain\Mail\DataTransferObjects\Sequence\SequenceMailScheduleAllowedDaysData;

class SequenceMailScheduleAllowedDaysCast implements CastsAttributes
{
    public function get($model, string $key, $value, array $attributes)
    {
        return SequenceMailScheduleAllowedDaysData::from(
            json_decode($value, true)
        );
    }

    public function set($model, string $key, $value, array $attributes)
    {
        return [
            'allowed_days' => json_encode($value),
        ];
    }
}

```

It works the same way as discussed in the broadcast chapter (with filters). This way, the `allowed_days` attribute will return a `SequenceMailScheduleAllowedDaysData` DTO, and we can use it as an object.

Already received

Next, we need a method that determines if the subscriber has already received a given mail. The

`sent_mails` table has a `subscriber_id`, which is how we can check it. So first, I'll add a relationship to the `Subscriber`:

```
namespace Domain\Subscriber\Models;

class Subscriber extends BaseModel
{
    public function received_mails(): HasMany
    {
        return $this->hasMany(SentMail::class);
    }
}
```

How you name this method is an interesting questions. I see three options:

- `sent_mails`. This is the most obvious one since it's a relationship to the `SentMail` model. And it's technically correct. However, writing `$subscriber->sent_mails` feels very weird to me. It's like saying the "the subscriber sent these mails". But subscribers do not send mails, they receive them. And even from technical point of view it can be weird in certain situations, for example:

```
Subscriber::query()
    ->whereNotNull('form_id')
    ->get()
    ->map->sent_mails
    ->map->getData()
```

When you have snippets like this, it feels weird to see the word `sent_mails` in the context of subscribers. I need to think about it. Even if it's just one second, it's an extra cognitive load.

- `received_mails`. From a technical point of view, it's incorrect and a bit weird. The method is called `received_mails`, but it references the `sent_mails` table. However, from a domain language perspective, this is the correct name, in my opinion. Imagine a feature request from a PM: "Can we list all the **mails** the subscriber has **received** when I click on his **profile**?" You say yes (not if you have a choice) and can do the following:
 - Go to the `GetSubscriberProfileViewModel` class
 - Create a new method called `receivedMails`
 - And you return this: `$this->subscriber->received_mails`

- There's a third option: `mails`. It's kinda in between the previous two. I don't like it. It's not specific enough. What is mail in this context? A `Broadcast`? A `SequenceMail`? A `SentMail`? Each can be referenced as a "mail." There's also an `EchoMail`, Laravel's `Mailable` interface, and a whole namespace called `Mails`. And we're working on an **e-mail** marketing software. It can cause too much confusion. Not now, but after five years and ten different developers. For these reasons, I think this is the poorest choice.

So I've decided to go with `received_mails`. Using this relation, we can write the following method:

```
namespace Domain\Subscriber\Builders;

class SubscriberBuilder extends Builder
{
    public function alreadyReceived(SequenceMail $mail): bool
    {
        return $this->model->received_mails()->whereSendable($mail)->exists();
    }
}
```

By using the `received_mails` relationship, we can call the `whereSendable` scope, and we're basically done.

Too early for

The last check we need to perform before sending a sequence mail to a subscriber is to determine if it's too early based on the schedule. Just a quick reminder, here are the important models to determine this:

- `Sequence` has many
- `SequenceMail` has one
- `SequenceMailSchedule`

And the `SequenceMailSchedule` has two essential attributes:

- `delay` the number of *time unit* has to pass before sending the given `SequenceMail`.
- `unit` is either `day` or `hour`.

The `unit` attribute has an enum cast which refers to this enum:

```
namespace Domain\Mail\Enums\Sequence;

enum SequenceMailUnit: string
{
    case Day = 'day';
    case Hour = 'hour';
}
```

Since we can write methods in enums, this is the perfect place to add a small helper function:

```
namespace Domain\Mail\Enums\Sequence;

enum SequenceMailUnit: string
{
    case Day = 'day';
    case Hour = 'hour';

    public function timePassedSince(Carbon $date): int
    {
        return match ($this) {
            self::Day => now()->diffInDays($date),
            self::Hour => now()->diffInHours($date),
        };
    }
}
```

It returns the time passed since the specific date. Either in days or hours.

With this simple helper method, we determine if **enough** time has passed to send a `SequenceMail`:

```
class SequenceMail extends BaseModel implements Sendable
{
    public function enoughTimePassedSince(SentMail $mail): bool
    {
        return $this
            ->schedule
            ->unit
            ->timePassedSince($mail->sent_at) ≥ $this->schedule->delay;
    }
}
```

If the schedule has a unit of day, both the `delay` and the result of `timePassedSince` are in days. This method can be used as:

```
$sequenceMail->enoughTimePassedSince($subscriber->received_mails()->last());
```

Using this method, we can determine if the `SequenceMail` can be sent or not. However, it would be very useful (you'll see in a minute) if we could do something like that:

```
$subscriber->tooEarlyFor($sequenceMail);
```

So let's make it possible. First, we need (don't really need, but it's a nice addition) a special relationship in the `Subscriber` model:

```
public function last_received_mail(): HasOne
{
    return $this->hasOne(SentMail::class)
        ->latestOfMany()
        ->withDefault();
}
```

Two important things:

- The `latestOfMany` makes it possible to create a `HasOne` relationship from a `HasMany`. A `Subscriber` has many `SentMail`, but in this case, we're only interested in the latest one.
- The `withDefault` will return a new, empty `SentMail` model if a subscriber does not yet have a `SentMail`. It avoids `Trying to get property on null` bugs.

Now let's add a helper method that uses this relationship:

```
public function tooEarlyFor(SequenceMail $mail): bool
{
    return !$mail->enoughTimePassedSince($this->last_received_mail);
}
```

All right. Take a deep breath, and let me summarize these methods in one snippet:

```
class Subscriber extends BaseModel
{
    public function last_received_mail(): HasOne
    {
        return $this->hasOne(SentMail::class)
            ->latestOfMany()
            ->withDefault();
    }

    public function tooEarlyFor(SequenceMail $mail): bool
    {
        return !$mail->enoughTimePassedSince($this->last_received_mail);
    }
}

class SequenceMail extends BaseModel implements Sendable
{
    public function enoughTimePassedSince(SentMail $mail): bool
    {
        return $this
            ->schedule
            ->unit
            ->timePassedSince($mail->sent_at) ≥ $this->schedule->delay;
    }
}
```

```
enum SequenceMailUnit: string
{
    case Day = 'day';
    case Hour = 'hour';

    public function timePassedSince(Carbon $date): int
    {
        return match ($this) {
            self::Day => now()->diffInDays($date),
            self::Hour => now()->diffInHours($date),
        };
    }
}
```

Putting it all together

```
namespace Domain\Mail\Actions\Sequence;

class ProceedSequenceAction
{
    public static function execute(Sequence $sequence): int
    {
        foreach ($sequence->mails()->wherePublished()->get() as $mail) {
            $subscribers = self::subscribers($mail);
            // ...
        }
    }

    private static function subscribers(SequenceMail $mail): Collection
    {
        if (!$mail->shouldSendToday()) {
            return collect([]);
        }

        return FilterSubscribersAction::execute($mail)
            ->reject->alreadyReceived($mail)
            ->reject->tooEarlyFor($mail);
    }
}
```

These are the three methods that will determine if a `SequenceMail` can be sent to a `Subscriber` and this is how they are being used. Now you can see why I created the `tooEarlyFor` method in the `Subscriber` method.

A quick note before moving on. These methods return `bool` values. If you always use the `is`, `has`, `should`, `can` prefixes, They become:

- `shouldSendToday` OR `canSendToday`
- `hasAlreadyReceived`
- `isTooEarlyFor`

Changes in FilterSubscribersAction

Now that we use the same action with sequence mails, we need to make a small change. Right now, this action accepts a `Broadcast`. To filter sequence mails, too, it needs to accept a `Sendable`:

```
class FilterSubscribersAction
{
    /**
     * @return Collection<Subscriber>
     */
    public static function execute(Sendable $mail): Collection
    {
        return app(Pipeline::class)
            →send(Subscriber::query())
            →through(self::filters($mail))
            →thenReturn()
            →get();
    }

    /**
     * @return array<Filter>
     */
    public static function filters(Sendable $mail): array
    {
        return collect($mail→filters()→toArray())
            →map(fn (array $ids, string $key) ⇒
                Filters::from($key)→createFilter($ids)
            )
            →values()
            →all();
    }
}
```

It does not require too much change, but this line `collect($mail→filters()→toArray())` requires a `filters` method in the `Sendable` interface. It's a very small change:

```

namespace Domain\Mail\Contracts;

use Domain\Mail\DataTransferObjects\FILTERData;

interface Sendable
{
    public function id(): int;
    public function subject(): string;
    public function content(): string;
    public function type(): string;
    public function filters(): FILTERData;
}

class Broadcast extends BaseModel implements Sendable
{
    public function filters(): FILTERData
    {
        return $this->filters;
    }
}

class SequenceMail extends BaseModel implements Sendable
{
    public function filters(): FILTERData
    {
        return $this->filters;
    }
}

```

Both `Broadcast` and `SequenceMail` implement this interface by returning some very basic attributes. So it can be a good idea to promote it to an abstract class that implements these methods by default. This way, you can remove duplicated code from `Broadcast` and `SequenceMail`.

It's important to note that a trait doesn't make sense in this scenario for the following reasons:

- First, a trait can be used to add behavior to a class. Sort of like a "plug and play." In this case, the `Sendable` interface has no behavior. It's just an abstraction above two similar yet different classes. So conceptually, a trait doesn't make any sense.
- But more importantly, you cannot type-hint a trait. So a method cannot have a `HasUser` argument, for example.

Now, back to the `ProceedSequenceAction` because it's not complete yet.

```
public static function execute(Sequence $sequence): int
{
    $sentMailCount = 0;

    foreach ($sequence->mails()->wherePublished()->get() as $mail) {
        $subscribers = self::subscribers($mail);

        foreach ($subscribers as $subscriber) {
            Mail::to($subscriber)->queue(new EchoMail($mail));

            $mail->sent_mails()->create([
                'subscriber_id' => $subscriber->id,
                'user_id' => $sequence->user->id,
            ]);
        }

        $sentMailCount += $subscribers->count();
    }

    return $sentMailCount;
}
```

After filtering the subscribers, it queues the e-mails and creates the `sent_mails` records. After sending a sequence mail, it also counts the number of subscribers so it can return the total number of e-mails that have been sent.

To use the `EchoMail` with a `SequenceMail` we need to make a small change. It needs to accept a `Sendable` as its argument. Broadcasts and sequence mails are both implementing the `Sendable` interface.

```
namespace Domain\Mail\Mails;

class EchoMail extends Mailable implements ShouldQueue
{
    use Queueable, SerializesModels;

    public function __construct(public readonly Sendable $mail)
    {
    }

    public function build()
    {
        return $this
            ->subject($this->mail->subject())
            ->view('emails.echo');
    }
}
```

And in the Blade template, we can use the `content()` method defined in the `Sendable` interface:

```
{!! $mail->content() !!}
```

Now the `EchoMail` and the template should work with both broadcasts and sequence mails.

In the first `foreach`, we query only the published mails for the given sequence. A simple scope implements it in the `SequenceMailBuilder` class:

```

namespace Domain\Mail\Builders\Sequence;

use Domain\Mail\Enums\Sequence\SequenceMailStatus;
use Illuminate\Database\Eloquent\Builder;

class SequenceMailBuilder extends Builder
{
    public function wherePublished(): self
    {
        return $this->whereStatus(SequenceMailStatus::Published);
    }
}

```

Now the app can proceed with a sequence. However, we still need a command that triggers this action.

```

namespace App\Console\Commands\Mail;

use Domain\Mail\Jobs\Sequence\ProceedSequenceJob;
use Domain\Mail\Models\Sequence\Sequence;
use Domain\Mail\Enums\Sequence\SequenceStatus;
use Illuminate\Console\Command;

class ProceedSequencesCommand extends Command
{
    protected $signature = 'sequence:proceed';
    protected $description = 'Send the next mail in sequences';

    public function handle(): int
    {
        $count = Sequence::with('mails.schedule')
            ->whereStatus(SequenceStatus::Published)
            ->get()
            ->each(fn (Sequence $sequence) =>
                ProceedSequenceJob::dispatch($sequence)
            )
            ->count();
    }
}

```

```

        $this->info("{ $count} sequences are being proceeded");

        return self::SUCCESS;
    }
}

```

It queries published sequences only dispatches the `ProceedSequenceJob`. This job is only a placeholder that triggers the action. It's required to queue the actions.

```

namespace Domain\Mail\Jobs\Sequence;

use Domain\Mail\Actions\Sequence\ProceedSequenceAction;

class ProceedSequenceJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(private readonly Sequence $sequence)
    {
    }

    public function handle()
    {
        ProceedSequenceAction::execute($this->sequence);
    }
}

```

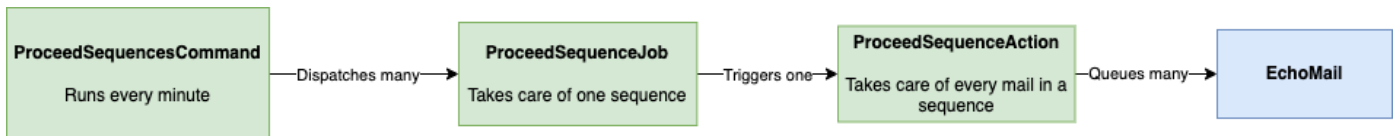
Now we can schedule the command:

```

class Kernel extends ConsoleKernel
{
    protected function schedule(Schedule $schedule)
    {
        $schedule->command('sequence:proceed')
            ->everyMinute()
            ->appendOutputTo(storage_path('logs/scheduler.log'));
    }
}

```

To sum it up in one context diagram:



Refactoring

Earlier I lied to you. Yet the `FilterSubscribersAction` won't work the way we expect. There's a problem with this function:

```
/**
 * @return Collection<Subscriber>
 */
public static function execute(Sendable $mail): Collection
{
    return app(Pipeline::class)
        ->send(Subscriber::query())
        ->through(self::filters($mail))
        ->thenReturn()
        ->get();
}
```

The problem is the `Subscriber::query()` call. A sequence (and a sequence mail) has its own audience in the `sequence_subscriber` table. So by calling `Subscriber::query()`, we throw out of the window the actual audience and simply query everyone. Fortunately, it's a straightforward fix. In the case of a `SequenceMail`, we need to use the sequence's audience:

```

/**
 * @return Collection<Subscriber>
 */
public static function execute(Sendable $mail): Collection
{
    $subscribers = Subscriber::query();

    if ($mail instanceof SequenceMail) {
        $subscribers = Subscriber::query()
            →whereIn(
                'id',
                $mail→sequence→subscribers()→select('subscribers.id')→pluck('id')
            );
    }

    return app(Pipeline::class)
        →send($subscribers)
        →through(self::filters($mail))
        →thenReturn()
        →get();
}

```

If the `$mail` is a `SequenceMail`, we query the subscriber IDs from the `sequence_subscriber` table. This should work as expected. What about the `instanceof` workaround? It certainly does not look good. Here's my opinion on this problem:

- If you have only one `if` that looks like some "workaround," I think it's okay. Generally, it's not the end of the world. If this function won't change that much, and you won't have other use-cases and more "workarounds," it's okay. At the end of the day, it's just a single `if` statement.
- When you need another `instanceof` or any other "hack," that's the exact moment I would start refactoring.

But to show you an example, this is how you can refactor such a thing. First of all, what's the cause of the "life-hack"? It's the difference between the audience of a `Broadcast` and a `SequenceMail`:

- A `Broadcast` has an audience of everyone. No matter what.
- A `SequenceMail` has an audience of subscribers who were subscribed when the `Sequence` was created.

By defining the problem, we got a new concept: the audience. It seems to me that an audience can be another abstraction in the `Sendable` interface:

```
namespace Domain\Mail\Contracts;

use Domain\Mail\DataTransferObjects\FILTERData;

interface Sendable
{
    public function id(): int;
    public function subject(): string;
    public function content(): string;
    public function type(): string;
    public function filters(): FILTERData;
    public function audience(): Builder;
}
```

I like this interface. It describes what defines a 'sendable mail' perfectly in our application:

- It has a subject line. Subscribers will see this in their e-mail client.
- It has content. The purpose of e-mail marketing software is content, right?
- It has a type because we have more than one thing that can be sent to subscribers.
- It has filters. Users don't want to send everything to everyone.
- And now it has an audience.

It sounds great, isn't it? But now we have another problem. A sendable has:

- Filters
- But also an audience?

It isn't very clear. Is the audience contain only filtered subscribers? No, and as a bonus confusion, it returns a Builder. Hmm, maybe it's just the name that is confusing. Let's try a different one:

```
interface Sendable
{
    public function id(): int;
    public function subject(): string;
    public function content(): string;
    public function type(): string;
    public function filters(): FILTERData;
    public function query(): Builder;
}
```

Now, what the heck is a query? It can be anything, basically. So the problem is not the name. It's the fact that we're using an action to filter the subscribers. Unfortunately, the `FilterSubscribersAction` has to die.

We wouldn't have this problem if the filter logic was implemented in the models. It would look like this:

```
interface Sendable
{
    public function id(): int;
    public function subject(): string;
    public function content(): string;
    public function type(): string;
    public function audience(): Collection;
}

class Broadcast extends BaseModel implements Sendable
{
    public function audience(): Collection
    {
        foreach ($this->filters as $filter)
        {
            // ...
        }
    }
}

class SequenceMail extends BaseModel implements Sendable
{
    public function audience(): Collection
    {
        foreach ($this->filters as $filter)
        {
            // ...
        }
    }
}
```

But of course, we don't want to duplicate the code in two models, and generally speaking, I try to avoid overloading models with query logic. Does any of it ring a bell? We want to "plug" some behavior into specific models. Yeah, it's a trait! Since we've discovered the concept of an audience, let's call it

`HasAudience`:

```
namespace Domain\Subscriber\Models\Concerns;

trait HasAudience
{
    /**
     * @return Collection<Subscriber>
     */
    public function audience(): Collection
    {
        $filters = collect($this->filters->toArray())
            ->map(fn(array $ids, string $key) =>
                Filters::from($key)->createFilter($ids)
            )
            ->values()
            ->all();

        return app(Pipeline::class)
            ->send($this->audienceQuery())
            ->through($filters)
            ->thenReturn()
            ->get();
    }
}
```

There are two assumptions here:

- The model that uses the trait has to have a `filters` attribute.
- And also an `audienceQuery` method.

It's kind of a "standard thing" to have these assumptions when people are working with traits, but I don't really like it. It's simply not strict enough and can cause unnecessary bugs. Did you know that you can write abstract methods in a trait? This solves the issue and looks very neat:

```

namespace Domain\Subscriber\Models\Concerns;

trait HasAudience
{
    abstract public function filters(): FilterData;
    abstract protected function audienceQuery(): Builder;

    /**
     * @return Collection<Subscriber>
     */
    public function audience(): Collection
    {
        $filters = collect($this->filters()->toArray())
            ->map(fn(array $ids, string $key) =>
                Filters::from($key)->createFilter($ids)
            )
            ->values()
            ->all();

        return app(Pipeline::class)
            ->send($this->audienceQuery())
            ->through($filters)
            ->thenReturn()
            ->get();
    }
}

```

Now it does not have any assumptions. Instead, it defines a contract: any model that implements this trait must define these two methods. I like it much better!

And now the difference between a `Broadcast` and a `SequenceMail` doesn't require an `if` statement. But instead, the two models can define a one-liner:

```

class Broadcast extends BaseModel implements Sendable
{
    use WithData;
    use HasUser;
    use HasAudience;

    protected function audienceQuery(): Builder
    {
        return Subscriber::query();
    }
}

class SequenceMail extends BaseModel implements Sendable
{
    use HasAudience;

    protected function audienceQuery(): Builder
    {
        return Subscriber::whereIn(
            'id',
            $this->sequence->subscribers()->select('subscribers.id')->pluck('id')
        );
    }
}

```

As you can see, I also moved the `filters` from the `Sendable` interface to the `HasAudience` trait. Now, the `Sendable` looks like this:

```

interface Sendable
{
    public function id(): int;
    public function subject(): string;
    public function content(): string;
    public function type(): string;
}

```

And if you think about it, what makes something "sendable"?

- A subject line that subscribers will see.
- Content that subscribers will read.
- A type and an id so it can be identified.

Do you really need filters to send something? Not really. So, conceptually, it's a good thing that it doesn't have a `filters` method anymore.

The `FilterSubscribersAction` was used in two places. When sending a broadcast:

```
public static function execute(Broadcast $broadcast): int
{
    // Instead of: FilterSubscribersAction::execute($broadcast)
    $subscribers = $broadcast->audience()
        =>each(fn (Subscriber $subscriber) =>
            Mail::to($subscriber)->queue(new EchoMail($broadcast))
        );
}
```

And when proceeding with a sequence:

```
private static function subscribers(SequenceMail $mail): Collection
{
    if (!$mail->shouldSendToday()) {
        return collect([]);
    }

    // Instead of: FilterSubscribersAction::execute($mail)
    return $mail->audience()
        =>reject->alreadyReceived($mail)
        =>reject->tooEarlyFor($mail);
}
```

By the way, both classes look more natural to me using the `$mail->audience()` function.

So we did all of this refactoring just to eliminate a single if statement:

```
public static function execute(Sendable $mail): Collection
{
    $subscribers = Subscriber::query();

    if ($mail instanceof SequenceMail) {
        $subscribers = Subscriber::query()
            →whereIn(
                'id',
                $mail→sequence→subscribers()→select('subscribers.id')→pluck('id')
            );
    }
}
```

Was it worth it? You can easily fool yourself by saying: "Yes because the overall architecture of the app has become much better." But here's my question: is it really a better design? And here's the truth: **we don't know it yet**. Only time will tell. If in the future we have more scenarios when we need to filter subscribers, then yes, it **might be** a better design. But not necessary; for example, if the `audienceQuery` is always a simple `Subscriber::query` then the original action was a more straightforward solution. But if the `audienceQuery` is getting more complicated with every new use case, we did the right thing.

Updating The Subscriber's Status

In progress

One more task needs to be done when proceeding with a sequence. It's the status update for a particular subscriber. Basically, there are two steps here:

- Marking every subscriber as in progress when queueing the e-mails for a sequence mail.
- Marking some of the subscribers as completed after the action is done with every sequence mail.

The first step is easy; all we need to do is a batch update for a collection of subscribers:

```
namespace Domain\Mail\Actions\Sequence;

class ProceedSequenceAction
{
    public static function execute(Sequence $sequence): int
    {
        $sentMailCount = 0;

        foreach ($sequence->mails()->wherePublished()->get() as $mail) {
            $subscribers = self::subscribers($mail);

            foreach ($subscribers as $subscriber) {
                // Queueing e-mails, inserting sent_mails records
            }

            self::markAsInProgress($sequence, $subscribers);

            $sentMailCount += $subscribers->count();
        }

        return $sentMailCount;
    }
}
```

After the inner loop has done queueing the e-mails and inserting `sent_mails` records, we can safely mark these subscribers as in progress:

```

/**
 * @param Sequence $sequence
 * @param Collection<Subscriber> $subscribers
 */
private static function markAsInProgress(Sequence $sequence, Collection
$subscribers): void
{
    SequenceSubscriber::query()
        ->whereBelongsTo($sequence)
        ->whereIn('subscriber_id', $schedulableAudience->pluck('id'))
        ->update([
            'status' => SubscriberStatus::InProgress,
        ]);
}

```

The `query` method doesn't do anything special; I just use it because this way, all the `where` statements start on a new line.

Completed

This part is much trickier than it sounds. We have several different approaches:

- We can count the number of e-mails a subscriber has received and compare it to the number of e-mails the sequence has. This solution works, but it's a bit more complicated than it sounds. We cannot just simply count the number of sequence mails because each mail has a different audience. So a sequence may have eight e-mails, and a subscriber has received six of them, but s/he has already completed the sequence because s/he is not part of the other two mails' audience. So this is how this approach can work:
 - Query every sequence mails where the subscriber is part of the audience.
 - Count these mails and compare them to the number of received mails.
- There's another more "tricky" approach:
 - Query the one sequence mail with the longest delay. For example, if a sequence has two mails, one with a three-hour delay and one with a two-day delay, the second one has the longest delay.
 - Query the last mail received by the subscriber from the current sequence.
 - Calculate the time difference between the current date and the latest received mail.
 - If this difference is greater than the longest delay (from step #1), the subscriber has completed the sequence.

We're going with the first "proper" approach. I just wanted to give you an idea. Sometimes it's good if you try to think about other, alternative solutions to a problem. In my experience, they might give you a better solution from a performance point of view.

So let's jump in. It's not going to be easy, but hey, you wanted to learn about complicated topics, so here we go. This is what the refactored `ProceedSequenceAction` looks like:

```

class ProceedSequenceAction
{
    /**
     * @var array<int, array<int>>
     */
    private static array $mailsBySubscribers = [];

    public static function execute(Sequence $sequence): void
    {
        foreach ($sequence->mails()->wherePublished()->get() as $mail) {
            [$audience, $schedulableAudience] = self::audience($mail);

            self::sendMails($schedulableAudience, $mail, $sequence);

            self::addMailToAudience($audience, $mail);

            self::markAsInProgress($sequence, $schedulableAudience);
        }

        self::markAsCompleted($sequence);
    }
}

```

Let's go through it step by step. The first step is the `audience` method:

```

/**
 * @return array<Collection<Subscriber>>
 */
private static function audience(SequenceMail $mail): array
{
    $audience = $mail->audience();

    if (!$mail->shouldSendToday()) {
        return [$audience, collect([])];
    }

    $schedulableAudience = $audience
        ->reject->alreadyReceived($mail)
        ->reject->tooEarlyFor($mail);

    return [$audience, $schedulableAudience];
}

```

There are two new concepts:

- Audience: every subscriber that the `HasAudience` trait returns.
- Schedulable audience: every subscriber who will receive an e-mail right now. In other words: we can schedule an e-mail for these subscribers. Naming stuff is easy.

I'll explain why we need to distinguish these two sets of subscribers in a minute. The next method is the `sendMails`:

```

private static function sendMails(
    Collection $schedulableAudience,
    SequenceMail $mail,
    Sequence $sequence,
): void {
    foreach ($schedulableAudience as $subscriber) {
        Mail::to($subscriber)→queue(new EchoMail($mail));

        $mail→sent_mails()→create([
            'subscriber_id' => $subscriber→id,
            'user_id' => $sequence→user→id,
        ]);
    }
}

```

It queues the mails and creates the `sent_mails` records. There are two caveats here:

I'm using the `schedulableAudience` for the variable name even though it's just an audience from the function's perspective. However, when I'm browsing the whole class, it's very useful to see exactly what kind of audience it is. This is one of the **hidden benefits of actions**. If this function were inside a model, I would have no idea what the `schedulableAudience` means. The concept is out of context in a model with dozens of other methods.

The next method is the `addMailToAudience`:

```

/**
 * @param Collection<Subscriber> $audience
 */
private static function addMailToAudience(
    Collection $audience,
    SequenceMail $mail
): void {
    foreach ($audience as $subscriber) {
        if (!Arr::get(self::$mailsBySubscribers, $subscriber->id)) {
            self::$mailsBySubscribers[$subscriber->id] = [];
        }

        self::$mailsBySubscribers[$subscriber->id][] = $mail->id;
    }
}

```

It builds an array indexed by subscriber ID:

```

[
    12 => [1, 2, 3],
    15 => [2, 3],
];

```

The keys are `Subscriber` IDs, and the values are `SequenceMail` IDs. Why do we need an array like this?

- Later on, when we want to mark subscribers as completed, we need something that tells us how many `SequenceMail` the subscriber should receive from the current `Sequence`. This array is that something.
- The first argument contains the total audience for a `SequenceMail`. The collection that `HasAudience` returns.
- In the example array, there are three mails for subscriber ID 12, right? But it doesn't mean that subscriber 12 has received all three mails. It means s/he **should** receive these mails at some point.
- If we query the `$subscriber->received_mails` for the current sequence and the `count` is three, it means the subscriber completed the sequence!

After that, we have a really simple function:

```

/**
 * @param Sequence $sequence
 * @param Collection<Subscriber> $schedulableAudience
 */
private static function markAsInProgress(
    Sequence $sequence,
    Collection $schedulableAudience
): void {
    $sequence
        →subscribers()
        →whereIn('subscriber_id', $schedulableAudience→pluck('id'))
        →update([
            'status' ⇒ SubscriberStatus::InProgress,
        ]);
}

```

It just updates every subscriber's status to in progress. Once again, it works with the `schedulableAudience`, so every subscriber who received an e-mail. They are progressing through the sequence for sure.

And the last step is to mark the subscribers who have completed the sequence:

```

public static function markAsCompleted(Sequence $sequence): void
{
    $subscribers = Subscriber::withCount([
        'received_mails' ⇒ fn (Builder $receivedMails) ⇒
            $receivedMails→whereSequence($sequence)
    ])
    →find(array_keys(self::$mailsBySubscribers))
    →mapWithKeys(fn (Subscriber $subscriber) ⇒ [
        $subscriber→id ⇒ $subscriber,
    ]);

    $completedSubscriberIds = [];
    foreach (self::$mailsBySubscribers as $subscriberId ⇒ $mailIds) {
        $subscriber = $subscribers[$subscriberId];
    }
}

```

```

if ($subscriber->received_mails_count === count($mailIds)) {
    $completedSubscriberIds[] = $subscriber->id;
}
}

```

```

SequenceSubscriber::query()
    ->whereBelongsTo($sequence)
    ->whereIn('subscriber_id', $completedSubscriberIds)
    ->update([
        'status' => SubscriberStatus::Completed,
    ]);
}

```

Earlier the `addMailToAudience` function has collected all of the subscriber IDs. The first part of the `markAsCompleted` function uses this array to get every subscriber in one database query eager loaded with the count of `sent_mails`. These things are important because it's very easy to cause N+1 query problems. I'm also using the `mapWithKeys`. This is unnecessary, but it's essential from a performance point of view. I'll explain it in a minute.

The second part of the function will collect every subscriber that can be marked as completed. Or in other words, they received every mail from the sequence they should receive. Once again, the `$mailsBySubscribers` array contains every `Subscriber` ID that is part of the audience of any mail of the sequence. It also contains the `SequenceMail` IDs that were sent to the subscriber. Here's the reason why I used `mapWithKeys` earlier. Without it, this line `$subscriber = $subscribers[$subscriberId];` would look like this:

```

$subscriber = $subscribers->where('id', $subscriberId)->first();

```

If the `$subscribers` collection contains 50000 items, this `where` will run a loop potentially 50000 times. And it's already inside a `foreach`! This `foreach` can also run 50000 times. So in this scenario, we could end up in a situation where we scan the `$subscribers` array 2 500 000 000 times. That's 2.5 billion with a B. This is called quadratic time complexity and can be expressed as $O(n^2)$. I use the `$subscriber` collection as a hash map to avoid that. This is beneficial because

```

$subscriber = $subscribers[$subscriberId];

```

This line has constant time complexity. So there's no extra loop to get the subscriber; it's as simple as accessing an array key. This can be expressed as $O(1)$. But this line is still inside a for loop, so the real complexity is $O(n)$ or linear.

The key takeaway: try to avoid any expensive operation inside a `foreach`. Expensive operations can be:

- Querying the database. Pay extra attention to avoid accessing relationships that are not eager-loaded. This is why I used the `withCount('received_mails')` before the loop.
- Looping through huge collections. Let's say you have a collection with 10000 items and a `where` call runs for 100ms. Nothing out of the ordinary, right? Now put this 100ms operation inside a `foreach` that loops through another 10000 items. $10000 * 100\text{ms}$ is exactly 1000 seconds or 16 hours. That's right; the quadratic time complexity is no joke. And as you can see, you don't need database queries to cause trouble.
- "Tricky" array functions. For example, `array_splice` or anything else that will move a part of the array from an index to another one. They are relatively expensive compared to a simple `$items[] = $item;`
- The obvious ones: sending e-mails synchronously, accessing files, making HTTP requests, checking permissions from the database.

Now that we've discussed this one single line, let's move on to the next important one:

```
if ($subscriber->received_mails_count === count($mailIds)) {
    $completedSubscriberIds[] = $subscriber->id;
}
```

It determines if a subscriber can be marked as completed or not. Please note that I'm not firing an update query inside the if. Instead, I'm collecting the IDs to avoid N+1 query problems. And this brings us to the last part, where the actual update is happening:

```
SequenceSubscriber::query()
    ->whereBelongsTo($sequence)
    ->whereIn('subscriber_id', $completedSubscriberIds)
    ->update([
        'status' => SubscriberStatus::Completed,
    ]);
```

One database query that takes care of every subscriber.

Although using the `$mailsBySubscribers` array isn't the most elegant solution, it allows us to update every subscriber with only two database queries. This is a class where performance is more important than clean code.

Once again, the whole method in its full glory:

```

public static function markAsCompleted(Sequence $sequence): void
{
    $subscribers = Subscriber::withCount('received_mails')
        ->find(array_keys(self::$mailsBySubscribers))
        ->mapWithKeys(fn (Subscriber $subscriber) => [
            $subscriber->id => $subscriber,
        ]);

    $completedSubscriberIds = [];
    foreach (self::$mailsBySubscribers as $subscriberId => $mailIds) {
        $subscriber = $subscribers[$subscriberId];

        if ($subscriber->received_mails_count === count($mailIds)) {
            $completedSubscriberIds[] = $subscriber->id;
        }
    }

    SequenceSubscriber::query()
        ->whereBelongsTo($sequence)
        ->whereIn('subscriber_id', $completedSubscriberIds)
        ->update([
            'status' => SubscriberStatus::Completed,
        ]);
}

```

This is not the best method from a clean code perspective, but this is another **hidden benefit of actions**. This is an entirely isolated class that does only one thing. In this one class, the logic is perfectly understandable and can be maintained easily. For example, if this function were inside a model, it would be much harder to understand what's going on, especially with the `$mailsBySubscribers` array.

And lastly, a quick note about the `get` and `set` prefixes. Take a look at this function:

```
public static function execute(Sequence $sequence): void
{
    foreach ($sequence->mails()->wherePublished()->get() as $mail) {
        [$audience, $schedulableAudience] = self::audience($mail);

        self::sendMails($schedulableAudience, $mail, $sequence);

        self::addMailToAudience($audience, $mail);

        self::markAsInProgress($sequence, $schedulableAudience);
    }

    self::markAsCompleted($sequence);
}
```

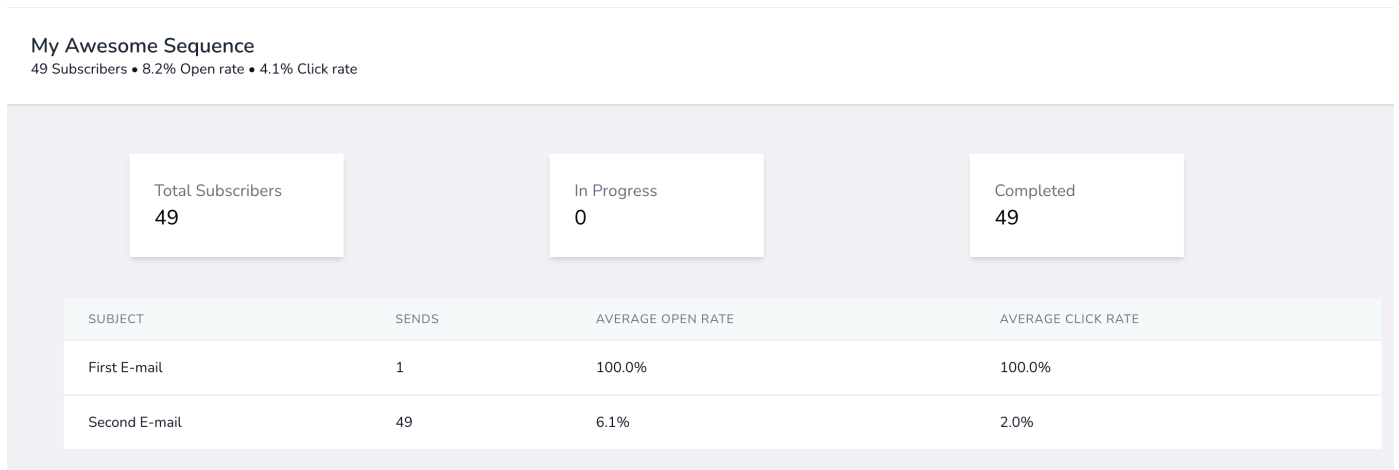
If you're in the mindset of not having setters at all, it's very easy to understand this. `audience` is a getter, and everything else is an "action."

Calculating The Performance Of A Sequence

Actually, we need to calculate two different kinds of performances:

- Sequence
- Sequence Mail

As I said in the design chapter, the performance of a sequence mail can be measured in isolation, similar to a broadcast. But a sequence's performance can also be measured as the "container" of the sequence mails. You can take a look at this screenshot:



In the upper left corner, you can see the overall performance of the whole sequence; meanwhile, in the list, you can see each mail's performance.

Right now, there's a `GetPerformanceAction` that looks like this:

```
namespace Domain\Mail\Actions;

use Domain\Mail\Contracts\Sendable;
use Domain\Mail\Models\SentMail;
use Domain\Mail\DataTransferObjects\PerformanceData;

class GetPerformanceAction
{
    public static function execute(Sendable $model): PerformanceData
    {
        $total = SentMail::countOf($model);

        return new PerformanceData(
            total: $total,
            open_rate: SentMail::openRate($model, $total),
            click_rate: SentMail::clickRate($model, $total),
        );
    }
}
```

```
);
}
}
```

And the good news is: it requires no change to work with `SequenceMail` since it also implements the `Sendable` interface. The bad news is: it won't work with a `Sequence` for these reasons:

- A `Sequence` is not sendable.
- When it comes to sequence, the `total` attribute is not going to be the number of `sent_mails` but the number of subscribers, as you can see on the screenshot.

We can apply a quick "life hack":

```
public static function execute(Sendable|Sequence $model): PerformanceData
{
    if ($model instanceof Sequence) {
        $total = $model->activeSubscriberCount();
    } else {
        $total = SentMail::countOf($model);
    }

    return new PerformanceData(
        total: $total,
        open_rate: SentMail::openRate($model, $total),
        click_rate: SentMail::clickRate($model, $total),
    );
}
```

Does any of it ring a bell? This is the same problem we encountered when dealing with the `FilterSubscribersAction`. There's a pattern here: whenever you're working with an action that deals with multiple types of models, using traits and polymorphism **might be** a better solution.

And in general, writing actions to get some data **might not be** the best solution. There's nothing wrong with getter actions, but as you can see in this particular example, they cause us some trouble.

So let's solve this problem by creating a new trait:

```

namespace Domain\Mail\Models\Concerns;

use Domain\Mail\DataTransferObjects\PerformanceData;
use Domain\Shared\ValueObjects\Percent;
use Illuminate\Database\Eloquent\Relations\Relation;

trait HasPerformance
{
    abstract public function performance(): PerformanceData;
    abstract public function sent_mails(): Relation;

    public function openRate(int $total): Percent
    {
        return Percent::from(
            $this->sent_mails()->whereOpened()->count(), $total
        );
    }

    public function clickRate(int $total): Percent
    {
        return Percent::from(
            $this->sent_mails()->whereClicked()->count(), $total
        );
    }
}

```

As you can see, I moved the `openRate` and `clickRate` methods into this trait. Previously they lived in the `SentMailBuilder` class. Either of those can be a good place. Here are my thoughts on this decision:

- **Technically** they belong to the `SentMailBuilder` because they query data from the `sent_mails` table.
- But **conceptually**, the `HasPerformance` trait is a better place for them, in my opinion. This trait will be used in `Broadcast`, `SequenceMail`, and the `Sequence` model. We can get the open rate by writing `$broadcast->openRate()`. So conceptually, I mean: `Broadcast` is the one that has an open rate, not the `SentMail` model.

By creating this trait, we have a perfect place for these methods. The trait also defines a `sent_mails` method. By adding this one, we have a more strict contract. And finally, it defines the `performance` method, which the individual models implement:

```

class Broadcast extends BaseModel implements Sendable
{
    use WithData;
    use HasUser;
    use HasAudience;
    use HasPerformance;

    public function performance(): PerformanceData
    {
        $total = SentMail::countOf($this);

        return new PerformanceData(
            total: $total,
            open_rate: $this->openRate($total),
            click_rate: $this->clickRate($total),
        );
    }
}

class SequenceMail extends BaseModel implements Sendable
{
    use HasAudience;
    use HasPerformance;

    public function performance(): PerformanceData
    {
        $total = SentMail::countOf($this);

        return new PerformanceData(
            total: $total,
            open_rate: $this->openRate($total),
            click_rate: $this->clickRate($total),
        );
    }
}

```

```

class Sequence extends BaseModel
{
    use WithData;
    use HasUser;
    use HasPerformance;

    public function performance(): PerformanceData
    {
        $total = $this->activeSubscriberCount();

        return new PerformanceData(
            total: $total,
            open_rate: $this->openRate($total),
            click_rate: $this->clickRate($total),
        );
    }
}

```

There are some duplications between these methods. We can remove this by using a template method; it would look something like this:

```

trait HasPerformance
{
    abstract public function totalInstances(): int;

    public function performance(): PerformanceData
    {
        $total = $this->totalInstances();

        return new PerformanceData(
            total: $total,
            open_rate: $this->openRate($total),
            click_rate: $this->clickRate($total),
        );
    }
}

```

And now, the models only need to override the `totalInstances` method:

```
class Broadcast extends BaseModel implements Sendable
{
    use WithData;
    use HasUser;
    use HasAudience;
    use HasPerformance;

    public function totalInstances(): int
    {
        return SentMail::countOf($this);
    }
}
```

It's a cool technique to remove these kinds of duplications where the difference is only a few lines of code. However, the main disadvantage is:

- Earlier, when I looked at the model's `performance` method, I knew exactly what it did.
- Now I only see a weird one-liner that returns some count.
- Even worse, in the case of `Broadcast` and `SequenceMail`, the total is the count of `sent_mail` records, but when it comes to `Sequences`, the same variable holds the number of active subscribers (completed or in progress). So I had to come up with some general names such as `totalInstances`. Which I don't like to be honest.

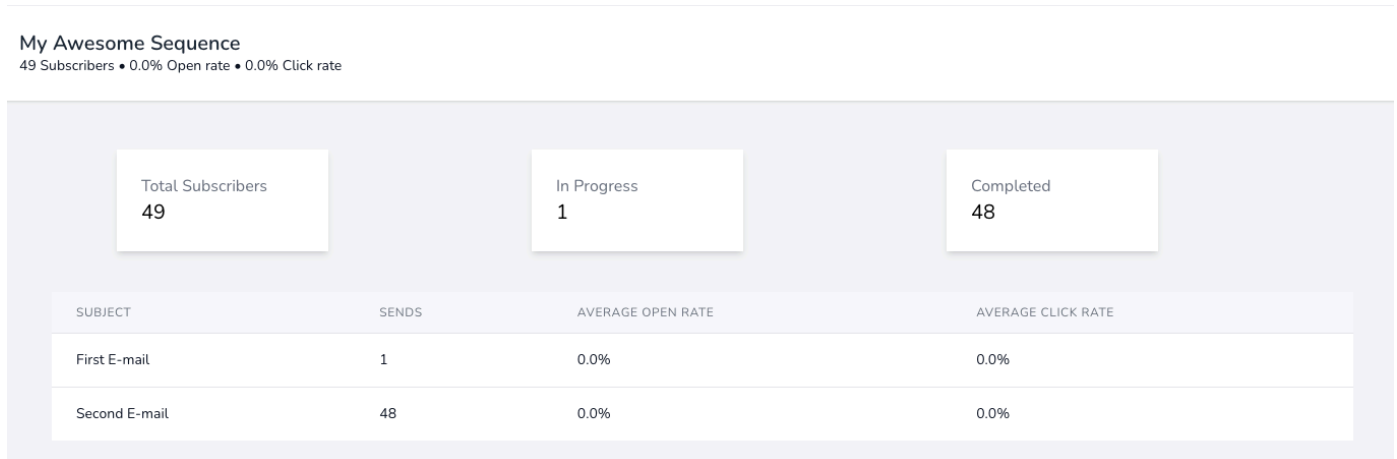
I've decided not to refactor the original `performance` method for these reasons. My general opinion is: sometimes it's okay to have three lines of duplicated code. The moment it causes some trouble, I start to refactor it.

As the last step, we can eliminate the `GetPerformanceAction` and replace it with `$model->performance` calls. For example, in the `GetBroadcastsViewModel`:

```
class GetBroadcastsViewModel extends ViewModel
{
    /**
     * @return Collection<int, PerformanceData>
     */
    public function performances(): Collection
    {
        return Broadcast::all()
            →mapWithKeys(fn (Broadcast $broadcast) ⇒ [
                $broadcast→id ⇒ $broadcast→performance(),
            ]);
    }
}
```

Progress Of A Sequence

The last exciting functionality of sequences is their progress. I'm talking about the three numbers in the boxes:



The meaning of the numbers:

- Total Subscribers: every subscriber who got at least one mail from the sequence. We can call them 'active subscribers'.
- In Progress: the subscriber's status is in progress, meaning they will receive more mail from the sequence.
- Completed: the subscriber's status is completed, meaning they won't receive any more mail from the sequence.

Since we store these statuses in the `sequence_subscriber` table, it's very easy to query them:

```
namespace Domain\Mail\Builders\Sequence;

use Domain\Mail\Enums\Sequence\SubscriberStatus;
use Illuminate\Database\Eloquent\Builder;

class SequenceBuilder extends Builder
{
    public function activeSubscriberCount(): int
    {
        return $this->model
            ->subscribers()
            ->whereNotNull('status')
            ->count();
    }
}
```

```

public function inProgressSubscriberCount(): int
{
    return $this->model
        ->subscribers()
        ->whereStatus(SubscriberStatus::InProgress)
        ->count();
}

public function completedSubscriberCount(): int
{
    return $this->model
        ->subscribers()
        ->whereStatus(SubscriberStatus::Completed)
        ->count();
}
}

```

They go into the `SequenceBuilder` class. We also need a new DTO to store and return these stats:

```

namespace Domain\Mail\DataTransferObjects\Sequence;

use Spatie\LaravelData\Data;

class SequenceProgressData extends Data
{
    public function __construct(
        public readonly int $total,
        public readonly int $in_progress,
        public readonly int $completed,
    ) {}
}

```

And finally, a view model that uses the DTO. This view model also contains:

- The sequence's overall performance.
- The performance of every individual sequence mail.

```

class GetSequenceReportsViewModel extends ViewModel
{
    public function __construct(private readonly Sequence $sequence)
    {
    }

    public function sequence(): SequenceData
    {
        return $this->sequence->getData();
    }

    public function totalPerformance(): PerformanceData
    {
        return $this->sequence->performance();
    }

    /**
     * @return Collection<int, TrackingData>
     */
    public function mailPerformances(): Collection
    {
        return $this->sequence->mails->mapWithKeys(fn (SequenceMail $mail) => [
            $mail->id => $mail->performance()
        ]);
    }

    public function progress(): SequenceProgressData
    {
        return new SequenceProgressData(
            total: $this->sequence->activeSubscriberCount(),
            in_progress: $this->sequence->inProgressSubscriberCount(),
            completed: $this->sequence->completedSubscriberCount(),
        );
    }
}

```

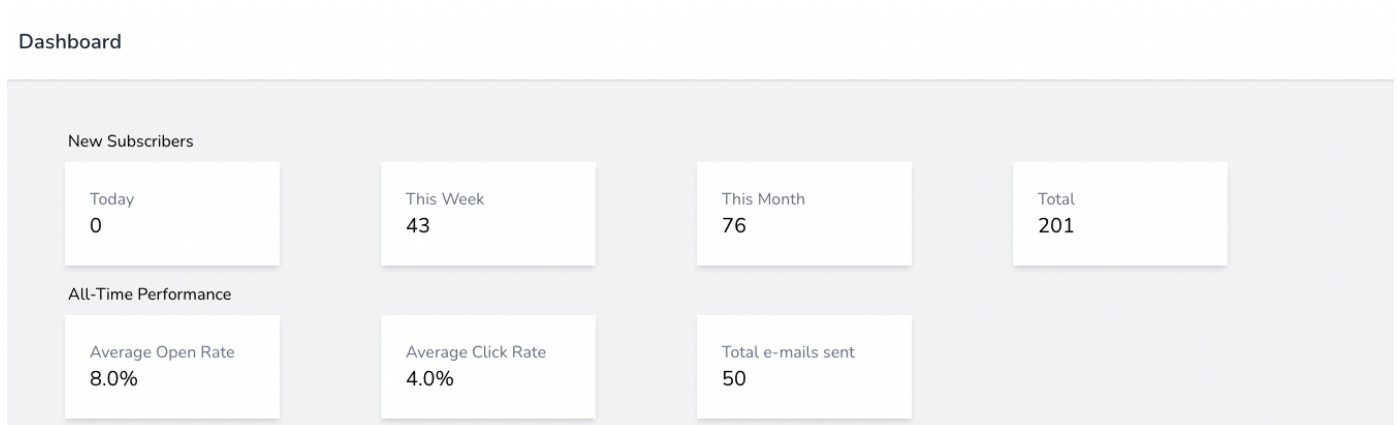
Dashboard And Reports

After all that complicated sequence stuff, let's do something more straightforward: a dashboard and some reports. In my experience, almost every project has features like that.

This application needs four types of reports on the dashboard:

- New subscriber counts
- All-time performance
- Most recent subscribers
- Daily new subscriber counts

New subscriber counts



The four boxes in the first row show how many people subscribed today, this week, this month, or in total. This is a perfect example of some unstructured data that can be improved using a DTO:

```
namespace Domain\Subscriber\DataTransferObjects;

use Spatie\LaravelData\Data;

class NewSubscribersCountData extends Data
{
    public function __construct(
        public readonly int $total,
        public readonly int $this_month,
        public readonly int $this_week,
        public readonly int $today,
    ) {}
}
```

This DTO will be used in the view model:

```
namespace Domain\Shared\ViewModels;

use Domain\Subscriber\DataTransferObjects\NewSubscribersCountData;
use Domain\Subscriber\Models\Subscriber;

class GetDashboardViewModel extends ViewModel
{
    public function newSubscribersCount(): NewSubscribersCountData
    {
        return new NewSubscribersCountData(
            total: 0,
            this_month: 0,
            this_week: 0,
            today: 0,
        );
    }
}
```

As you can see, this class is in the shared domain. There are several reasons:

- It does not belong to any other domain. This class will use subscriber-related classes and queries from the `sent_mail` table.
- This single feature (the dashboard itself) is not big enough to create a separate domain.

Now let's discuss how we're approaching the date filters. One option is this:

```

public function newSubscribersCount(): NewSubscribersCountData
{
    return new NewSubscribersCountData(
        this_month: Subscriber::whereBetween(
            'subscribed_at',
            [
                now() -> startOfMonth(),
                now()
            ]
        ),
        this_week: Subscriber::whereBetween(
            'subscribed_at',
            [
                now() -> startOfWeek(),
                now()
            ]
        ),
    );
}

```

It's pretty nice and readable, and generally speaking, there's nothing wrong with it. However, in projects like this (where there are several other opportunities for more exotic reports), I often find myself writing the same Carbon logic over and over again. And almost certainly, the PM will ask me: *"Can we change the 'This week' report for 'Past seven days' instead?"*. *"Yes, we can."* I answer and immediately see myself using the `Cmd + Shift + F` feature of PhpStorm and going through the 39 occurrences of "startOfWeek." (*Except when we have a new junior dev. Then he can practice using PhpStorm a little bit 😈*)

To avoid these kinds of problems, we can use a very simple `DateFilter` class:

```
namespace Domain\Shared\Filters;

use Carbon\Carbon;

class DateFilter
{
    public function __construct(
        public readonly Carbon $startDate,
        public readonly Carbon $endDate
    ) {}

    public static function today(): self
    {
        return new self(
            startDate: now() -> startOfDay(),
            endDate: now(),
        );
    }

    public static function thisWeek(): self
    {
        return new self(
            startDate: now() -> startOfWeek(),
            endDate: now(),
        );
    }

    public static function thisMonth(): self
    {
        return new self(
            startDate: now() -> startOfMonth(),
            endDate: now(),
        );
    }
}
```

```

public function toArray(): array
{
    return [
        $this->startDate,
        $this->endDate
    ];
}

```

There's nothing complicated going on here. It has some useful factory functions that we can use like this:

```

public function newSubscribersCount(): NewSubscribersCountData
{
    return new NewSubscribersCountData(
        this_month: Subscriber::whereBetween(
            'subscribed_at',
            DateFilter::thisMonth()->toArray()
        ),
        this_week: Subscriber::whereBetween(
            'subscribed_at',
            DateFilter::thisWeek()->toArray()
        ),
    );
}

```

And, of course, we can make it a bit more clean by using a scope in the `SubscriberBuilder` class:

```

namespace Domain\Subscriber\Builders;

use Domain\Shared\Filters\DateFilter;
use Illuminate\Database\Eloquent\Builder;

class SubscriberBuilder extends Builder
{
    public function whereSubscribedBetween(DateFilter $dateFilter): self
    {
        return $this->whereBetween('subscribed_at', $dateFilter->toArray());
    }
}

```

Now the view model looks very clean, in my opinion, and we have a simple, but reusable `DateFilter`:

```

class GetDashboardViewModel extends ViewModel
{
    public function newSubscribersCount(): NewSubscribersCountData
    {
        return new NewSubscribersCountData(
            total: Subscriber::count(),

            this_month: Subscriber::whereSubscribedBetween(
                DateFilter::thisMonth()
            )->count(),

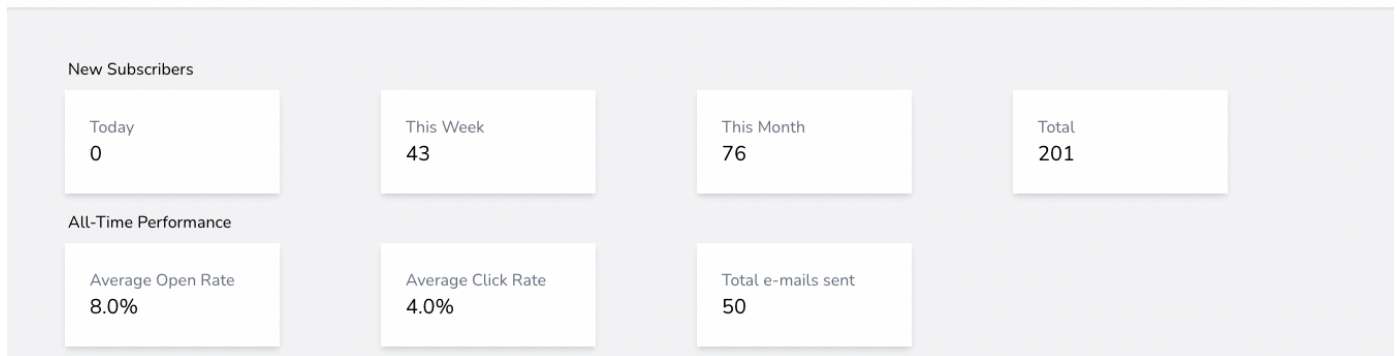
            this_week: Subscriber::whereSubscribedBetween(
                DateFilter::thisWeek()
            )->count(),

            today: Subscriber::whereSubscribedBetween(
                DateFilter::today()
            )->count(),
        );
    }
}

```

All-Time Performance

Dashboard



The three boxes in the second row show you the all-time performance of your e-mails. We already used reports like this for a particular model, such as a broadcast or a sequence mail. Calculating the average open rate for every mail is no more complicated than calculating it for a specific broadcast. In fact, it's much easier. But there's a significant difference:

- The open/click rate of a broadcast is required in multiple places.
- The open/click rate of a sequence mail is required in multiple places.
- The average open rate of every mail is only required on the dashboard.

For that reason, there's no need to extract these queries. They can be some simple functions in the view model:

```

class GetDashboardViewModel extends ViewModel
{
    public function performance(): PerformanceData
    {
        $total = SentMail::count();

        return new PerformanceData(
            total: $total,
            open_rate: $this->averageOpenRate($total),
            click_rate: $this->averageClickRate($total),
        );
    }

    private function averageOpenRate(int $total): Percent
    {
        return Percent::from(
            SentMail::whereOpened()->count(), $total
        );
    }

    private function averageClickRate(int $total): Percent
    {
        return Percent::from(
            SentMail::whereClicked()->count(), $total
        );
    }
}

```

Of course, we can use the same classes as before, the `PerformanceData` DTO and the `Percent` value object.

Subscribers

Daily subscribers

For this feature, the dashboard needs to show a list grouped by day:

Daily Subscribers	
DATE	COUNT
2022-03-12	6
2022-03-11	5
2022-03-10	5
2022-03-09	10
2022-03-08	4
2022-03-07	13

This means that on March 12, 2022, the user had six new subscribers and so on.

This report requires a raw query:

```
/**
 * @return Collection<DailySubscribersData>
 */
public function dailySubscribers(): Collection
{
    return DB::table('subscribers')
        ->select(
            DB::raw("count(*) count, date_format(subscribed_at, '%Y-%m-%d') day")
        )
        ->groupBy('day')
        ->orderByDesc('day')
        ->whereUserId($this->user->id)
        ->get()
        ->map(fn (object $data) => DailySubscribersData::from((array) $data));
}
```

The select statement will return two attributes:

- `count`
- `day`

The query will return the count by day because of the `group by` statement. Since this is not an Eloquent query but a classic `DB::table` one, we **have to** filter by `user_id`. The `HasUser` trait, which is being used in the `Subscriber` model, has no effect here. Because it's a classic database query, it returns a collection of `stdClass`. This is why the `object` type-hint is being used in the `map` function. At this point, you may recognize that I'm not a big fan of arrays or standard objects, so let's map them into DTOs:

```
namespace Domain\Subscriber\DataTransferObjects;

use Spatie\LaravelData\Data;

class DailySubscribersData extends Data
{
    public function __construct(
        public readonly string $day,
        public readonly int $count,
    ) {}
}
```

The result converted to an array looks like this:

```
[
    [
        'day' => '2022-03-12',
        'count' => 6
    ],
];
```

Most recent subscribers

The last and most simple report is the most recent subscribers. It's just a list of subscribers ordered by their subscription date:

Most Recent Subscribers				
FULL NAME	E-MAIL	FORM	SUBSCRIBED AT	TAGS
John Doe	john@doe.io	Free Tutorial	2022-03-12 20:13:29	NewTag
Emmie Veum	welch.sam@example.com	Free Tutorial	2022-03-12 19:10:15	
Cortney Hane	neoma.rutherford@example.net	Free Tutorial	2022-03-12 19:10:15	Laravel Vue MySQL Inertia DDD TDD Clean Code Microservices Docker
Alberto Tromp	antonia01@example.net	Join The Newsletter	2022-03-12 19:10:15	
Frances Jaskolski	mkuphal@example.net	Free Tutorial	2022-03-12 19:10:15	
Queen Raynor	pietro.schmeler@example.net	Free Tutorial	2022-03-12 19:10:15	Laravel Vue MySQL Inertia DDD TDD Clean Code Microservices Docker
Cristian Nietzsche	udare@example.org	Waiting List	2022-03-11 19:10:15	

This is just a simple Eloquent query:

```
/**
 * @return Collection<SubscriberData>
 */
public function recentSubscribers(): Collection
{
    return Subscriber::with(['form', 'tags'])
        ->orderByDesc('subscribed_at')
        ->take(10)
        ->get()
        ->map
        ->getData();
}
```

Automations

The last feature I want to show you is automation. This is going to be one of the most fun features. At its core, automation is something like this:

"When an **event** happens *then* do this **action**."

In this chapter, we are focusing on one type of event:

- When a subscriber subscribes to a form.

And two types of actions:

- Then attach a tag to the subscriber.
- Or add the subscriber to a sequence.

For example, before I released this book, I created two things:

- A landing page where people can submit their e-mail addresses to get informed about the progress and get a discount.
- An e-mail sequence of 8 or so e-mails about DDD concepts.

When someone submitted their e-mail address on the landing page, I added them to my e-mail sequence. This was done by automation in ConvertKit.

As a quick reminder, here's the database design for this feature:



Both events and actions are automation steps, and for now, every automation has only one event. This event triggers the whole automation.

Automation is also a new domain.

Upserting automations

We won't spend much time creating and updating automations because triggering them is a more existing task. But I wanted to show you an example of how you can handle situations when the request has a different shape than the database and the models.

DTO

This is the form where you can create or update automations:

New Automation

NAME

Waiting List -> Sequence

WHEN A SUBSCRIBER

Subscribed To Form

Waiting List

THEN

Add To Sequence

My Awesome Sequence

Remove

Add Action

Save
Cancel

The frontend sends this JSON:

```
{
  "name": "Waiting List → Sequence",
  "steps": {
    "event": {
      "name": "subscribedToForm",
      "value": 1
    },
    "actions": [
      {
        "name": "addToSequence",
        "value": 19
      }
    ]
  }
}
```

It's quite good, but the `steps` key is unnecessary and a bit confusing. It's called `steps`, which is plural, but it's an object instead of an array. Let's assume for a minute that we don't have control over this structure; we need to accept and process it.

We can use a simple DTO like this:

```
namespace Domain\Automation\DataTransferObjects;

class AutomationData extends Data
{
    public function __construct(
        public readonly ?int $id,
        public readonly string $name,
        public readonly AutomationStepData $event,
        /** @var DataCollection<AutomationStepData> */
        public readonly DataCollection $actions,
    ) {}
}
```

As you can see, there are no `steps` here, only `event` and `actions`. This is great but won't work out of the box because laravel-data tries to map the whole request to this DTO. And since they are different, it won't work.

Of course, it can be fixed very easily:

```
public static function fromRequest(Request $request): self
{
    return self::from([
        'id' => $request->id,
        'name' => $request->name,
        'event' => $request->steps['event'],
        'actions' => $request->steps['actions'],
    ]);
}
```

By overriding the `fromRequest`, we can write our own mapping logic, but we can still use the request's structure for validation:

```

public static function rules(): array
{
    return [
        'name' => ['required'],
        'steps' => ['required', 'array'],
        'steps.event' => ['required', 'array'],
        'steps.event.name' => ['required', 'string'],
        'steps.event.value' => ['required', 'numeric'],
        'steps.actions' => ['required', 'array', 'min:1'],
        'steps.actions.*.name' => ['required', 'string'],
        'steps.actions.*.value' => ['required', 'numeric'],
    ];
}

```

We need to make one more small change. When laravel-data tries to validate a request, it will use some automatic rules based on the shape of your DTO. For example, looking at these properties:

```

public function __construct(
    public readonly ?int $id,
    public readonly string $name,
    public readonly AutomationStepData $event,
    /** @var DataCollection<AutomationStepData> */
    public readonly DataCollection $actions,
) {}

```

The package will assume that:

- `id` is optional.
- `name` is a string and is required.
- `event` and `action` are also required.

But it's not true. `event` and `actions` are not required in the request. Fortunately, it's easy to override this default assumption:

```

public static function withValidator(Validator $validator): void
{
    $validator->setRules(self::rules());
}

```

Here we instruct laravel-data to use the array defined in `rules` no matter what properties the class has. Now it works as expected. It will:

- Validate the request by the array returned from `rules`.
- Map the request to the DTO.

Upsert action

Now you can take a look at the action that creates or updates automations:

```
namespace Domain\Automation\Actions;

class UpsertAutomationAction
{
    public static function execute(
        AutomationData $data,
        User $user
    ): Automation {
        return DB::transaction(function () use ($data, $user) {
            $automation = self::upsertAutomation($data, $user);

            $automation->steps->each->delete();

            self::upsertEvent($automation, $data);

            self::upsertActions($automation, $data);

            return $automation->load('steps');
        });
    }
}
```

You can see there are four well-defined steps.

Upserting automation:

```
private static function upsertAutomation(
    AutomationData $data,
    User $user
): Automation {
    return Automation::updateOrCreate(
        [
            'id' => $data->id,
        ],
        [
            ... $data->toArray(),
            'user_id' => $user->id,
        ],
    );
}
```

At this point, I guess it's familiar. After that, the action will delete every step that the automation has right now:

```
$automation->steps->each->delete();
```

It only has an effect when the user is updating existing automation. After the steps are deleted, we can save the event:

```
private static function upsertEvent(
    Automation $automation,
    AutomationData $data
): void {
    $automation->steps()->updateOrCreate(
        [
            'id' => $data->event->id,
        ],
        [
            'type' => AutomationStepType::Event,
            'name' => $data->event->name,
            'value' => ['id' => $data->event->value],
        ]
    );
}
```

```
);  
}
```

For now, I've decided to use a JSON column for the `value`, and it always has this shape:

```
{  
  "id": 1  
}
```

Right now, every automation step will have one "target" or one value and is always an ID. For example:

- When subscribed to a form. In this case, the form ID is a single value.
- Then add to a sequence. In this case, the sequence ID is the single value.

After saving the event, this method takes care of actions:

```

private static function upsertActions(
    Automation $automation,
    AutomationData $data
): void {
    $data->actions->toCollection()->each(fn (AutomationStepData $stepData) =>
        $automation->steps()->updateOrCreate(
            [
                'id' => $stepData->id,
            ],
            [
                'type' => AutomationStepType::Action,
                'name' => $stepData->name,
                'value' => ['id' => $stepData->value],
            ]
        )
    );
}

```

Since the `actions` property in the `AutomationData` class is a `DataCollection`, we need to call the `toCollection` to get a Laravel Collection instead.

And basically, that's the whole action. Now we can move on to the exciting part. How can we trigger and run these automations?

Running automations

We store the event and actions in the `automation_steps` table, which looks like this:

automation_id	type	name	value
1	event	subscribedToForm	{"id": 1}
1	action	addToSequence	{"id": 2}

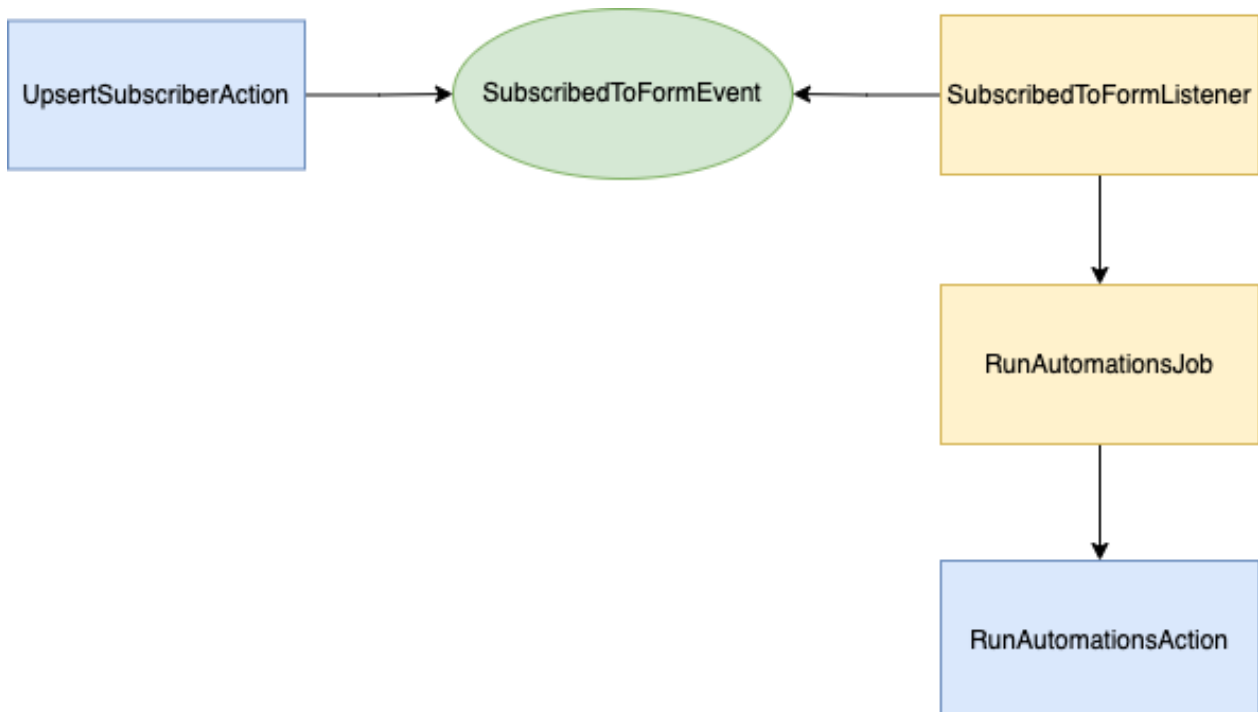
We need some classes representing the `subscribedToForm` event and the `addToSequence` action. If you think about the types, it's a perfect mapping between classes:

- Events are going to be `Event` classes.
- Actions are going to be `Action` classes.

They can be linked together by using an event listener.

Now let's think about *when* automations will be triggered? For now, the only event is when a subscriber subscribes to a form. Put in different words: a new subscriber is created and has a `form_id`. The `UpsertSubscriberAction` seems like an excellent place to start.

So the plan is to dispatch an event after the subscriber has been created. There are going to be a lot of classes, so here's the whole flow in a hand-crafted diagram:



We need to change the `UpsertSubscriberAction`:

```
class UpsertSubscriberAction
{
    public static function execute(
        SubscriberData $data,
        User $user
    ): Subscriber {
        $subscriber = Subscriber::updateOrCreate(
            [
                'id' => $data->id,
            ],
            [
                ... $data->all(),
                'form_id' => $data->form?->id,
                'user_id' => $user->id,
            ],
        );

        $subscriber->tags()->sync($data->tags->toCollection()->pluck('id'));

        // Only fires automations if it's a new subscriber
        if (!$data->id && $data->form) {
            event(new SubscribedToFormEvent($subscriber, $user));
        }

        return $subscriber->load('tags', 'form');
    }
}
```

After creating the subscriber, it checks if `$data` has an ID and a `form_id`. This makes sure that we only dispatch the event if:

- It's a new subscriber
- And has a form

Just as a side note, if you have more complicated `if` statements, it might be a good idea to extract them out to the `SubscriberData` class, for example:

```

class SubscriberData extends Data
{
    public function whenSubscribedViaForm(callable $callback): void
    {
        if (!$data->id && $data->form) {
            $callback();
        }
    }
}

// Usage:
class UpsertSubscriberAction
{
    public static function execute(SubscriberData $data)
    {
        $data->whenSubscribedViaForm(fn () =>
            event(new SubscribedToFormEvent($subscriber, $user));

        );
    }
}

```

It does not bring much benefit to this use case, but it's a very cool technique for these reasons:

- You can encapsulate the whole if statement.
- You have a readable, understandable name instead of technical details.
- It's reusable.

All right, back to automations. So when creating a new subscriber, the action will dispatch a `SubscribedToFormEvent` event:

```

namespace Domain\Automation\Events;

use Domain\Shared\Models\User;
use Domain\Subscriber\Models\Subscriber;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class SubscribedToFormEvent
{
    use Dispatchable, SerializesModels;

    public function __construct(
        public readonly Subscriber $subscriber,
        public readonly User $user,
    ) {}
}

```

It's your regular "placeholder" class for an event. It's worth noting that it lives inside the automation domain. After that, we need to list it in the `EventServiceProvider`:

```

namespace App\Providers;

use Domain\Automation\Events\SubscribedToFormEvent;
use Domain\Automation\Listeners\SubscribedToFormListener;

class EventServiceProvider extends ServiceProvider
{
    protected $listen = [
        SubscribedToFormEvent::class => [
            SubscribedToFormListener::class,
        ],
    ];
}

```

And in the listener, we can trigger the whole process:

```
namespace Domain\Automation\Listeners;

use Domain\Automation\Events\SubscribedToFormEvent;
use Domain\Automation\Jobs\RunAutomationsJob;

class SubscribedToFormListener
{
    public function handle(SubscribedToFormEvent $event)
    {
        RunAutomationsJob::dispatch(
            $event->subscriber,
            $event->user,
            Events::SubscribedToForm,
        );
    }
}
```

One important thing: when the app handles a `SubscribedToFormEvent` it's very important to run only automations with this particular type (yeah, right now, it's the only one, but we can have more use-cases in the future). This is why the listener passes an `Events::SubscribedToForm` value to the job.

I created a job for this because I want it to run in the background. There's no need to block the current request. The job also passes the event type to the action:

```
namespace Domain\Automation\Jobs;

class RunAutomationsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(
        private readonly Subscriber $subscriber,
        private readonly User $user,
        private readonly Events $event,
    ) {}

    public function handle()
    {
        RunAutomationsAction::execute(
            $this->subscriber,
            $this->user,
            $this->event
        );
    }
}
```

And finally, the interesting stuff:

```

namespace Domain\Automation\Actions;

class RunAutomationsAction
{
    public static function execute(
        Subscriber $subscriber,
        User $user,
        Events $event
    ): void {
        $automations = Automation::with('steps')
            ->whereBelongsTo($user)
            ->whereHas('steps', function ($steps) use ($subscriber, $event) {
                $steps
                    ->whereType(AutomationStepType::Event)
                    ->whereName($event)
                    ->where('value->id', $subscriber->form_id);
            })
            ->get();

        foreach ($automations as $automation) {
            $steps = $automation
                ->steps()
                ->whereType(AutomationStepType::Action)->get();

            foreach ($steps as $step) {
                $action = Actions::from($step->name)->createAction();
                $action($subscriber, $step);
            }
        }
    }
}

```

The first part of this function queries every automation where there's a `subscribedToForm` event with the given `form_id`. This line will check a value inside a JSON column:

```
→where('value→id', $subscriber→form_id);
```

Where the value is a JSON like this:

```
{
  "id": 1
}
```

The second part of the function queries every action for the automation and runs them one by one. Let's take a closer look at what's happening here:

```
foreach ($steps as $step) {
    $action = Actions::from($step→name)→createAction();
    $action($subscriber, $step);
}
```

The `Actions` enum is a list of all possible actions with a factory function:

```
namespace Domain\Automation\Enums;

use Domain\Automation\Actions\Steps\AddTagAction;
use Domain\Automation\Actions\Steps\AddToSequenceAction;

enum Actions: string
{
    case AddToSequence = 'addToSequence';
    case AddTag = 'addTag';

    public function createAction()
    {
        return match ($this) {
            self::AddTag => app(AddTagAction::class),
            self::AddToSequence => app(AddToSequenceAction::class),
        };
    }
}
```

As I said earlier, every automation event has an event class, and every automation action has an action class. This factory function will create a new instance from these action classes. I'll show you these classes in a minute. Now let's go back to the `RunAutomationsAction`:

```
$action = Actions::from($step->name)->createAction();  
$action($subscriber, $step);
```

After the action is created, it is being called. To call an object this way, it has to be invocable. If you haven't seen this before, it's just a PHP magic function that looks like this:

```
class AddTagAction  
{  
    public function __invoke(Subscriber $subscriber, AutomationStep $step)  
    {  
        echo 'The action is called';  
    }  
}
```

It can be used this way:

```
$action = new AddTagAction();  
$action($subscriber, $step);
```

So this class doesn't need a function. The class itself acts as a function and can be called. Of course, you can simply use a non-static `execute` method instead:

```
$action = Actions::from($step->name)->createAction();  
$action->execute($subscriber, $step);
```

There's no real difference between the two. It's important to note that a static function would be a poor choice in this case because it cannot be called dynamically. Or I guess there's a way to do this, but it must be very low-level and technical.

So this is how we can instantiate and run the automation actions on the fly. Now let's see the actions:

```
namespace Domain\Automation\Actions\Steps;

use Domain\Automation\Models\AutomationStep;
use Domain\Subscriber\Models\Subscriber;

class AddTagAction
{
    public function __invoke(
        Subscriber $subscriber,
        AutomationStep $step
    ): void {
        $subscriber->tags()->attach($step->value['id']);
    }
}

class AddToSequenceAction
{
    public function __invoke(
        Subscriber $subscriber,
        AutomationStep $step
    ): void {
        $subscriber->sequences()->attach($step->value['id']);
    }
}
```

It couldn't be easier. At the end of the day, running automations require some classes like events, listeners, and jobs, but it was really simple at its core.

Conclusion

I know that it was a lot to take in, so I recommend you to take a break and come back once again to check out the main ideas in the book. In my opinion, these are the most important takeaways:

- Using actions is a great way to write reusable classes for a single use case. Each action takes care of one user story.
- Using Data Transfer Objects can help you eliminate random, unstructured arrays. No more functions with an `array $data` argument.
- Writing custom query builders help you to get rid of models with 1000 lines of code. In my experience, a model can grow 200+ lines without having a single query in it. That's more than enough for one class.
- Use value objects whenever possible. Some typical examples would be:
 - Percent
 - Money
 - E-mail address
 - Address
 - Financial metrics
 - Unit of measure

However, domain-driven design comes at a cost:

- You have a lot of classes. Just consider a simple feature such as creating and updating a subscriber. We wrote the following classes:
 - SubscriberData
 - FormData
 - TagData
 - UpsertSubscriberAction
 - SubscriberController
 - UpsertSubscriberViewModel
- You can see there are a lot of classes compared to having a model and a controller. After a while, it can be pretty annoying to maintain this many classes. However, I cannot name a real disadvantage outside of being annoying. In general, I prefer having 50 small classes over ten huge ones.
- Using actions and view models makes it easy to duplicate code. You have smaller queries all over the place, so it's harder to notice if something is duplicated. However, if you write your queries inside models and have 1000 lines of code in every model, is it better to avoid duplication? I know you have reusable methods, but can you spot the 5-liner you need in the given situation? Or do you end up rewriting it with a different name? Avoiding code duplication in a large project is hard, no matter what.

Thank You

Thank you for reading this book! I hope you got a lot of value from it and learned a few things about domain-driven design and Laravel.

My name is Martin Joo, and I published several other books about Laravel:

- [Test-Driven APIs with Laravel and Pest](#)
- [Microservices with Laravel](#)
- [Laravel Eloquent Recipes](#)
- [Proper API Design with Laravel](#)

I'm also publishing blog articles and tweets regularly. You can find every work of mine on [Twitter](#).